



上海理工大学机器智能研究院

INSTITUTE OF MACHINE INTELLIGENT, UNIVERSITY OF SHANGHAI FOR SCIENCE AND TECHNOLOGY

强化学习基础算法



目录

CONTENTS

1

2

3

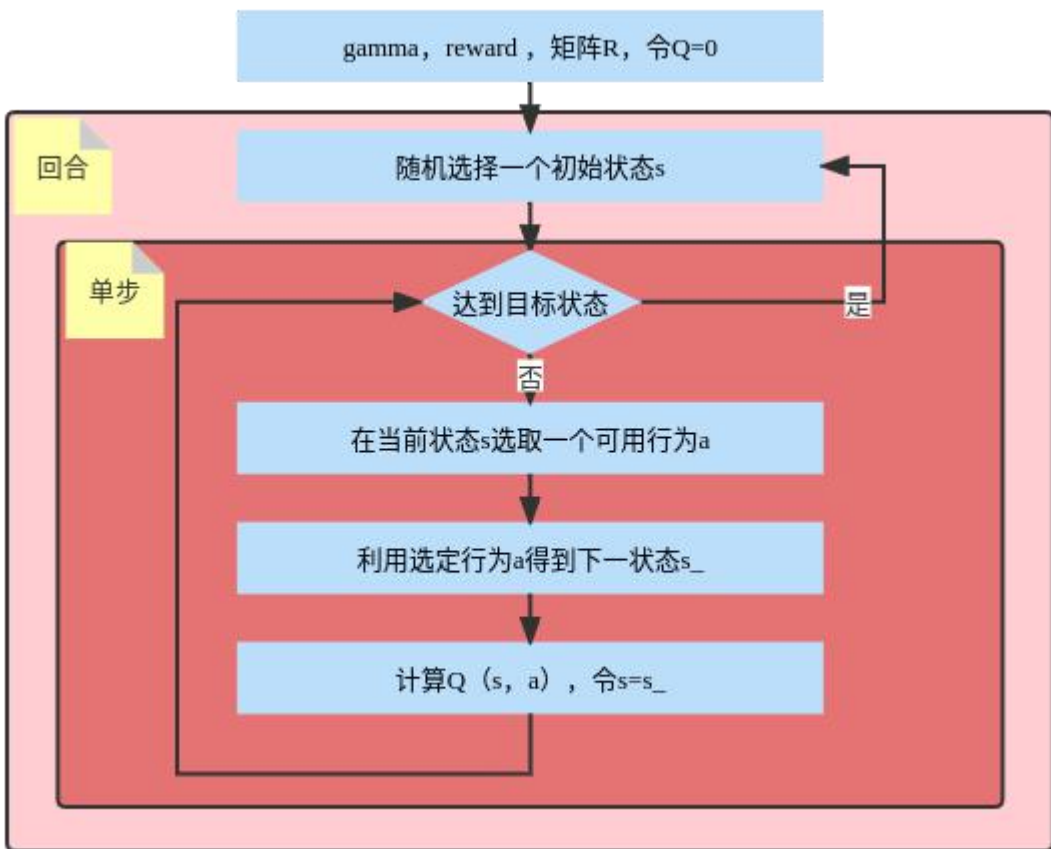
4

5



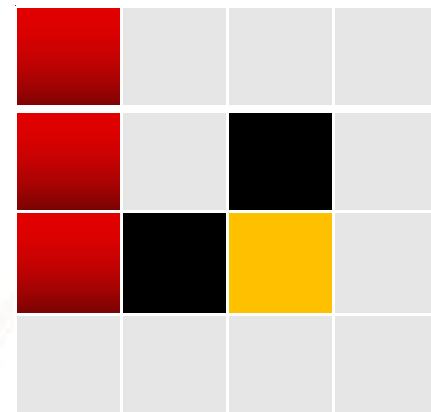
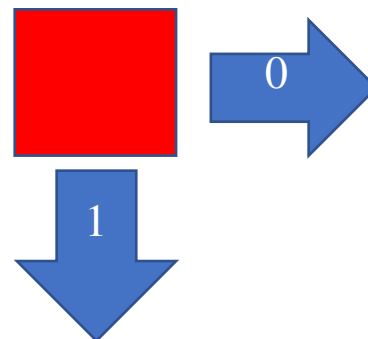
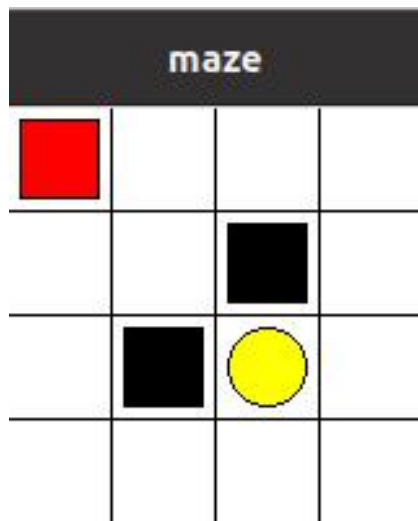


1.Q_learning



Q-table的更新：

$$Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$



x	y	0	1	2	3
[5.0, 5.0, 35.0, 35.0]		0.0	0.0	0.00	0.0
[5.0, 45.0, 35.0, 75.0]		0.0	0.0	0.00	0.0
[5.0, 85.0, 35.0, 115.0]		0.0	0.0	-0.01	0.0
terminal		0.0	0.0	0.00	0.0

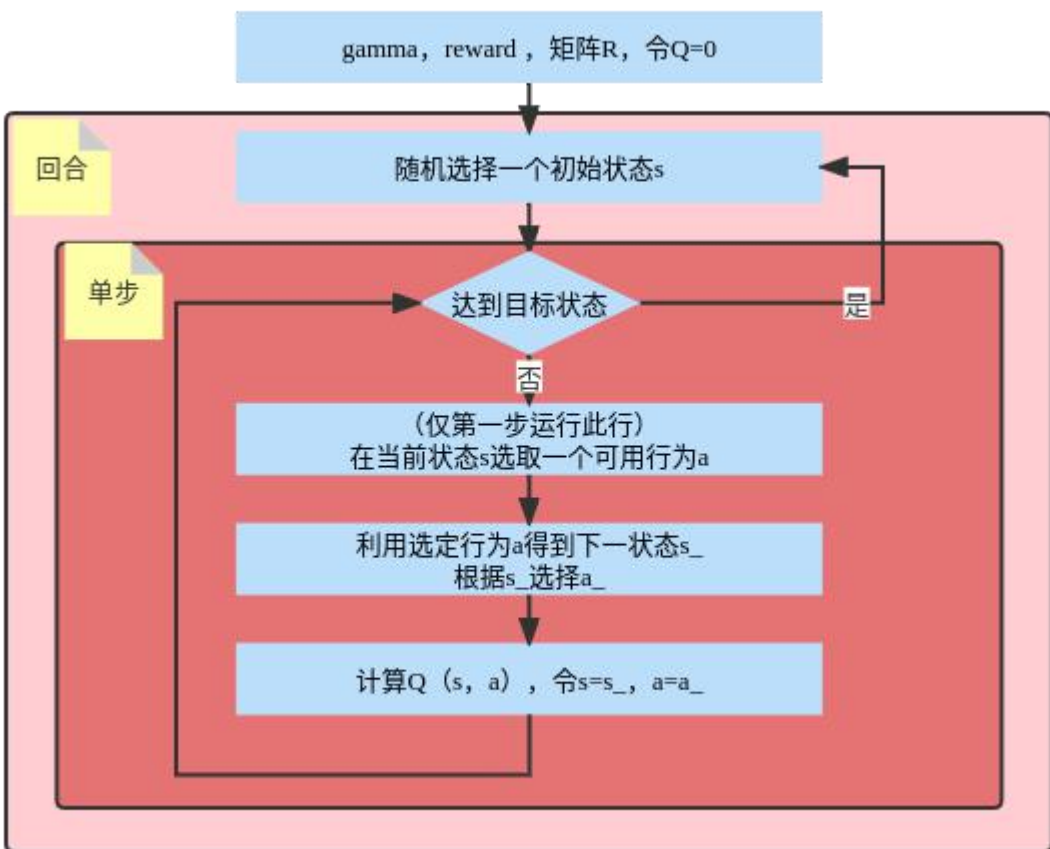
	0	1	2	3
[5.0, 5.0, 35.0, 35.0]	0.000000	1.625152e-11	0.000000	0.000000
[45.0, 5.0, 75.0, 35.0]	0.000000	0.000000e+00	0.000000	0.000000
[5.0, 45.0, 35.0, 75.0]	0.000000	3.583091e-09	0.000000	0.000000
[45.0, 45.0, 75.0, 75.0]	0.000000	-2.970100e-02	-0.010000	0.000000
[85.0, 5.0, 115.0, 35.0]	0.000000	-3.940399e-02	0.000000	0.000000
terminal	0.000000	0.000000e+00	0.000000	0.000000
[125.0, 5.0, 155.0, 35.0]	0.000000	8.100000e-07	0.000000	0.000000
[125.0, 45.0, 155.0, 75.0]	0.000000	2.682000e-04	0.000000	-0.019900
[5.0, 85.0, 35.0, 115.0]	0.000000	4.941734e-07	-0.039404	0.000000
[5.0, 125.0, 35.0, 155.0]	0.000000	0.000000e+00	0.000044	0.000000
[45.0, 125.0, 75.0, 155.0]	-0.010000	0.000000e+00	0.002421	0.000000
[125.0, 85.0, 155.0, 115.0]	0.000000	0.000000e+00	0.000000	0.029701
[85.0, 125.0, 115.0, 155.0]	0.077255	0.000000e+00	0.000000	0.000000
[125.0, 125.0, 155.0, 155.0]	0.000000	0.000000e+00	0.000000	0.000000



2.Sarsa

Sarsa的更新：

$$Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma Q(s_-, a_-) - Q(s,a)]$$

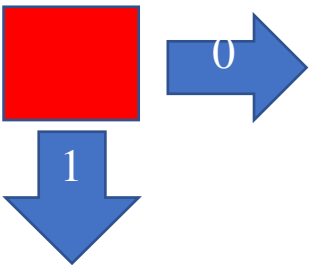
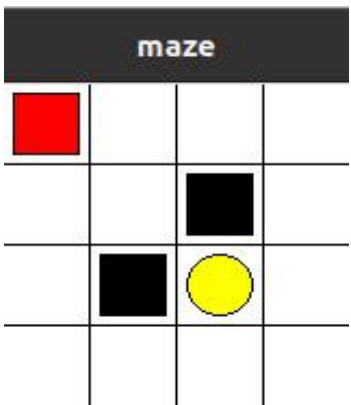


Q-Learning算法，先假设下一步选取最大奖赏的动作，更新值函数。但是下一步不一定选取最大奖励的动作

Sarsa算法先选取下一状态执行的动作，然后根据所执行的动作，更新值函数。



Q-learning与Sarsa



动作（action）依据q表选取，正负励（reward）相邻。

q-learning更勇敢，sarsa更谨慎

q-learning	0	1	2	3	sarsa	0	1	2	3
[5.0, 5.0, 35.0, 35.0]	0.000000e+00	0.000003	0.000000	0.000000	[5.0, 5.0, 35.0, 35.0]	-2.730493e-06	-0.000012	-0.000010	-2.678999e-06
[5.0, 45.0, 35.0, 75.0]	0.000000e+00	0.000052	0.000000	0.000000	[45.0, 5.0, 75.0, 35.0]	-2.167740e-06	-0.000076	-0.000155	-1.963583e-06
[5.0, 85.0, 35.0, 115.0]	2.635484e-07	0.000070	-0.019900	0.000000	[45.0, 45.0, 75.0, 75.0]	-2.258759e-06	-0.019900	-0.019900	-2.332490e-06
terminal	0.000000e+00	0.000000	0.000000	0.000000	[5.0, 45.0, 35.0, 75.0]	-2.109227e-06	-0.000228	-0.000078	-3.629596e-06
[45.0, 5.0, 75.0, 35.0]	0.000000e+00	0.000000	0.000000	0.000000	[5.0, 85.0, 35.0, 115.0]	0.000000e+00	0.000049	-0.029701	0.000000e+00
[45.0, 45.0, 75.0, 75.0]	0.000000e+00	-0.010000	-0.010000	0.000000	terminal	0.000000e+00	0.000000	0.000000	0.000000e+00
[5.0, 125.0, 35.0, 155.0]	3.004258e-06	0.000081	0.007605	0.000052	[85.0, 5.0, 115.0, 35.0]	0.000000e+00	-0.029701	0.000008	-5.107615e-15
[45.0, 125.0, 75.0, 155.0]	-1.000000e-02	0.000000	0.061529	0.000000	[125.0, 5.0, 155.0, 35.0]	3.910765e-07	0.000294	0.000000	-9.000000e-05
[85.0, 125.0, 115.0, 155.0]	3.443408e-01	0.001414	0.000026	0.000348	[125.0, 45.0, 155.0, 75.0]	1.610423e-06	0.008659	0.000042	-1.000000e-02
[85.0, 5.0, 115.0, 35.0]	0.000000e+00	-0.019900	0.000000	0.000000	[125.0, 85.0, 155.0, 115.0]	7.859820e-06	0.000000	0.000000	1.399416e-01
[125.0, 5.0, 155.0, 35.0]	0.000000e+00	0.000000	0.000000	0.000000	[125.0, 125.0, 155.0, 155.0]	0.000000e+00	0.000000	0.000000	0.000000e+00
[125.0, 45.0, 155.0, 75.0]	0.000000e+00	0.000000	0.000000	0.000000	[85.0, 125.0, 115.0, 155.0]	2.063857e-01	0.000000	0.000000	2.389662e-06
[125.0, 85.0, 155.0, 115.0]	0.000000e+00	0.000002	0.000000	0.000000	[5.0, 125.0, 35.0, 155.0]	2.685969e-08	0.000007	0.001235	0.000000e+00
[125.0, 125.0, 155.0, 155.0]	0.000000e+00	0.000000	0.000000	0.004698	[45.0, 125.0, 75.0, 155.0]	0.000000e+00	0.000000	0.019807	0.000000e+00



3. Sarsa-lambda

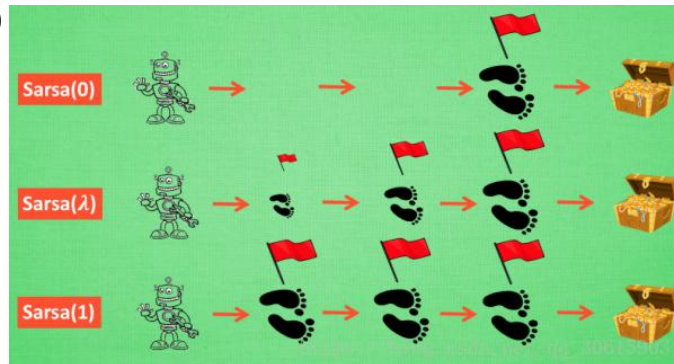
Sarsa-lambda的更新：

```
delta ← R + gamma * Q(s_, a_) - Q(s, a)
E(s, a) ← E(s, a) + 1
For all s ∈ S, a ∈ A(s)
    Q(s, a) ← Q(s, a) + alpha * delta * E(s, a)
    E(s, a) ← gamma * lambda * E(s, a)
```

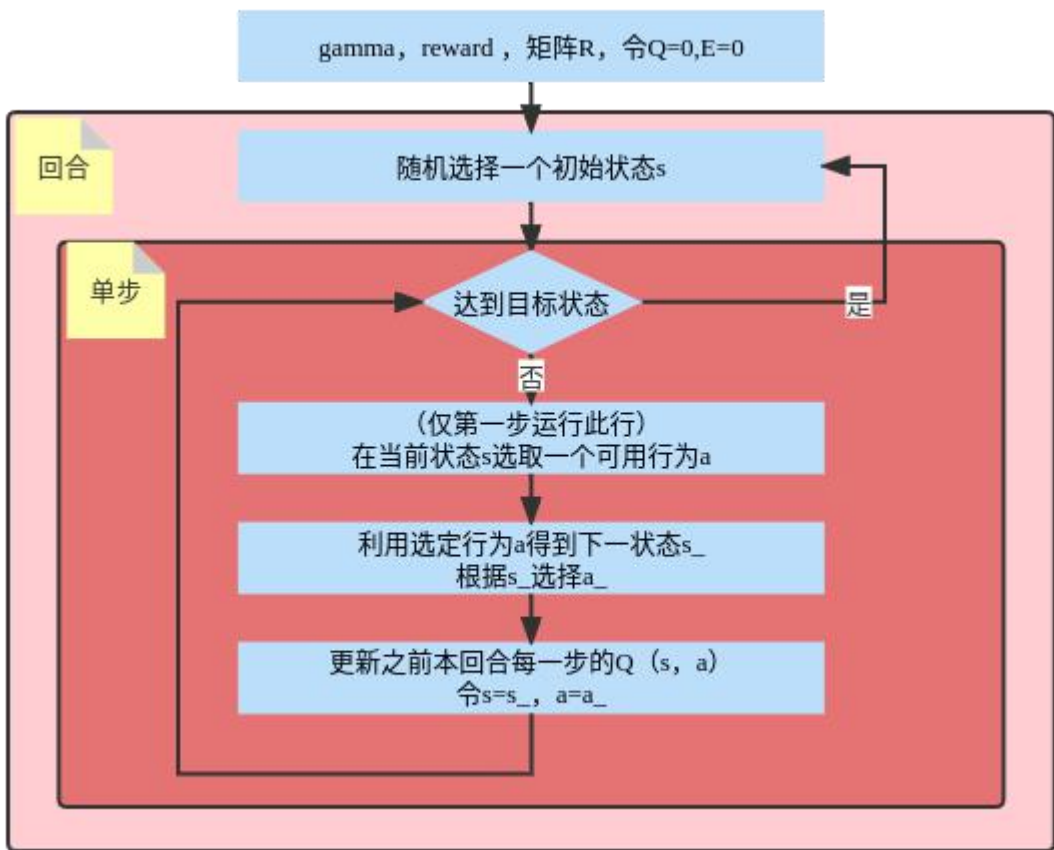


若取lambda=0

```
delta ← R + gamma * Q(s_, a_) - Q(s, a)
E(s, a) ← E(s, a) + 1
For all s ∈ S, a ∈ A(s)
    Q(s, a) ← Q(s, a) + alpha * delta * E(s, a)
    E(s, a) ← gamma * 1ambda (值为0) * E(s, a)
```

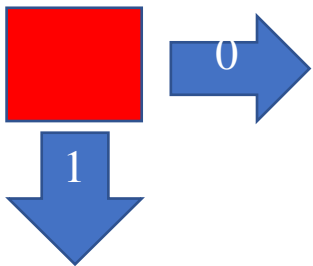
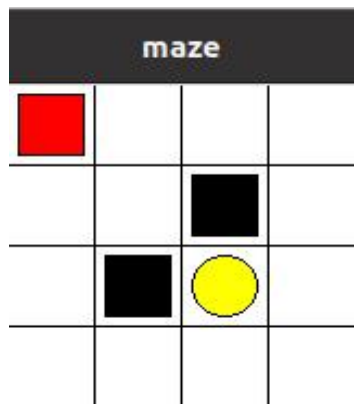


Sarsa-lambda是Sarsa 的一种提速方法

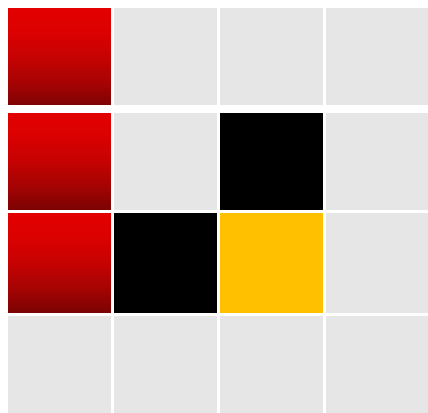




Sarsa与Sarsa-lambda



每次采取动作 (action) 获得reward后，Sarsa只对前一步 $Q(s,a)$ 进行更新，Sarsa(lambda) 则会对获得reward之前的步进行更新。



运行一个回合的q表

Sarsa

	0	1	2	3
[5.0, 5.0, 35.0, 35.0]	0.0	0.0	0.00	0.0
[5.0, 45.0, 35.0, 75.0]	0.0	0.0	0.00	0.0
[5.0, 85.0, 35.0, 115.0]	0.0	0.0	-0.01	0.0
terminal	0.0	0.0	0.00	0.0

Sarsa-lambda

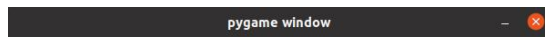
	0	1	2	3
[5.0, 5.0, 35.0, 35.0]	0.0	-0.005314	0.0000	0.0
[5.0, 45.0, 35.0, 75.0]	0.0	0.000000	-0.0081	0.0
[45.0, 45.0, 75.0, 75.0]	0.0	0.000000	-0.0100	0.0
terminal	0.0	0.000000	0.0000	0.0



2022.10.27



关于gym的 Cart Pole (推杆)

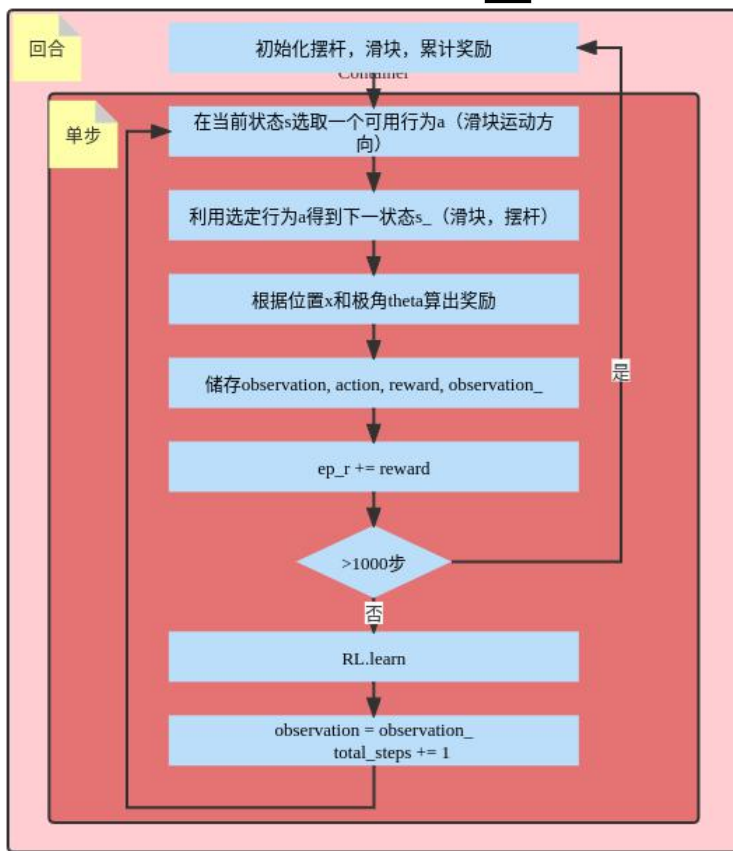


智能体 (Agent) : 黑色滑块

动作 (Action) : 向左1, 向右0

观测值 (observation) : 位置 (中心点左右各4.8之间), 速度, 极角
(竖直左右极角各0.42之间), 极角速度

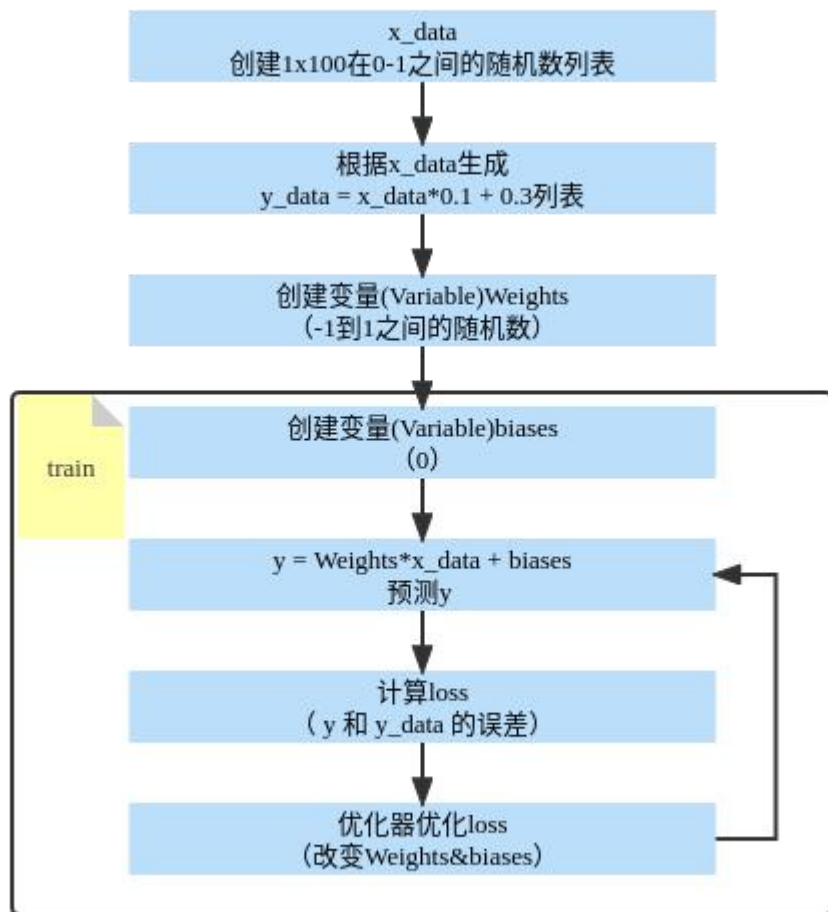
奖励 (reward) : 依据极角和位置而定



RL.learn



神经网络



第 0 步	优化所得	W(真值0.1)= [0.20745501]	b(真值0.3)= [0.3279074]	误差为 0.0074165594
第 20 步	优化所得	W(真值0.1)= [0.11830144]	b(真值0.3)= [0.29043722]	误差为 2.8485643e-05
第 40 步	优化所得	W(真值0.1)= [0.10459109]	b(真值0.3)= [0.29760107]	误差为 1.792624e-06
第 60 步	优化所得	W(真值0.1)= [0.10115174]	b(真值0.3)= [0.2993982]	误差为 1.12812955e-07
第 80 步	优化所得	W(真值0.1)= [0.10028892]	b(真值0.3)= [0.29984903]	误差为 7.0992456e-09
第 100 步	优化所得	W(真值0.1)= [0.1000725]	b(真值0.3)= [0.29996213]	误差为 4.4689394e-10
第 120 步	优化所得	W(真值0.1)= [0.10001817]	b(真值0.3)= [0.2999905]	误差为 2.8064173e-11
第 140 步	优化所得	W(真值0.1)= [0.10000458]	b(真值0.3)= [0.29999763]	误差为 1.7717117e-12
第 160 步	优化所得	W(真值0.1)= [0.10000116]	b(真值0.3)= [0.29999942]	误差为 1.1310952e-13
第 180 步	优化所得	W(真值0.1)= [0.10000028]	b(真值0.3)= [0.29999986]	误差为 6.714629e-15
第 200 步	优化所得	W(真值0.1)= [0.10000011]	b(真值0.3)= [0.29999995]	误差为 1.0302869e-15



2022.11.5





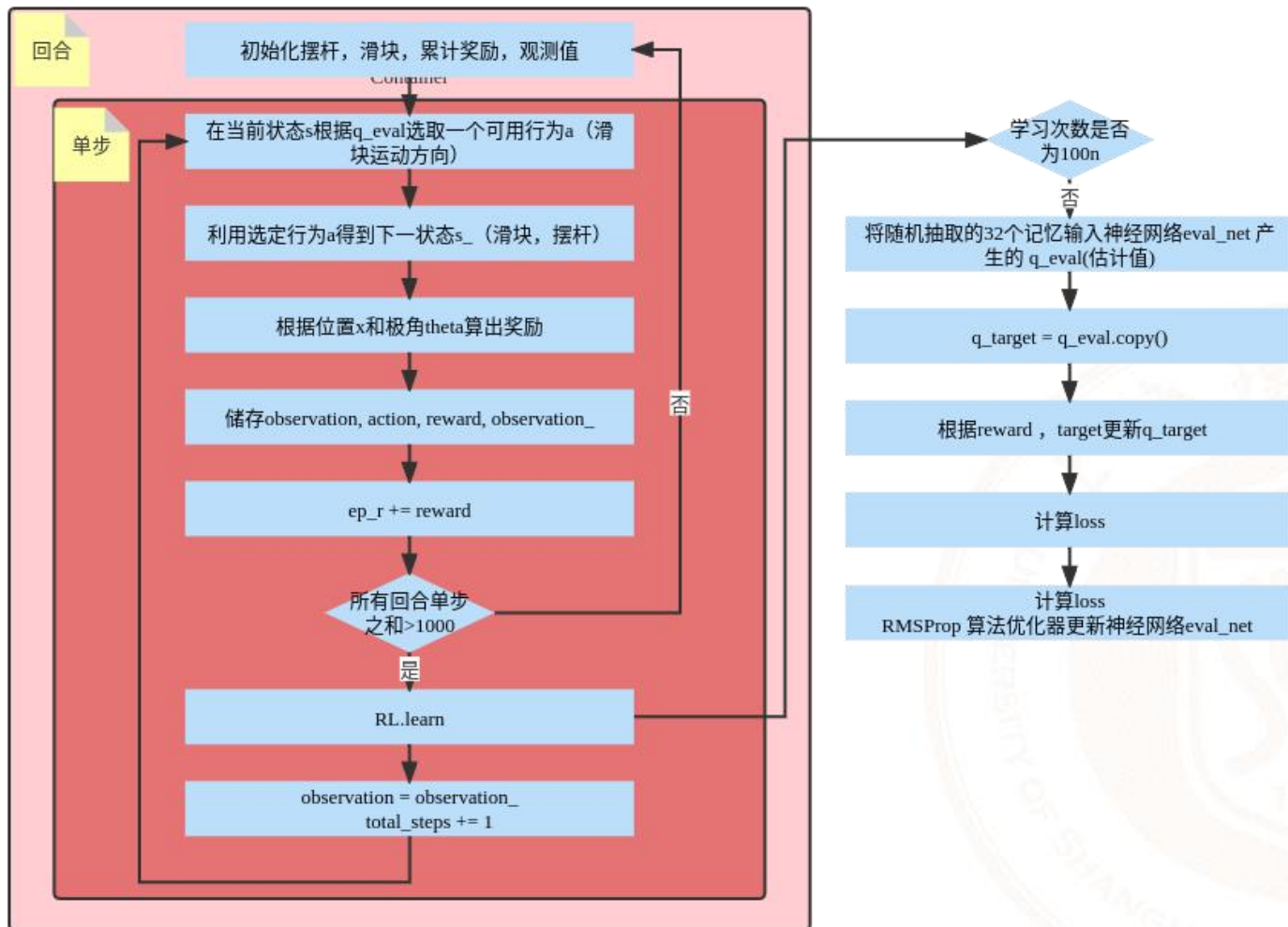
维度灾难!

Q_learning: Q值在试验的同时反复更新。

函数(近似)表示
 $Q(s, a) \approx f(s, a, w)$
 w 是函数 f 的参数

用神经网络来表示这个函数 f 。

DQN

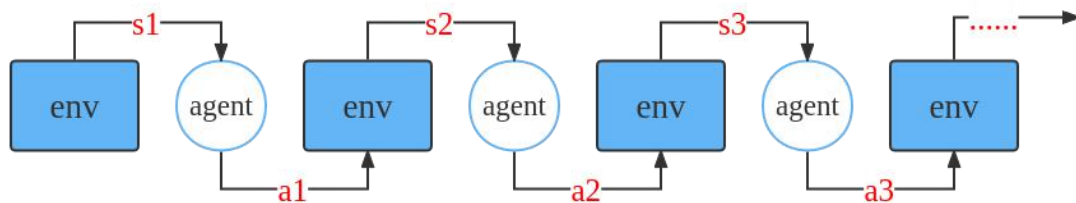




2022.11.8



Policy Gradients



$$\theta \leftarrow \theta + \eta \nabla \bar{R}_\theta$$

$$\nabla \bar{R}_\theta = \frac{1}{N} \sum_{n=1}^N \sum_{t=1}^{T_n} R(\tau^n) \nabla \log p_\theta(a_t^n | s_t^n)$$

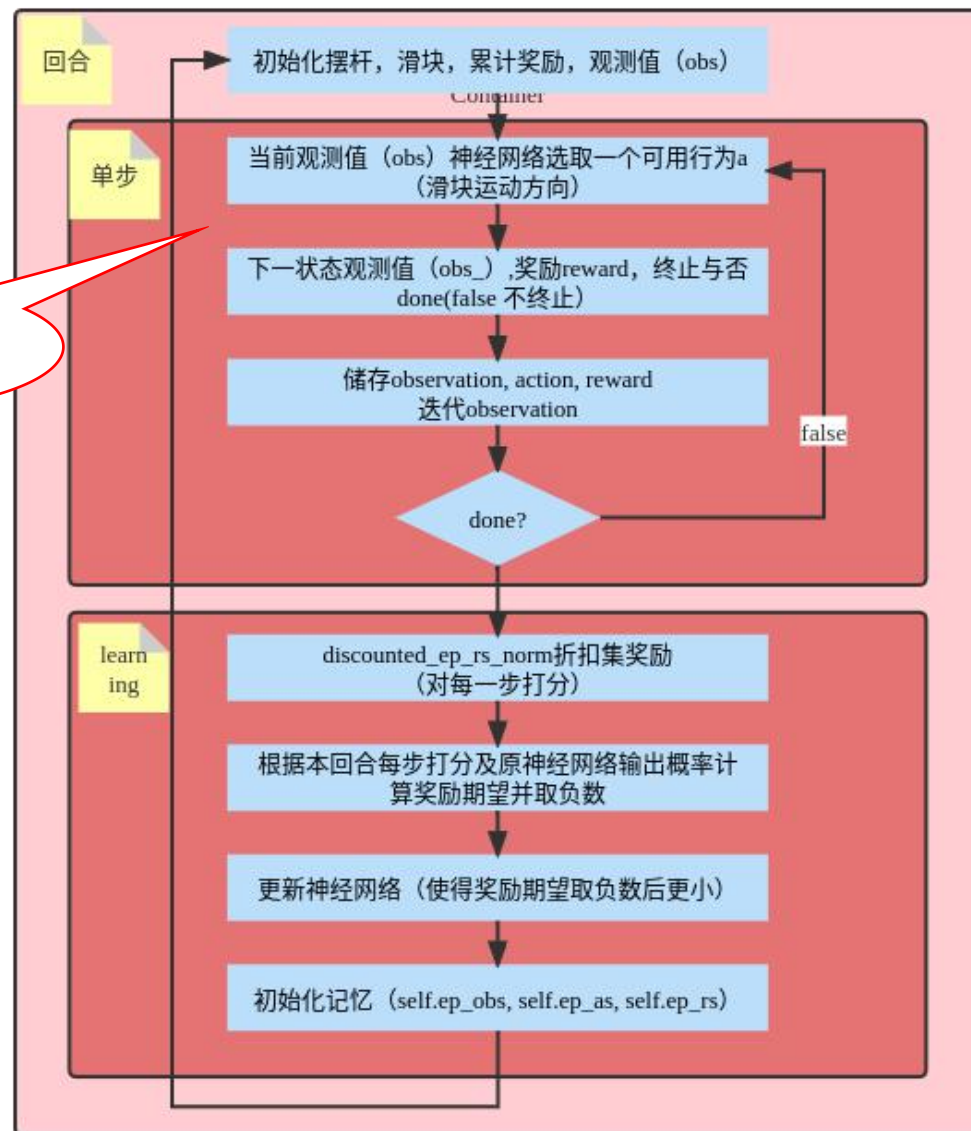
不同于DQN的Q值，
此处神经网络输出的是概率值

输入：N个蒙特卡罗完整序列,训练步长 α

输出：策略函数的参数 θ

- for 每个蒙特卡罗序列:
 - 用蒙特卡罗法计算序列每个时间位置 t 的状态价值 v_t
 - 对序列每个时间位置 t ，使用梯度上升法，更新策略函数的参数 θ ：
$$\theta = \theta + \alpha \nabla \log \pi_\theta(s_t, a_t) v_t$$
- 返回策略函数的参数 θ

回合更新



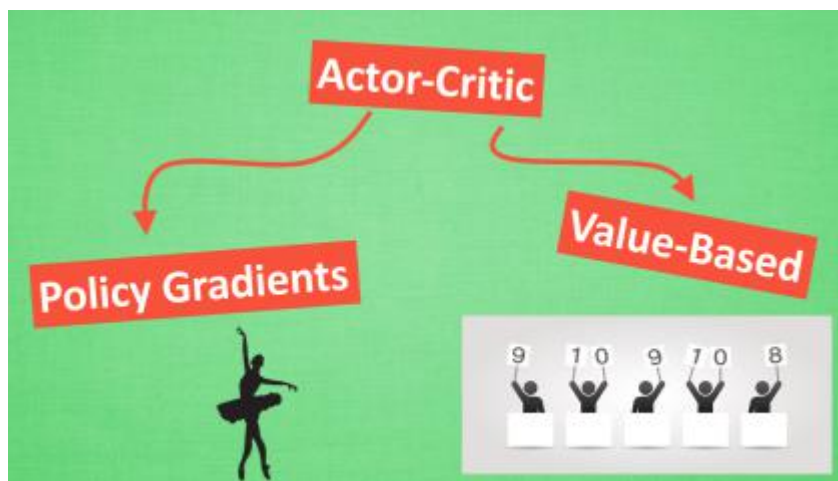


2022.11.12

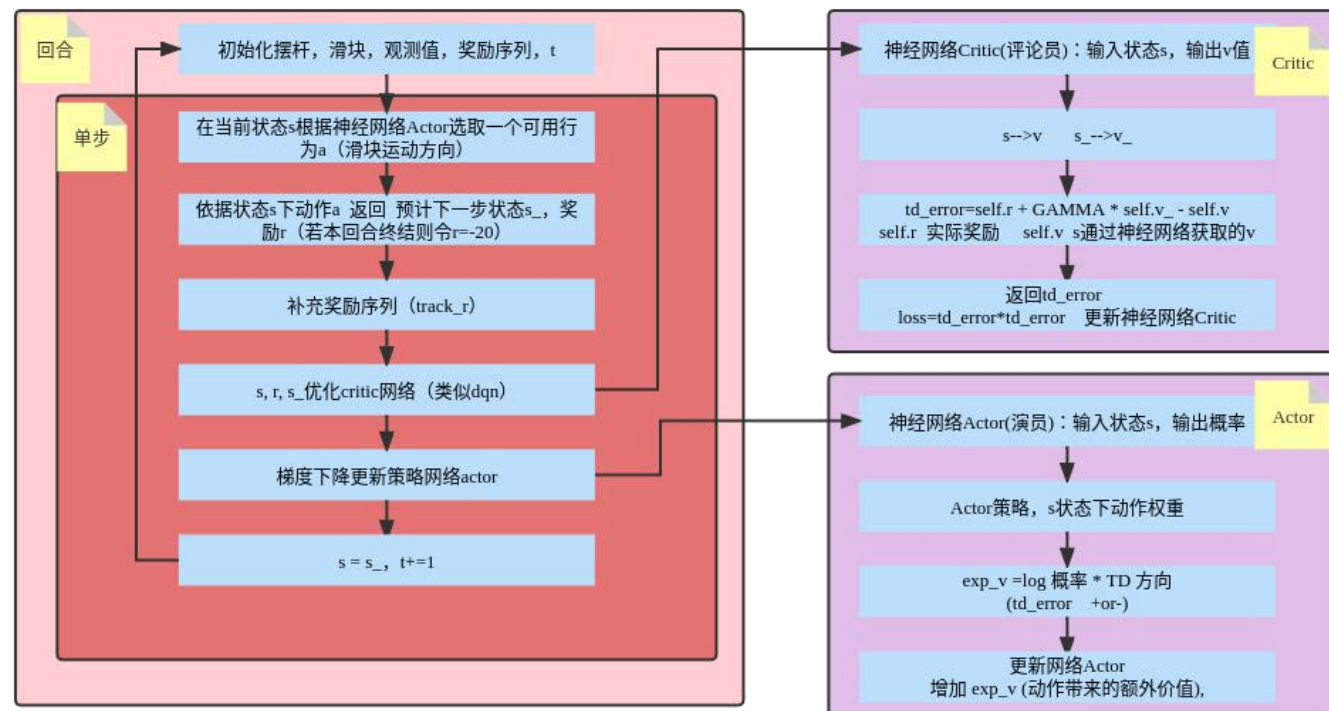


Actor-Critic 演员评论家

Actor (演员) 是策略函数 $\pi_{\theta}(s)$
Critic (评论家) 是值函数 $V^{\pi}(s)$



策略函数Actor，类似于策略梯度算法，但是没有Critic的（使用蒙特卡罗法来计算每一步的价值部分替代Critic的功能）。



拟合值函数V区别于q表，可以近似做以下理解

$$Q^{\pi}(s_t, a_t) \approx r(s_t, a_t) + V^{\pi}(s_{t+1})$$



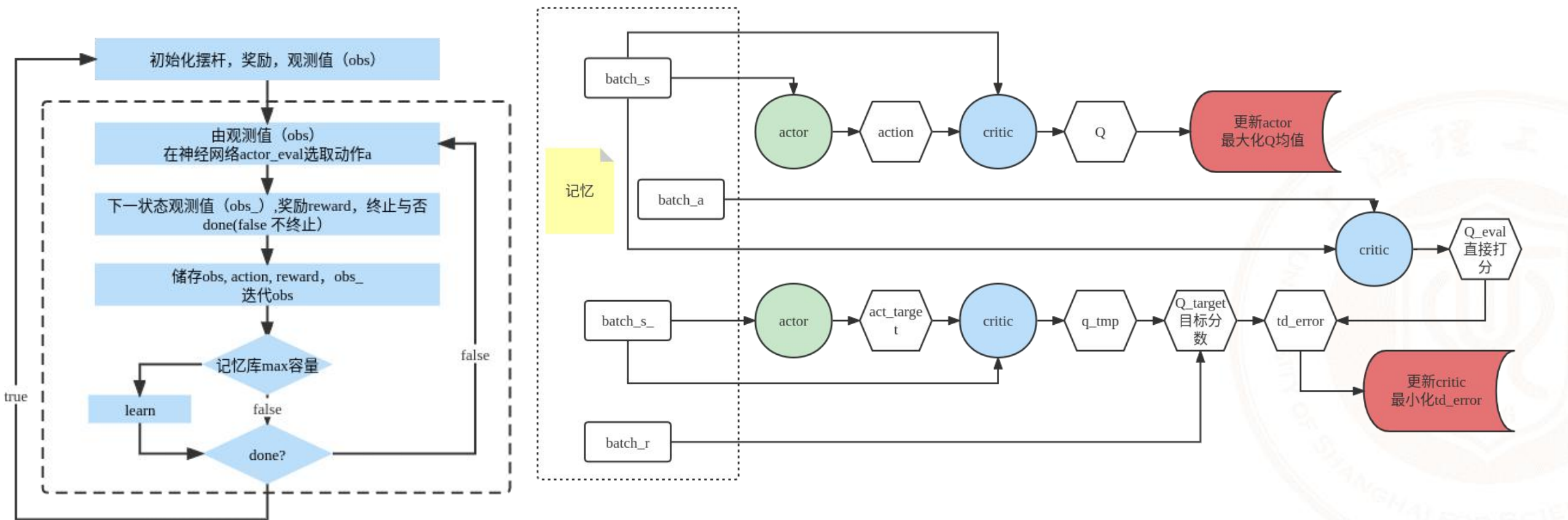
2022.11.22



DDPG(Deep Deterministic Policy Gradient)深度确定性策略梯度算法

CriticNet
输入观测值&扭矩
输出价值

ActorNet
输入为观测值
输出扭矩（动作）





2022.11.26

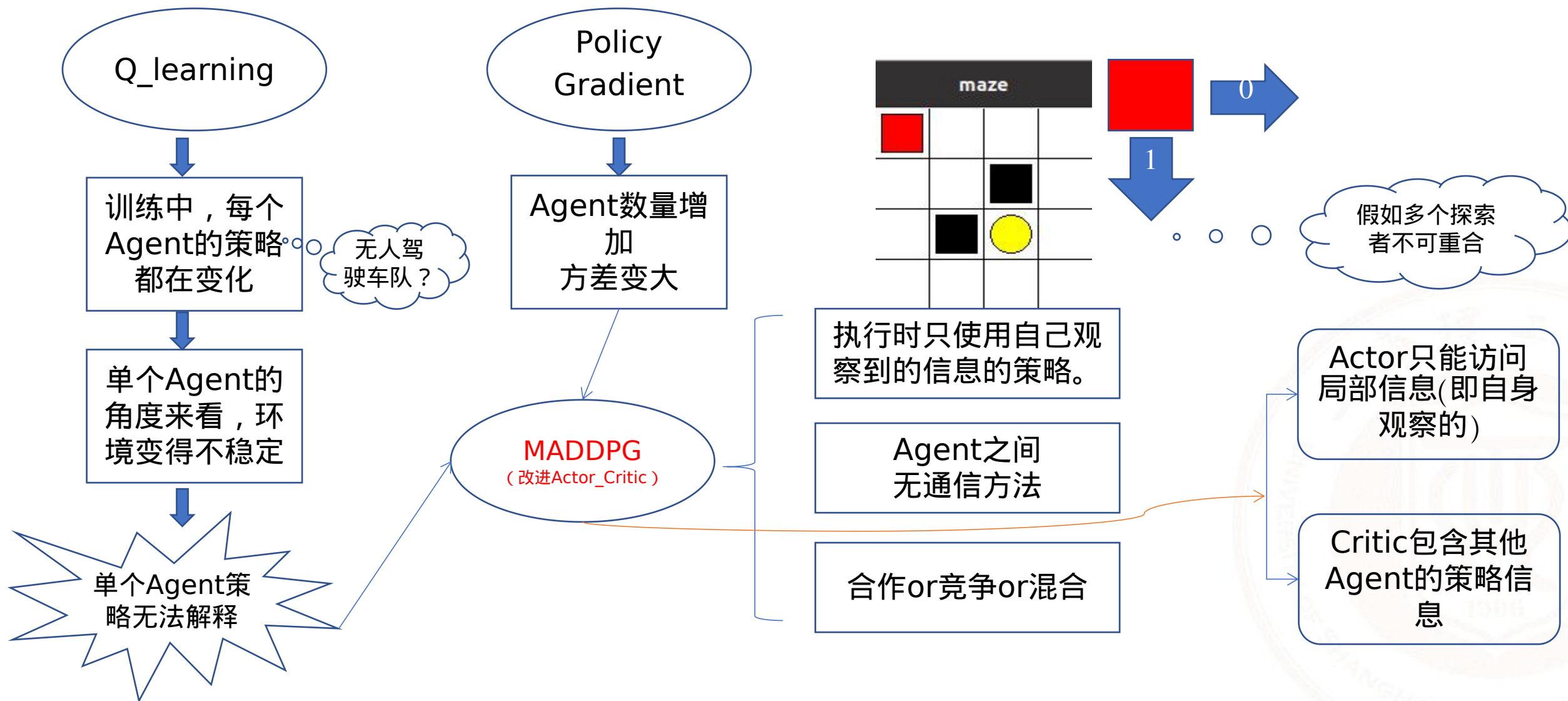
配色
流程名字
保存模型
paper with code

RLN



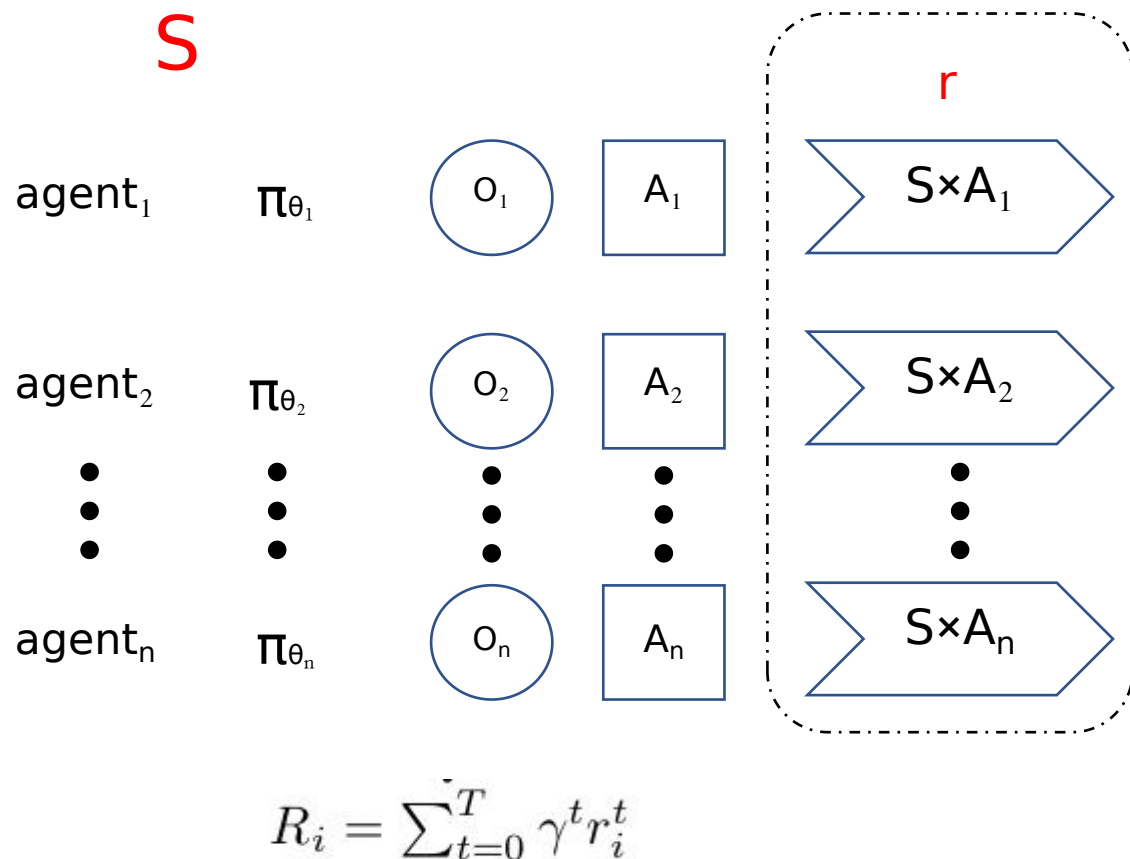
Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments

混合合作竞争环境中多智能体Actor-Critic算法



多智能体马尔可夫博弈

$$S \times A_1 \times \dots \times A_N \rightarrow S^2$$



agent_i的目标是最大化其自身的总预期收益 R_i ，其中 γ 是折扣系数， T 是时间步骤。

Q_learning&DQN

动作价值公式： $Q^\pi(s,a)=E[R|s^t=s,a^t=a]$

递归公式：

$$Q^\pi(s,a)=E_{s'}[r(s,a)+\gamma E_{a' \sim \pi}[Q^\pi(s',a')]]$$



DQN优化方法

$$\text{loss}(\theta)=E_{s,a,r,s'}[(Q^*(s,a|\theta) - y)^2]$$

$$y = r + \gamma \max_{a'} \bar{Q}^*(s', a')$$

目标Q函数

$$P(s'|s, a, \pi_1, \dots, \pi_N) \neq P(s'|s, a, \pi'_1, \dots, \pi'_N) \text{ when any } \pi_i \neq \pi'_i.$$

多智能体策略集合不同时，对于单智能体违反了Q_learning收敛所需的马尔可夫假设。

策略梯度 (PG)

$$J(\theta) = \mathbb{E}_{s \sim p^\pi, a \sim \pi_\theta} [R]$$

策略 π (参数 θ) 某一回合期望奖励为 $J(\theta)$

根据定义的Q函数, 策略的梯度推导出

$$\nabla_\theta J(\theta) = \mathbb{E}_{s \sim p^\pi, a \sim \pi_\theta} [\nabla_\theta \log \pi_\theta(a|s) Q^\pi(s, a)]$$

策略梯度定理与不同 Q^π 估计方法产生了几种实用的算法

简单地使用样本返回

$$R^t = \sum_{i=t}^T \gamma^{i-t} r_i$$

REINFORCE
算法

取决于多
Agent

r
 $S \times A_n$

agent_i

仅以Agent自己的Action为条件的奖励表现出更大的可变性, 增加了其梯度的方差。

震荡
局部最优

学习真实作用值函数
 $Q^\pi(s, a)$ 的近似值

actor-critic



2022.11.29

RLN

多智能体具体应用以及场景

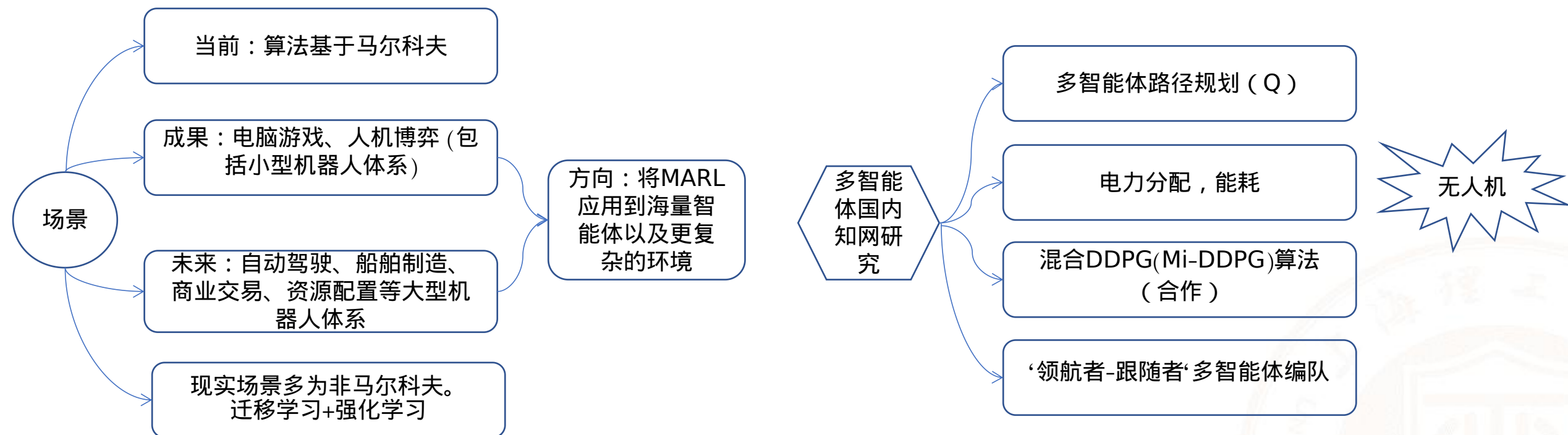
具体模型场景

gym模型能否改进为多智能体

多智能体国内知网研究



多智能体具体应用以及场景

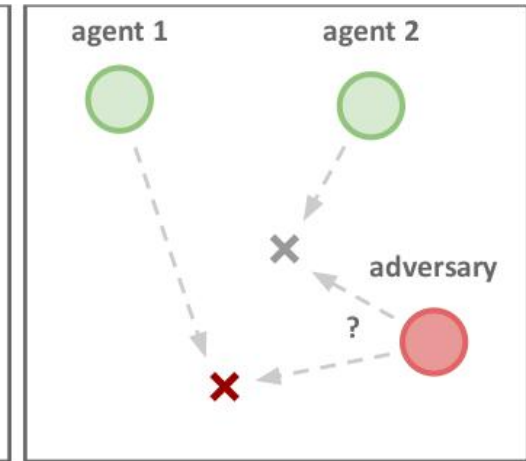
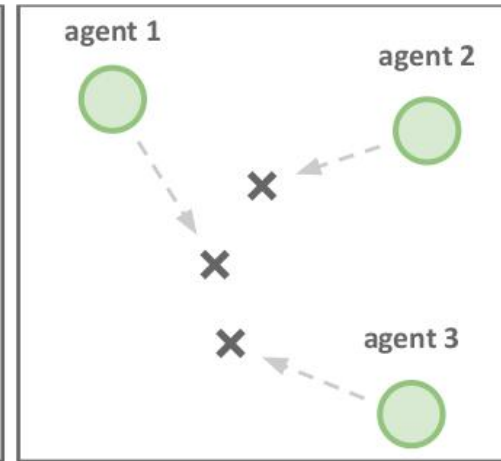
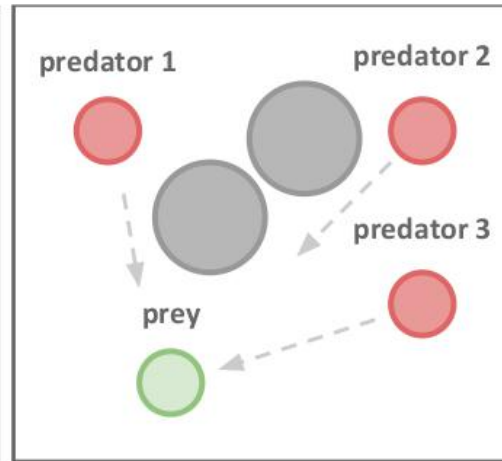
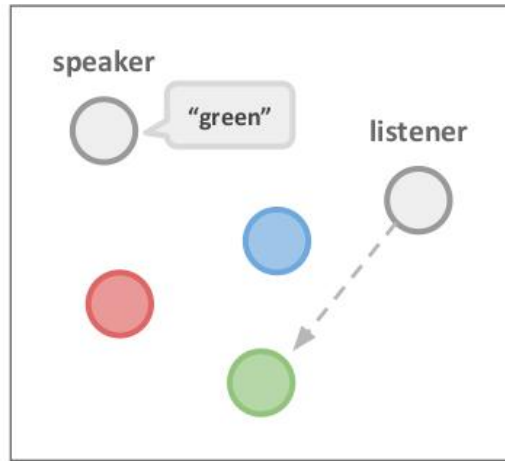




2022.12.3

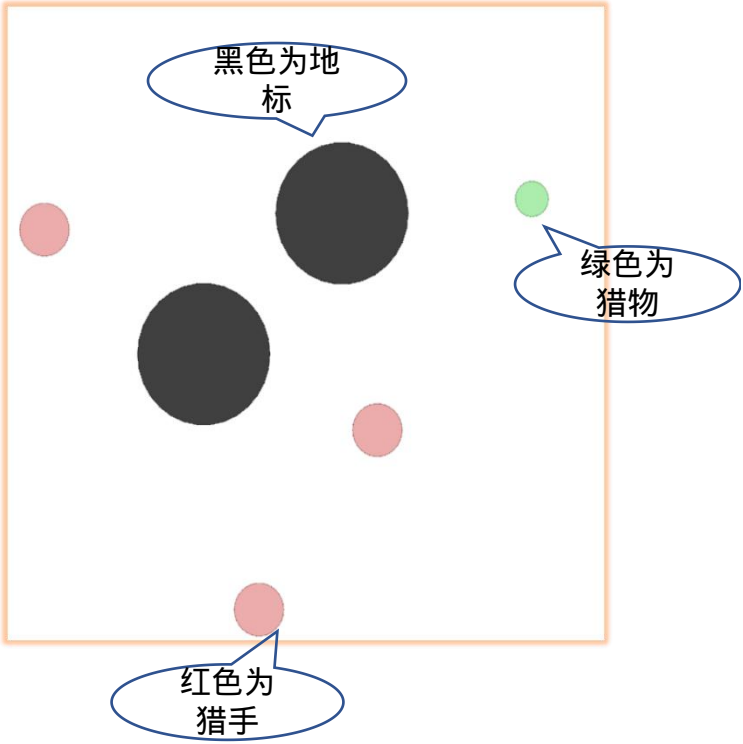


MADDPG论文场景



Cooperative communication (合作沟通)	Predator-Prey (猎手-猎物)	Cooperative navigation (协同导航)	Physical deception (物理欺骗)
speaker : 观察正确的地标颜色	Predator : 抓到猎物	agent通过物理行动进行合作并编队	
listener : 依据speaker发布信息导航到特定颜色的地标	Prey : 逃脱猎手	逃脱猎手	

MADDPG论文场景



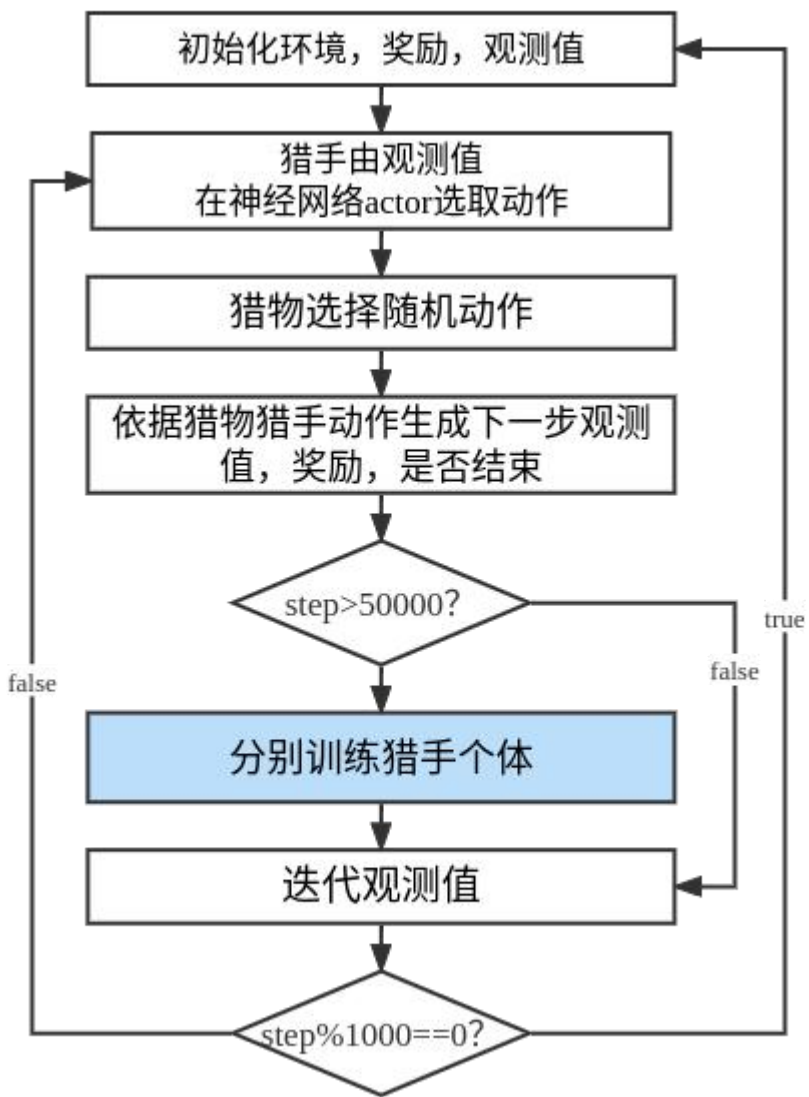
物体	颜色	数量	速度	目的	奖励机制
Predator (猎手)	红	3	慢	碰到猎物	任一猎手碰到猎物，所有猎手均+10分
Prey (猎物)	绿	1	快	逃脱猎手	被任意猎手碰到-10分
Landmarks (地标)	黑	2	固定	路障	无





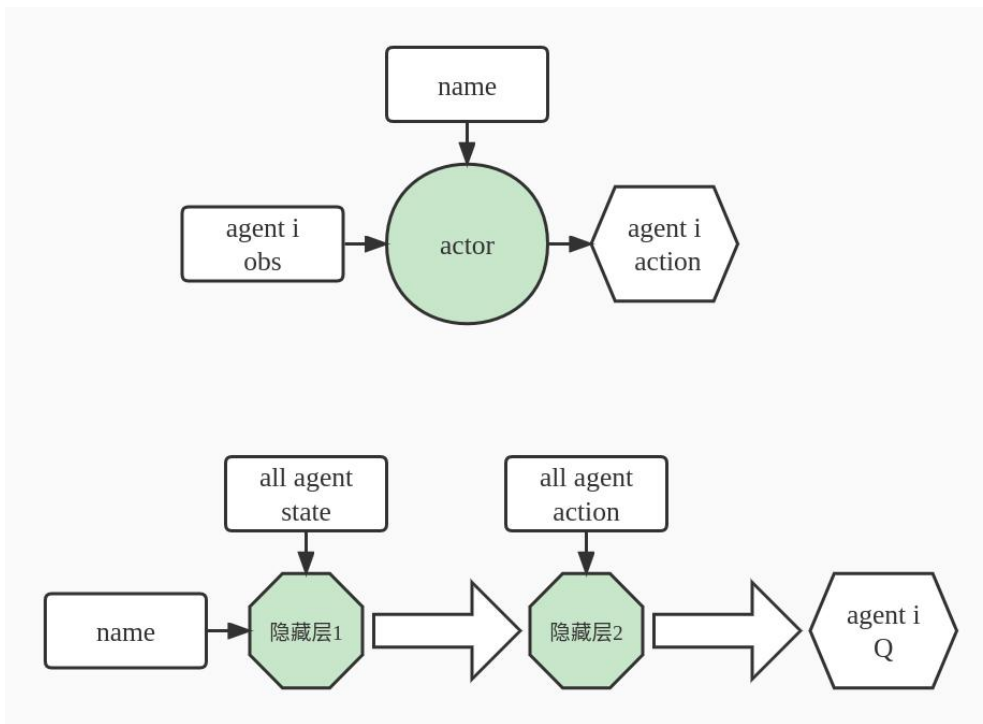
2022.12.6

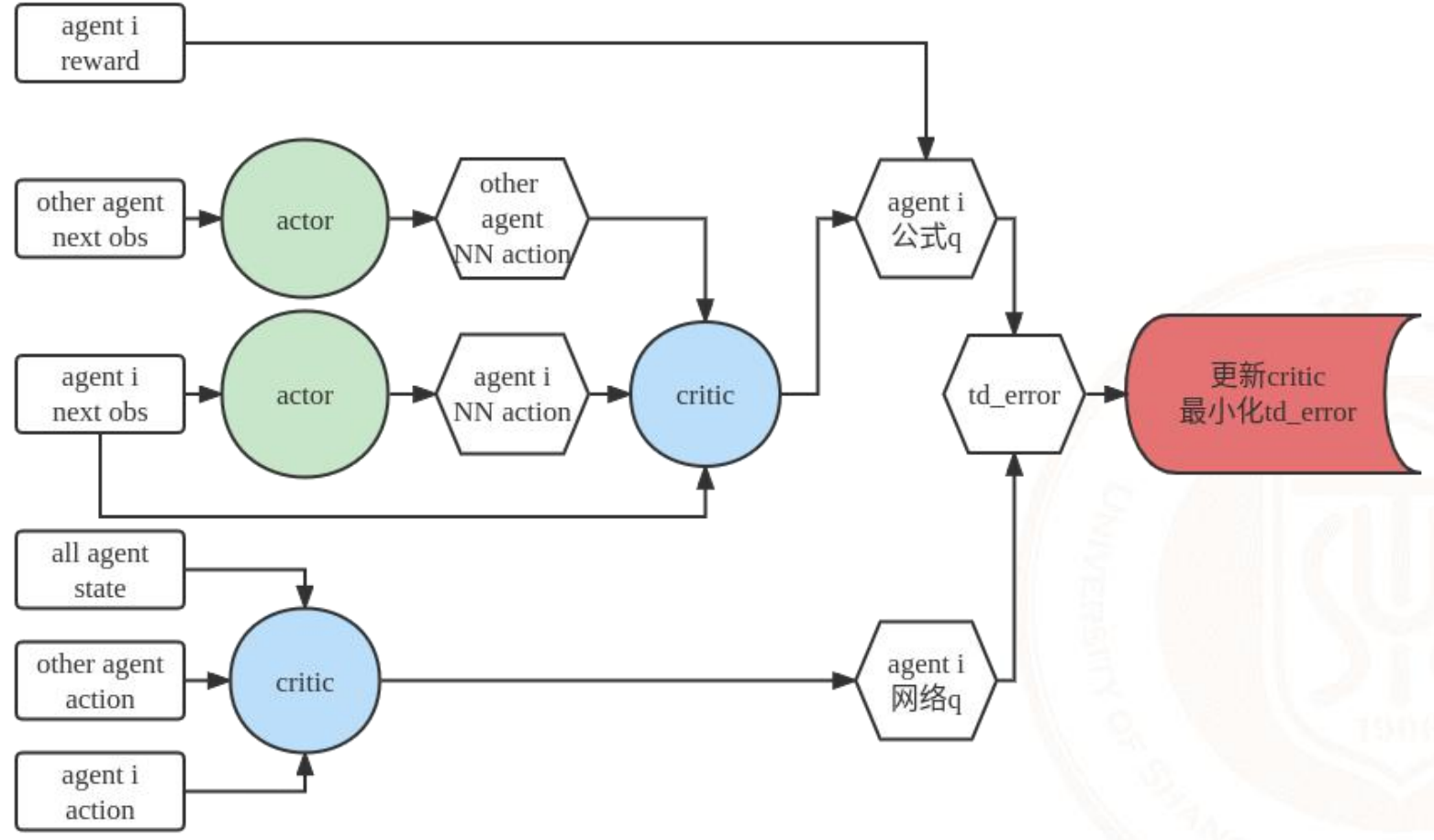
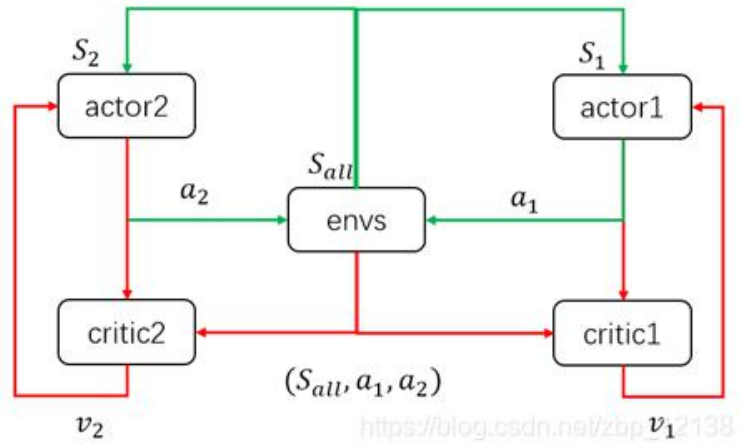
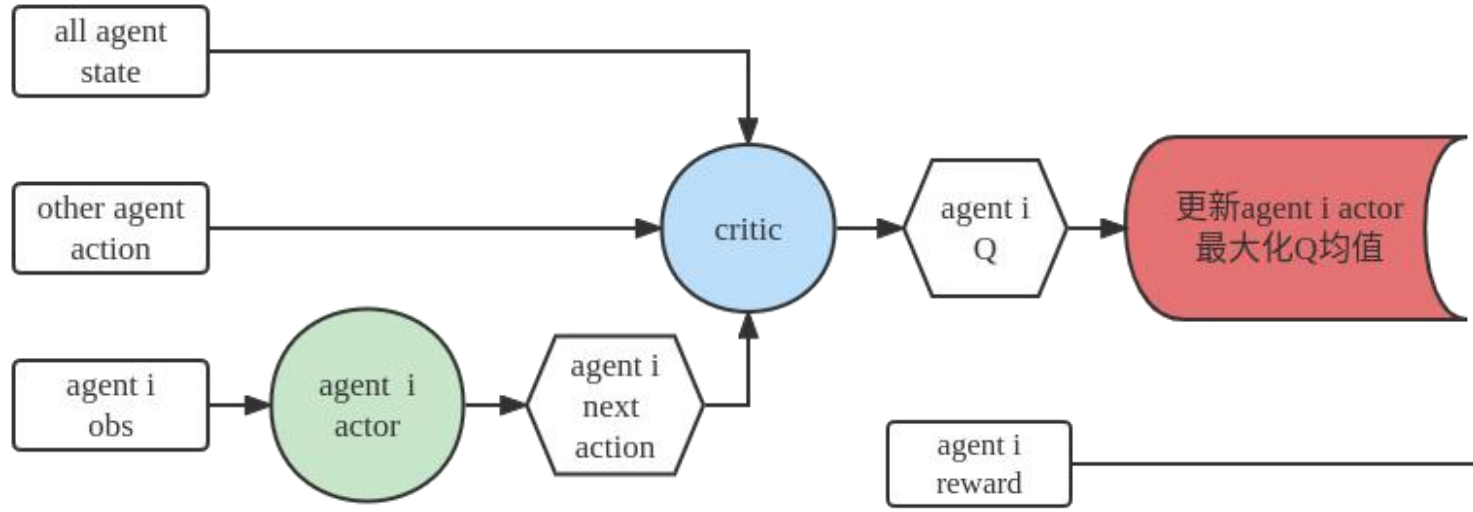




猎手2记忆单元

猎手2当前观测值	猎手3当前观测值	猎手1当前观测值
猎手2动作	猎手3动作	猎手1动作
单个猎手2奖励		
猎手2下一步观测值	猎手3下一步观测值	猎手1下一步观测值





- 0. 开题-- 基于多智能体的强化学习算法（有待细化，限制场景）
- 1. 开题时间 --安排多智能体学习
- 2. 代码 -- 流程图
- 3. 猎物 不随机行动（简单，简单，再简单）

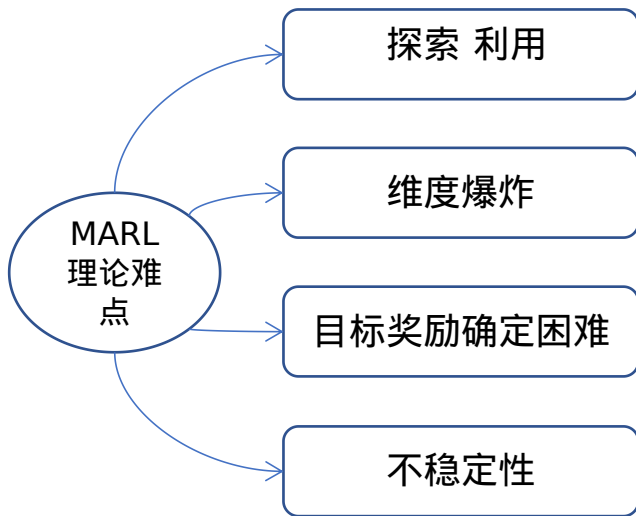




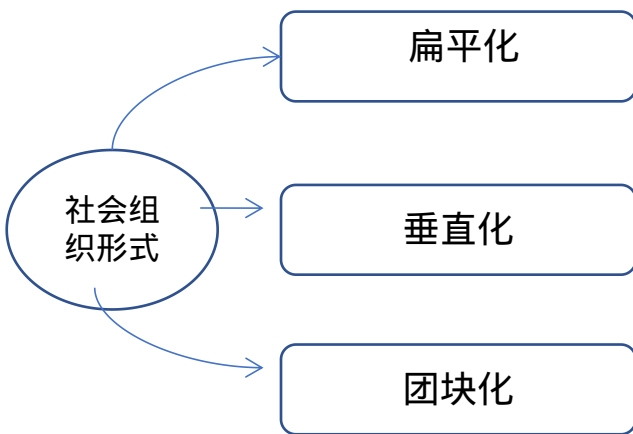
2022.12.10



基础知识与博弈：多智能体博弈



MARL目标：均衡



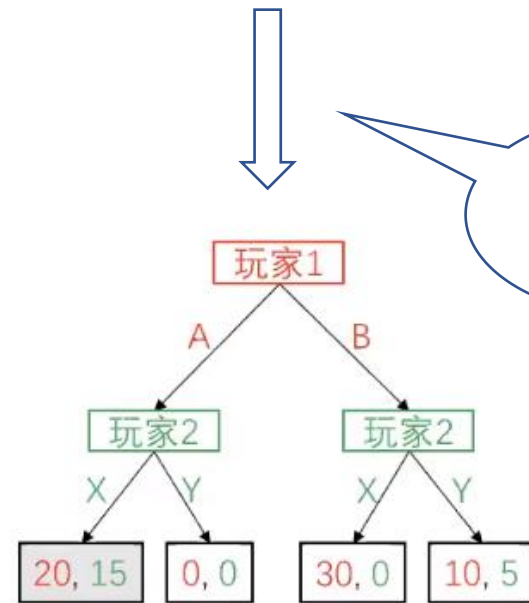
联结动作：
 $A_t = [a_{1,t}, a_{2,t}, \dots, a_{n,t}]^T$

联结策略： $(\pi_1, \dots, \pi_n)$

纳什均衡：在纳什均衡处，**对于所有智能体而言都不能在仅改变自身策略的情况下，来获得更大的奖励。**

	X	Y
A	20, 15	0, 0
B	30, 0	10, 5

纳什均衡



现实：状态、动作空间大
博弈元素不完全可知

obs $\neq$ state

如何自动学习出使
智能体群体利益最大化的社会结构？

引入通讯机制可改变均衡



2022.2.11

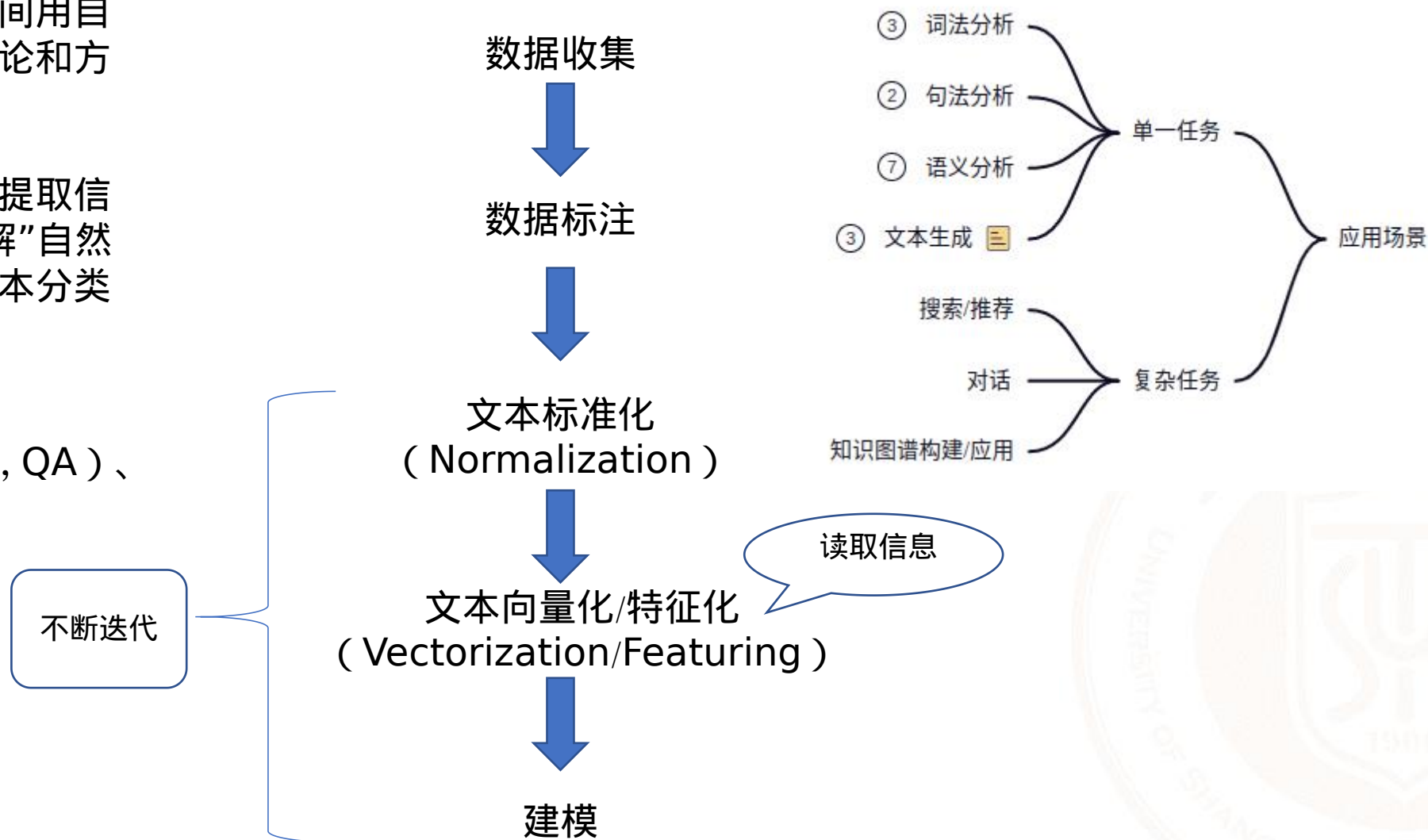


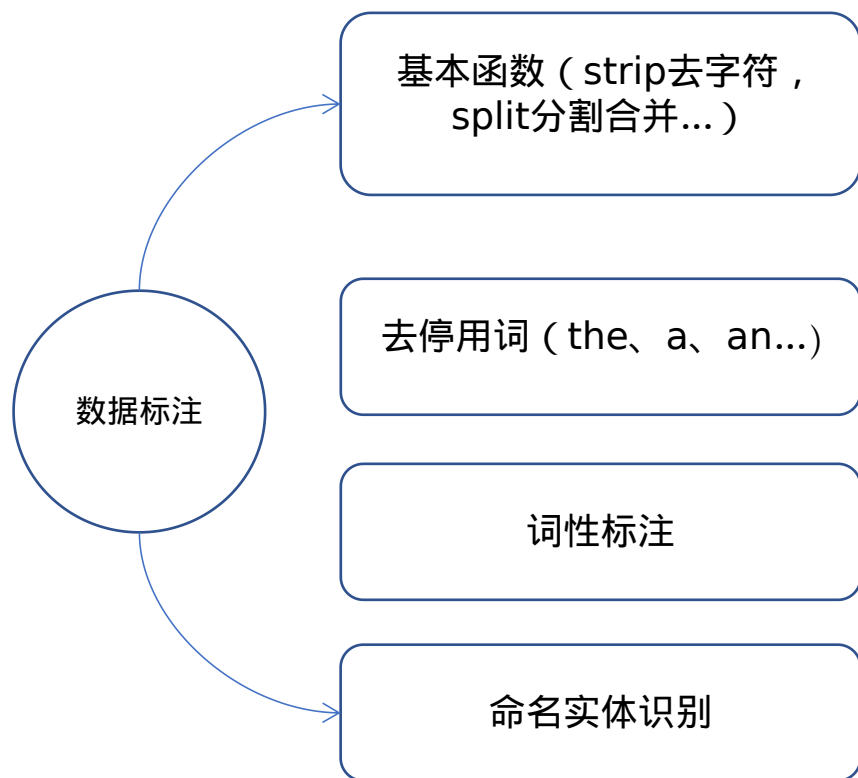
自然语言处理 (Natural Language Processing , NLP)

研究对象：人与计算机之间用自然语言进行有效通信的理论和办法。

研究目的：从文本数据中提取信息。让计算机处理或“理解”自然语言，执行自动翻译、文本分类和情感分析等。

技术范畴：问答系统 (Question Answering, QA)、语音识别 (Speech Recognition)





Today's weather is good , very windy and sunny.
['weather', '.', 'good', 'sunny', 'today', 'windy', ',', '']

(PERSON Edison/NNP)
went/VBD
to/TO
(ORGANIZATION Tsinghua/NNP University/NNP)
today/NN)



TF-IDF

IDF越大，说明词条越具有很好的类别区分能力。

TF(词频,Term Frequency):
表示词条(关键字)在文本中出现的频率。

IDF:逆向文件频率(Inverse Document Frequency)

$$TF_w = \frac{\text{在某一类中词条}w\text{出现的次数}}{\text{该类中所有的词条数目}}$$

$$IDF = \log \frac{\text{语料库的文档总数}}{\text{包含词条}w\text{的文档数}+1} \quad (\text{分母之所以要加1,是为了避免分母为0})$$

TF-IDF

某一特定文件内的高
词语频率

该词语在整个文件集
合中的低文件频率

高权重的
TF-IDF

过滤掉常见的词语，
保留重要的词语。



2022.2.14



词向量

一篇文章是一个
向量

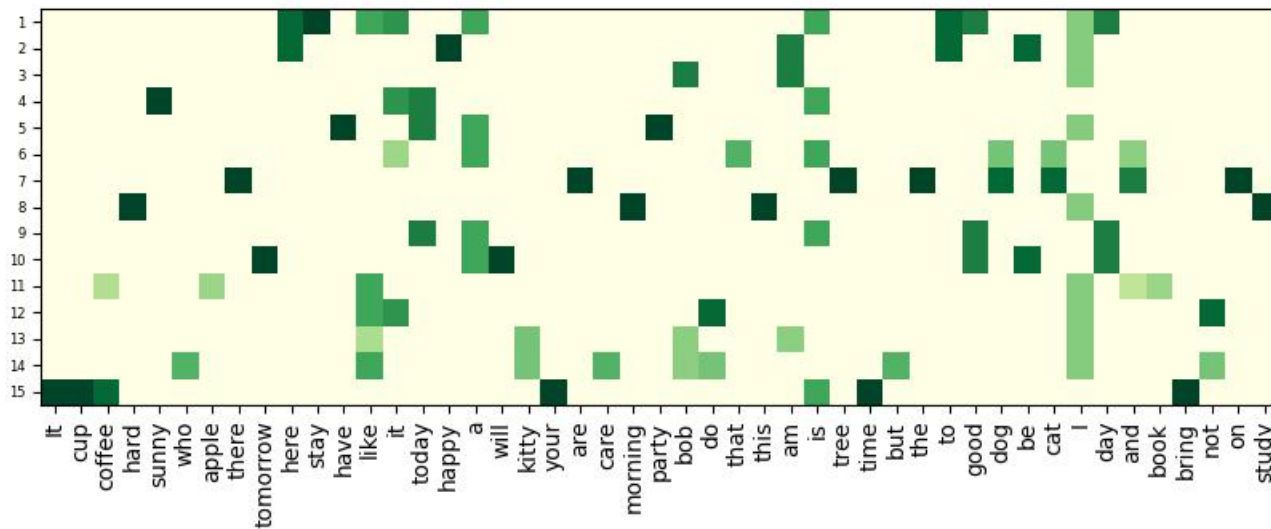
15 句话

47个不同词汇

```
docs = [  
    "it is a good day, I like to stay here",  
    "I am happy to be here",  
    "I am bob",  
    "it is sunny today",  
    "I have a party today",  
    "it is a dog and that is a cat",  
    "there are dog and cat on the tree",  
    "I study hard this morning",  
    "today is a good day",  
    "tomorrow will be a good day",  
    "I like coffee, I like book and I like apple",  
    "I do not like it",  
    "I am kitty, I like bob",  
    "I do not care who like bob, but I like kitty",  
    "It is coffee time, bring your cup",  
]
```

9、10两句

TF-IDF算法是通过统计学的方式来将文章中的词的重要程度，转化成向量来表示一篇文章。





2022.2.16



Continuous Bag of Words (CBOW)

挑一个要预测的词，来学习这个词前后文中词语和预测词的关系。

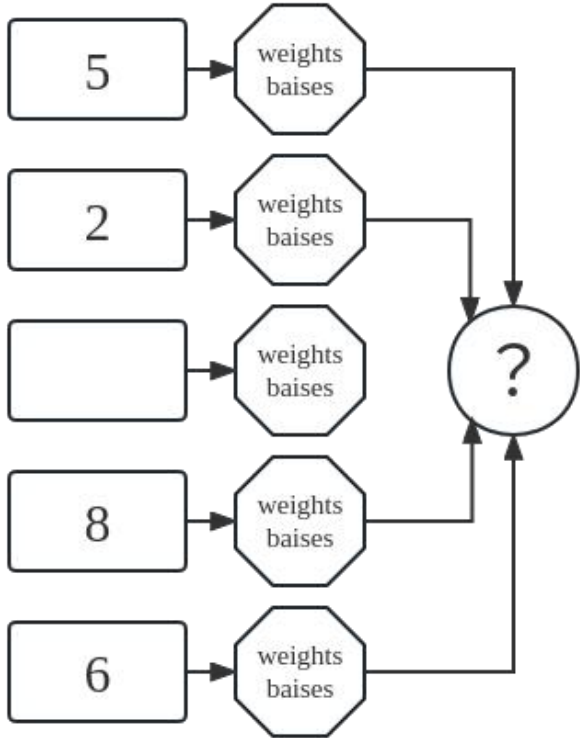
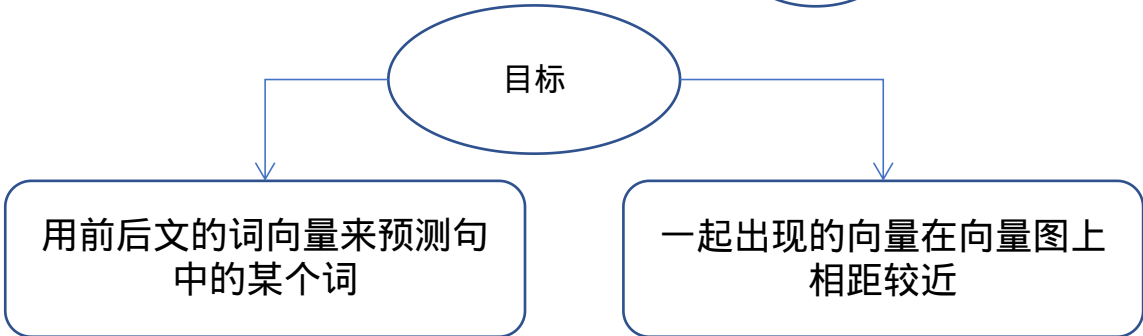
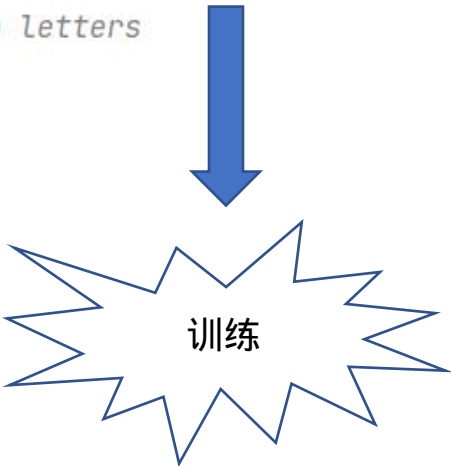
```
# numbers
"5 2 4 8 6 2 3 6 4",
"4 8 5 6 9 5 5 6",
"1 1 5 2 3 3 8",
"3 6 9 6 8 7 4 6 3",
"8 9 9 6 1 4 3 4",
"1 0 2 0 2 1 3 3 3 3",
"9 3 3 0 1 4 7 8",
"9 9 8 5 6 7 1 2 3 0 1 0",
```

alphabets, expecting that 9 is close to letters

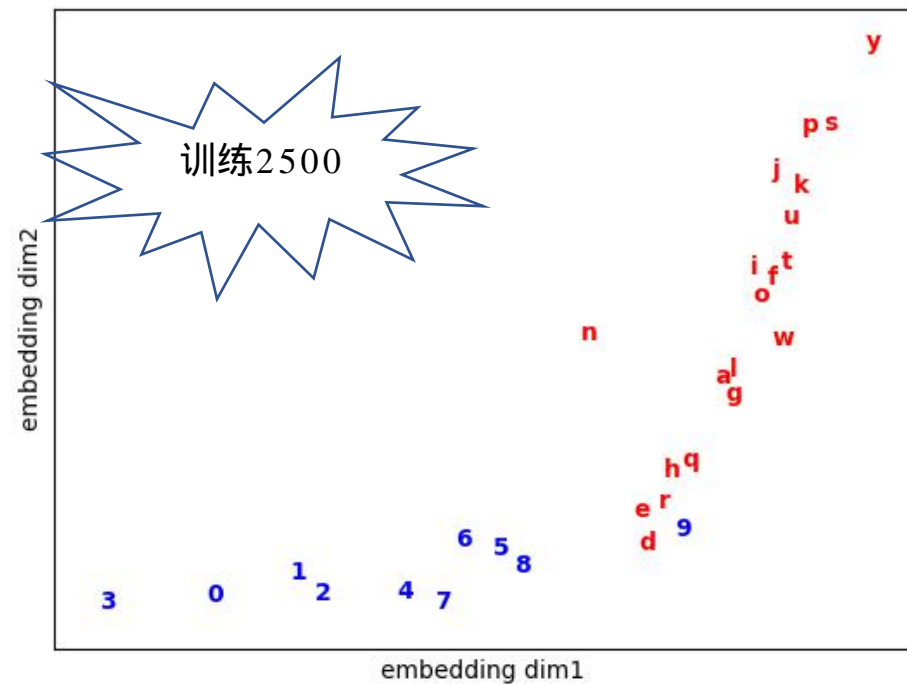
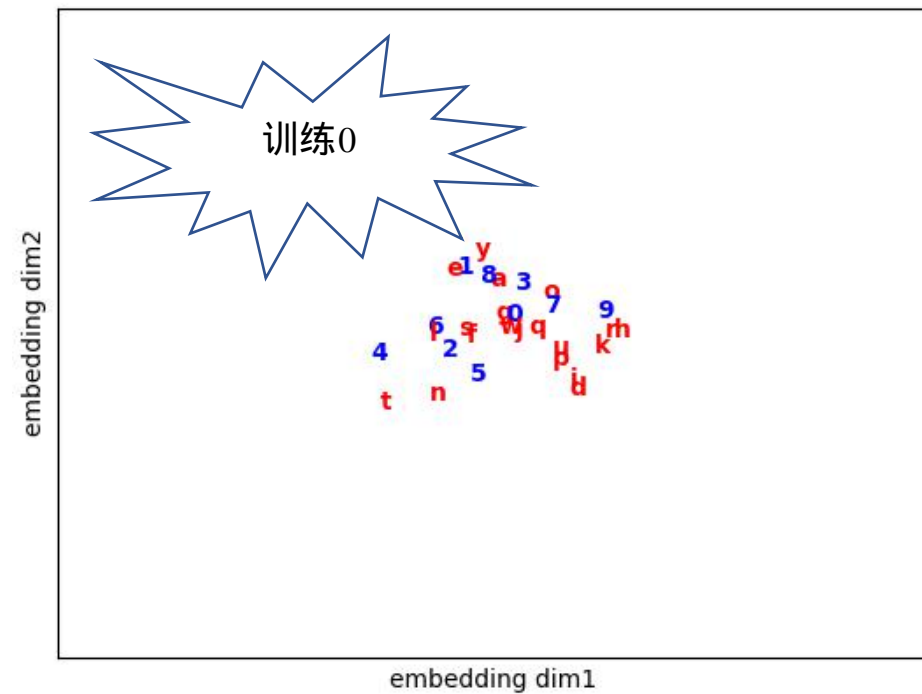
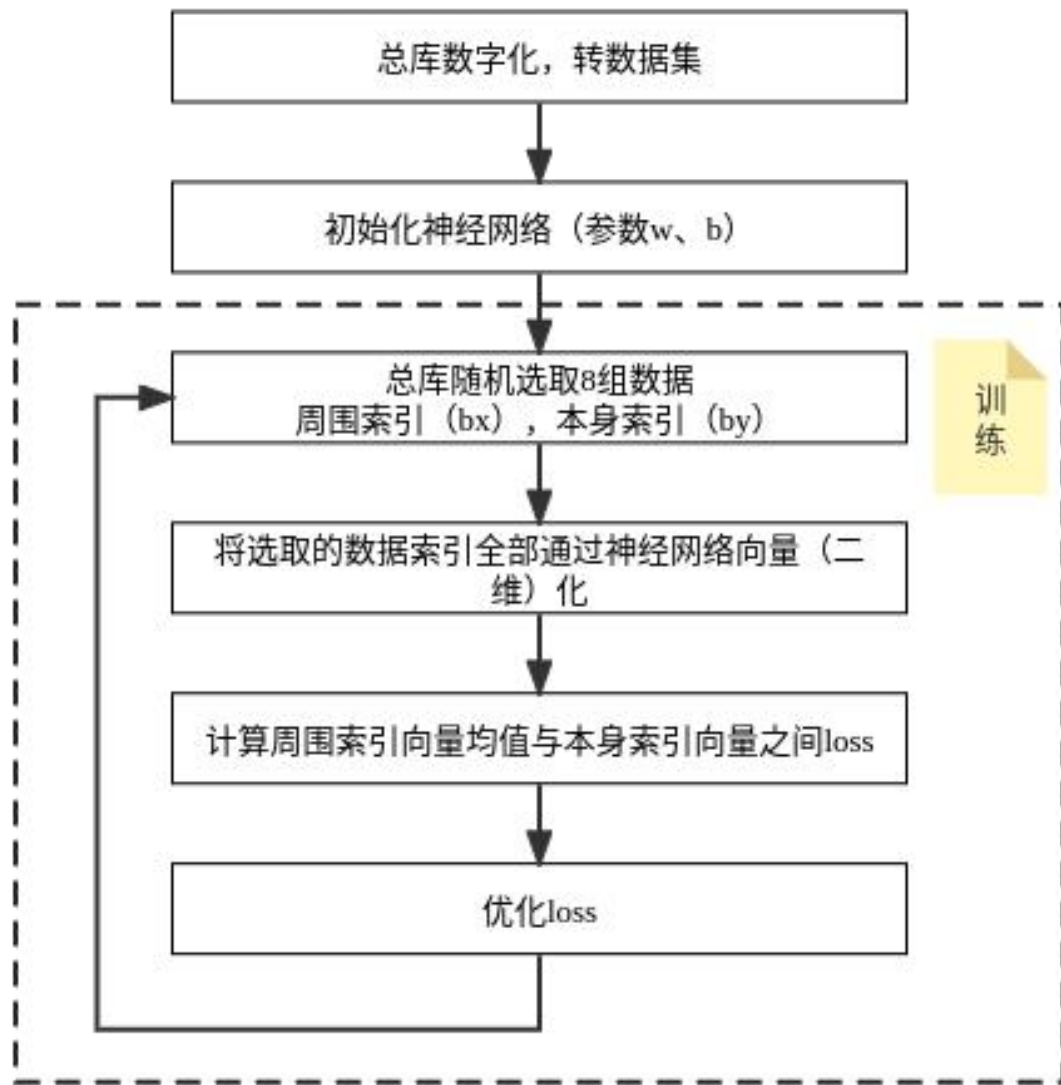
字母，期望9接近字母，9 夹杂在字母中

```
"a t g q e h 9 u f",
"e q y u o i p s",
"q o 9 p l k j o k k o p",
"h g y i u t t a e q",
"i k d q r e 9 e a d",
"o p d g 9 s a f g a",
"i u y g h k l a s w",
"o l u y a o g f s",
"o p i u y g d a s j d l",
"u k i l o 9 l j s",
"y g i s h k j l f r f",
"i o h n 9 9 d 9 f a 9",
```

数据集（向量化）	
[前2索引，前1索引， 后1索引，后2索引]	
本身索引	
{去重元素：索引}	
{索引：去重元素}	



$$(V_5 + V_2 + V_8 + V_6) / 4 = V_4$$





2022.2.21





Embedding (嵌入式、嵌入层)

one-hot (独热) 编码

小 普 喜 欢 星 海 湾 的 朋 友
0 1 2 3 4 5 6 7 8 9

小普喜欢星海湾的朋友	星海湾的朋友喜欢小普
[1 0 0 0 0 0 0 0 0 0]	[0 0 0 0 0 0 0 0 0 1 0]
[0 1 0 0 0 0 0 0 0 0]	[0 0 0 0 0 0 0 0 0 0 1]
[0 0 1 0 0 0 0 0 0 0]	[0 0 0 0 0 0 0 1 0 0 0]
[0 0 0 1 0 0 0 0 0 0]	[0 0 0 0 0 0 0 0 1 0 0]
[0 0 0 0 1 0 0 0 0 0]	[1 0 0 0 0 0 0 0 0 0 0]
[0 0 0 0 0 1 0 0 0 0]	[0 1 0 0 0 0 0 0 0 0 0]
[0 0 0 0 0 0 1 0 0 0]	[0 0 1 0 0 0 0 0 0 0 0]
[0 0 0 0 0 0 0 1 0 0]	[0 0 0 1 0 0 0 0 0 0 0]
[0 0 0 0 0 0 0 0 1 0]	[0 0 0 0 1 0 0 0 0 0 0]
[0 0 0 0 0 0 0 0 0 1]	[0 0 0 0 0 1 0 0 0 0 0]

我们的CBOW编码是将30维降为二维

此处本质是降维

计算简单，维度爆炸过于稀疏，占用资源

词向量都是彼此正交，体现不出相似关系

$$\begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \end{pmatrix}$$

A

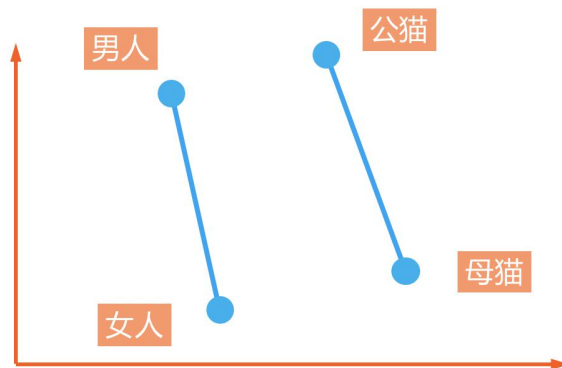
*

$$\begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \\ w_{31} & w_{32} & w_{33} \\ w_{41} & w_{42} & w_{43} \\ w_{51} & w_{52} & w_{53} \\ w_{61} & w_{52} & w_{53} \end{bmatrix} = \begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \end{pmatrix}$$

B

=

C



$$\begin{matrix} \text{男人} & - & \text{女人} \\ \approx & & \\ \text{公猫} & - & \text{母猫} \end{matrix}$$



Embedding (嵌入式、嵌入层)

keras.layers.Embedding函数

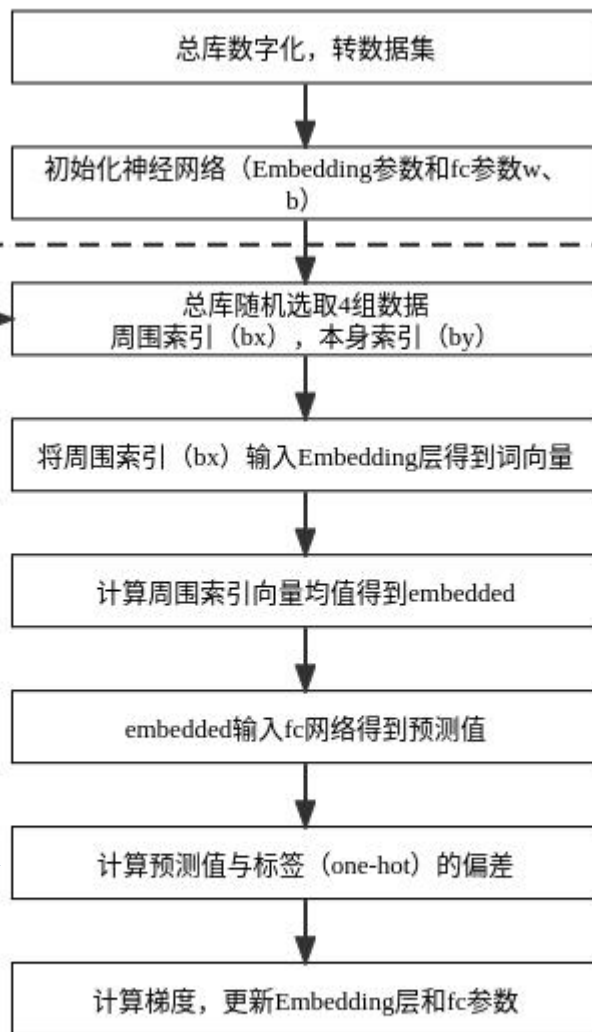
此处Embedding层是把向量中每一个标签值映射为一个2维向量

fc输入为4X2矩阵
embedded, 输出为4X30矩阵, 与每个元素的预测概率正相关

n_vocab
(30)

one-hot矩阵相当于每个元素概率列表

训练



元素9→索引0→

元素3→索引1→

Embedding Dim (2)	
...	...
...	...

元素w→索引28→

元素n→索引29→

索引

独热编码

词向量

对比CBOW

Skin-Gram

目标：预测文本中某个字周围可能出现的词

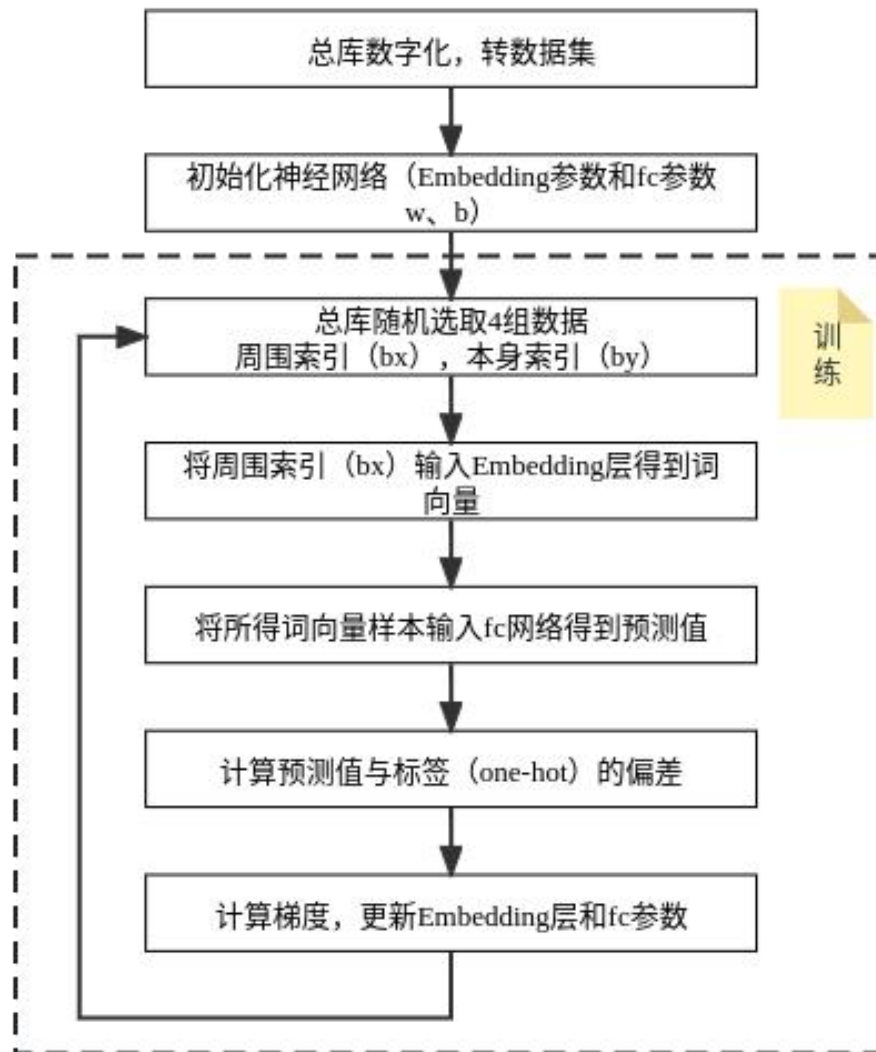
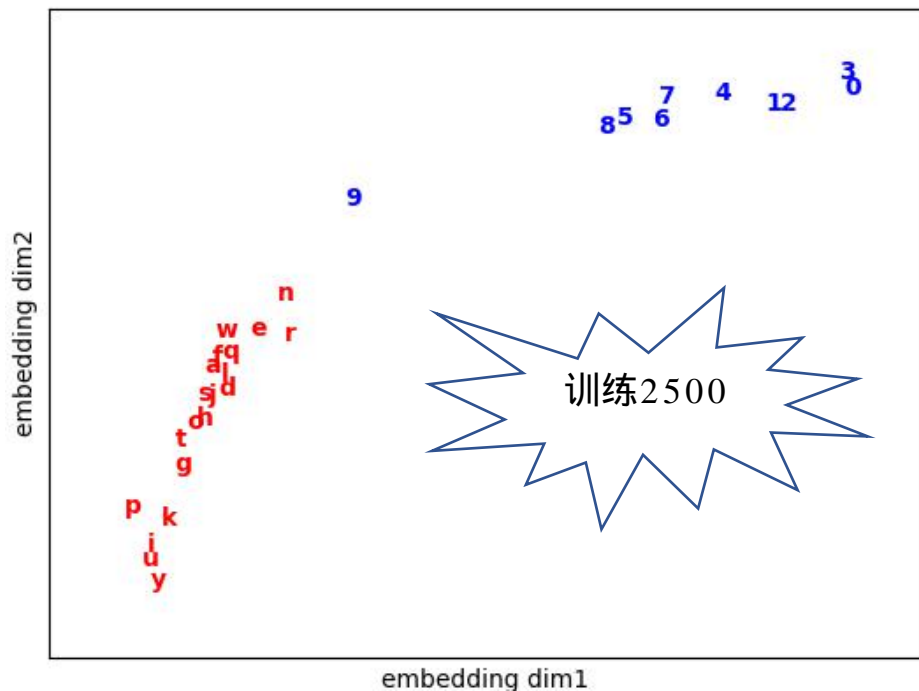
a t g q e h 9 u f

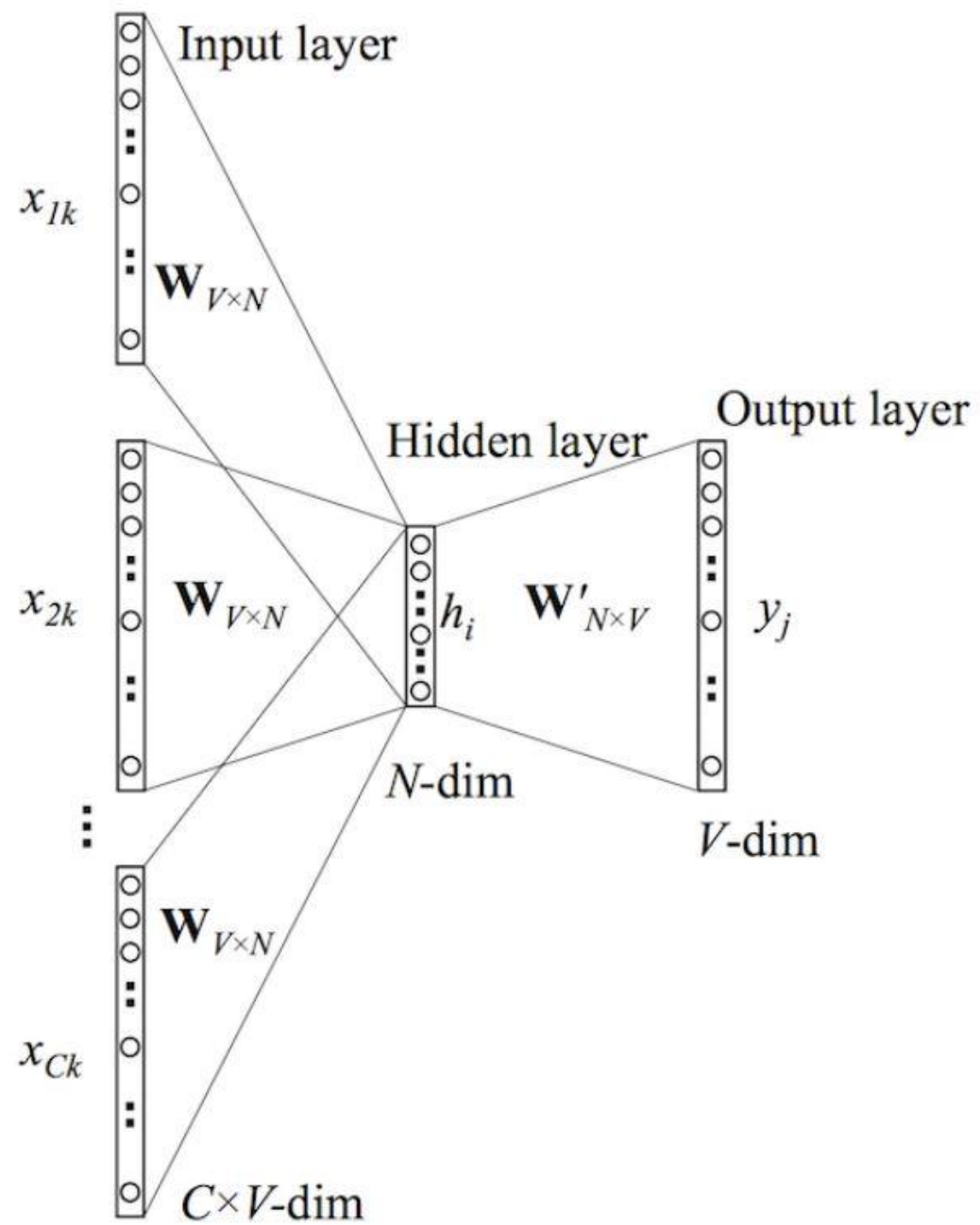
a t g q e h 9 u f

a t g q e h 9 u f

a t g q e h 9 u f

依据深褐色已知元素，预测浅褐色未知元素







2023.2.28

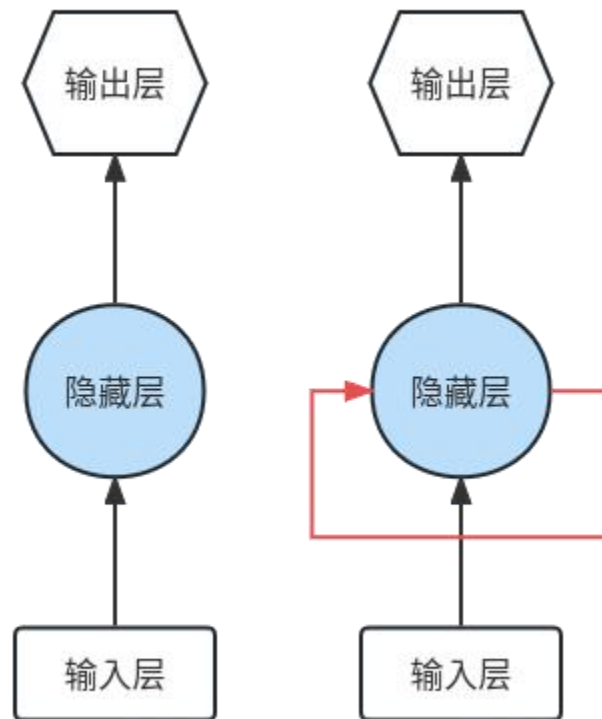
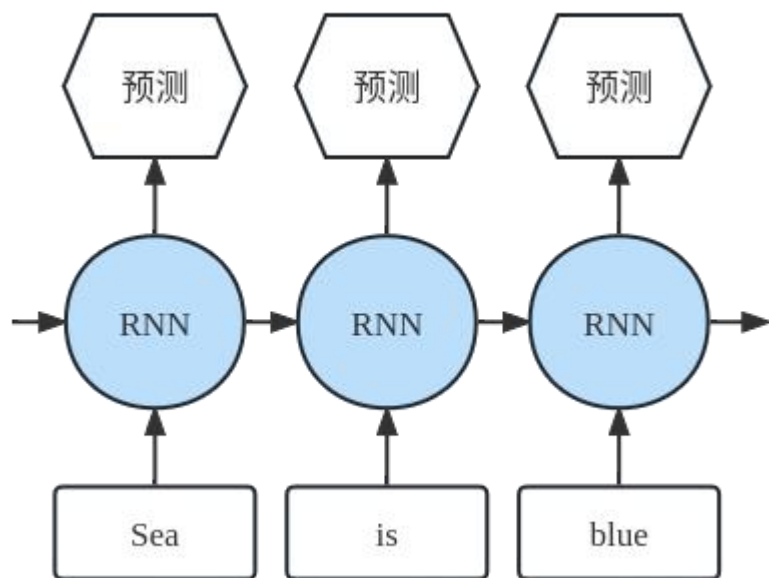
loss曲线



递归神经网络 (RNN, Recurssion Neural Network)

使用前馈神经网络时，神经网络会认为我们（ $t+1$ ）时刻输入的内容与 t 时刻输入的内容完全无关。

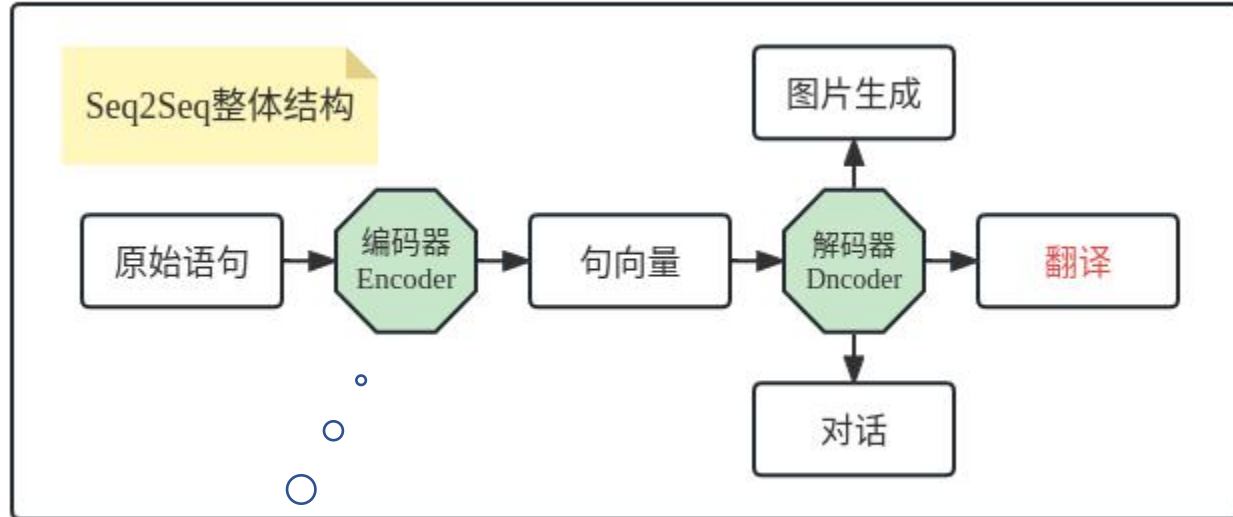
NLP分析时，需要合理运用之前的输入。



想象设计了一个基于简单RNN的句子自动补全器，显然第二步的预测值和第一步输入值（‘Sea’）有关。

Seq2Seq 模型

Seq2Seq 是一种循环神经网络的变种，包括编码器 (Encoder) 和解码器 (Decoder) 两部分。



编码器本质是
压缩器

Seq2Seq核心是训练
Encoder和Decoder



例程

目标：训练转化日期格式的模型

31-04-26转化为26/Apr/2031

数据集生成

生成时间戳列表（单位：秒）

列表内容分别转化为两种时间格式
20-05-04 (cn) 04/May/2020 (en)

生成元素字典并排序

cn格式列表内容转元素索引生成样本x

en格式列表内容转元素索引生成标签y

	3	1	-	0	4	-	2	6			
样本	6	4	1	3	7	1	5	9			
	GO	2	6	/	Apr	/	2	0	1	3	EOS
标签	14	5	9	2	15	2	5	3	6	4	13

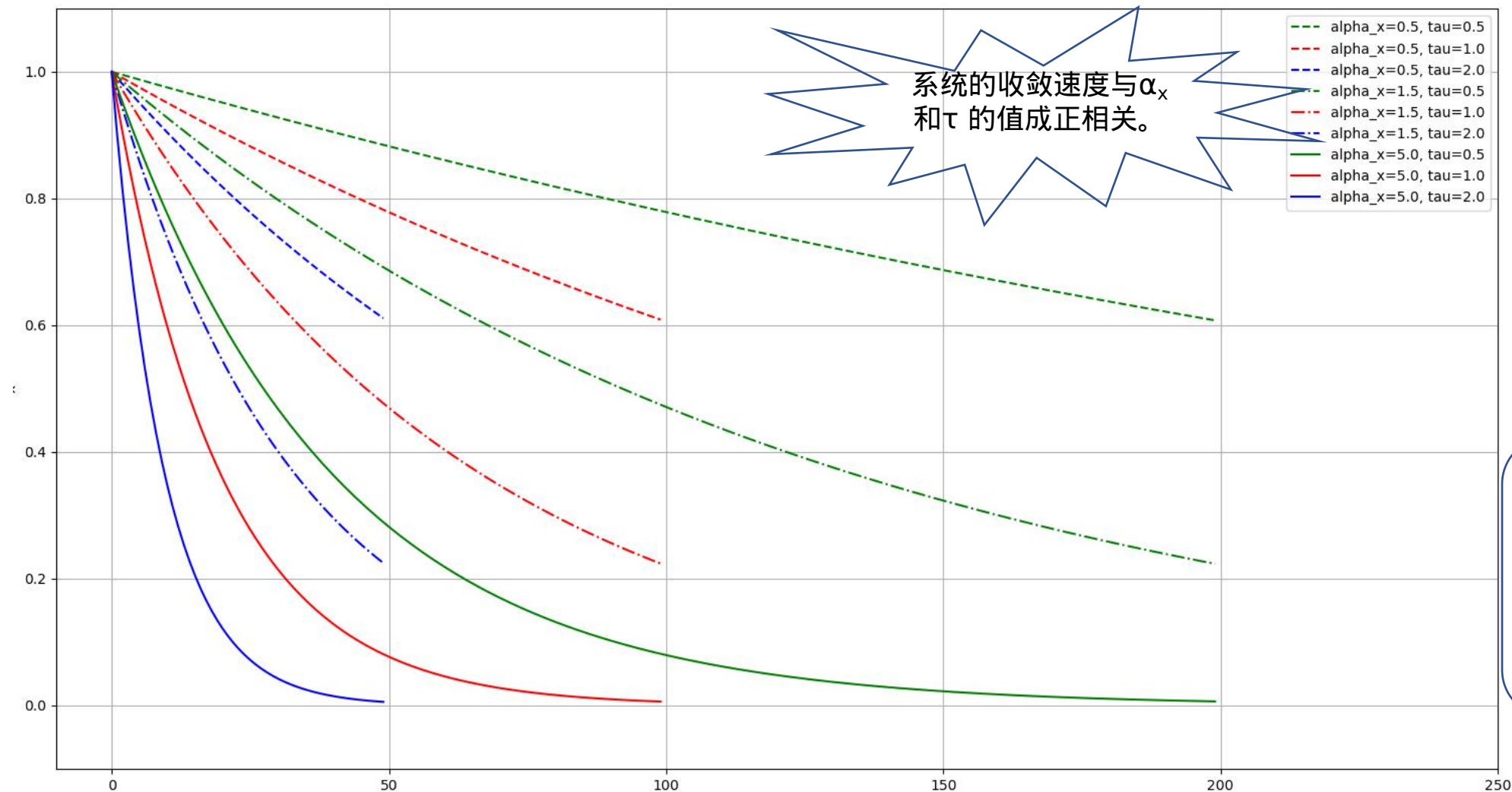


DMP
code

夹 (cs.py dmp_dis.py dmp文件)



弹簧阻尼系统 (PD控制器)



$$\frac{dx}{dt} = -\tau\alpha_x x$$

$t=0$ 时 $x=1$

$$x = \exp(-\tau\alpha_x t)$$

$$x = x + (dx/dt) * dt$$

$$x = x - \tau\alpha_x x dt$$

迭代

$$ma = F_{\text{弹}} - f_{\text{阻尼}} + F$$

$$a = \alpha (x - x_0) - \beta v + F/m$$

$$\ddot{y} = \alpha_y (\beta_y (g - y) - \dot{y}) + f$$

$$T_{\text{原}} = t_{\text{后}} / \tau$$

$$dy/dT = \tau dy/dt$$

$$d^2y/dT^2 = \tau^2 d^2y/dt^2$$

最终收敛于g

$$\tau^2 \ddot{y} = \alpha_y (\beta_y (g - y) - \dot{y}) + f$$

当前状态为y

高斯基函数(径向基函数)

基函数对应的权重

起始状态

$$f(t) = \frac{\sum_{i=1}^N \Psi_i(t) \omega_i}{\sum_{i=1}^N \Psi_i(t)} x(g - y_0)$$

最终收敛于g

x可以保证f(t)会随着x的收敛而消失

(g - y₀) 决定了f的“幅值”，用于后续对轨迹的形状进行“放缩”。

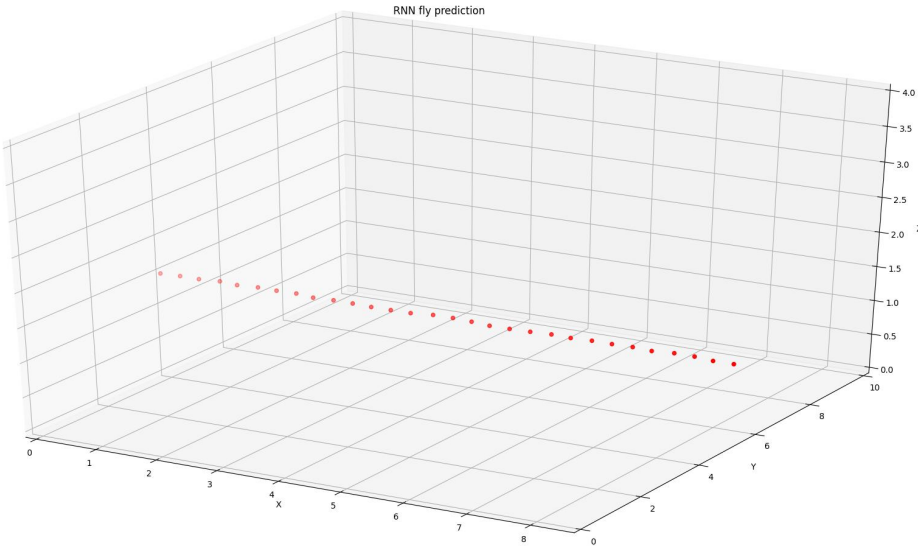
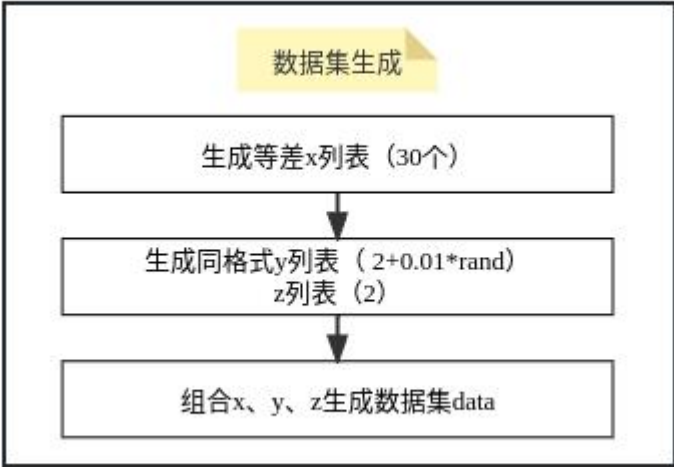


2023.3.4

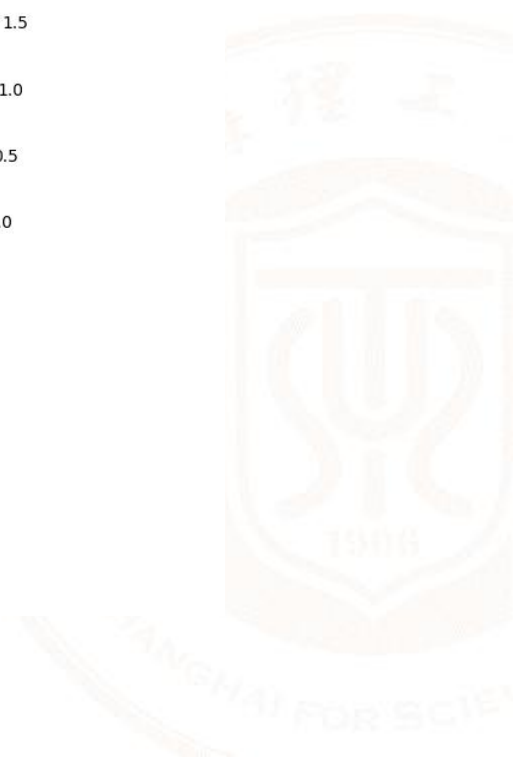
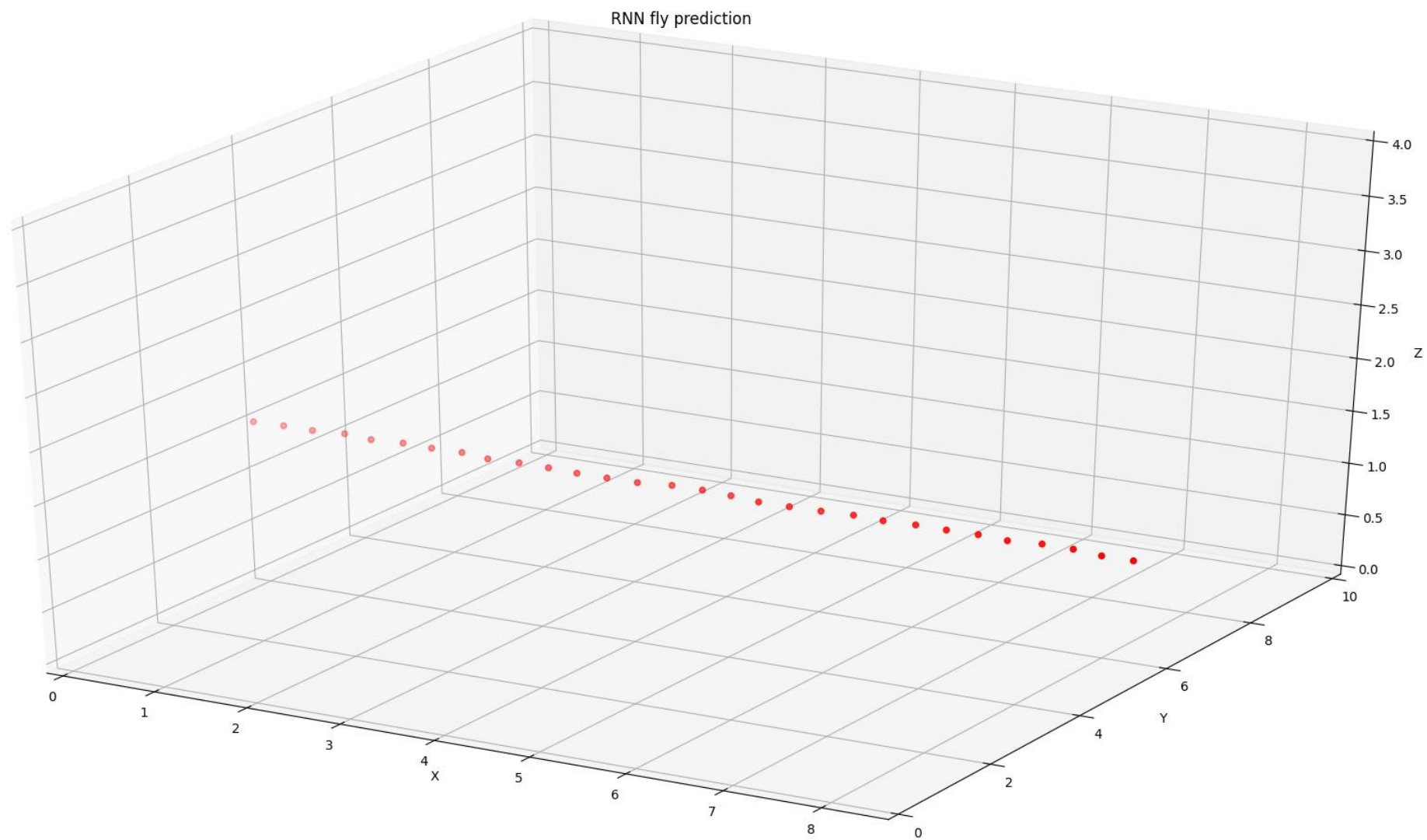


RNN模型：预测航迹

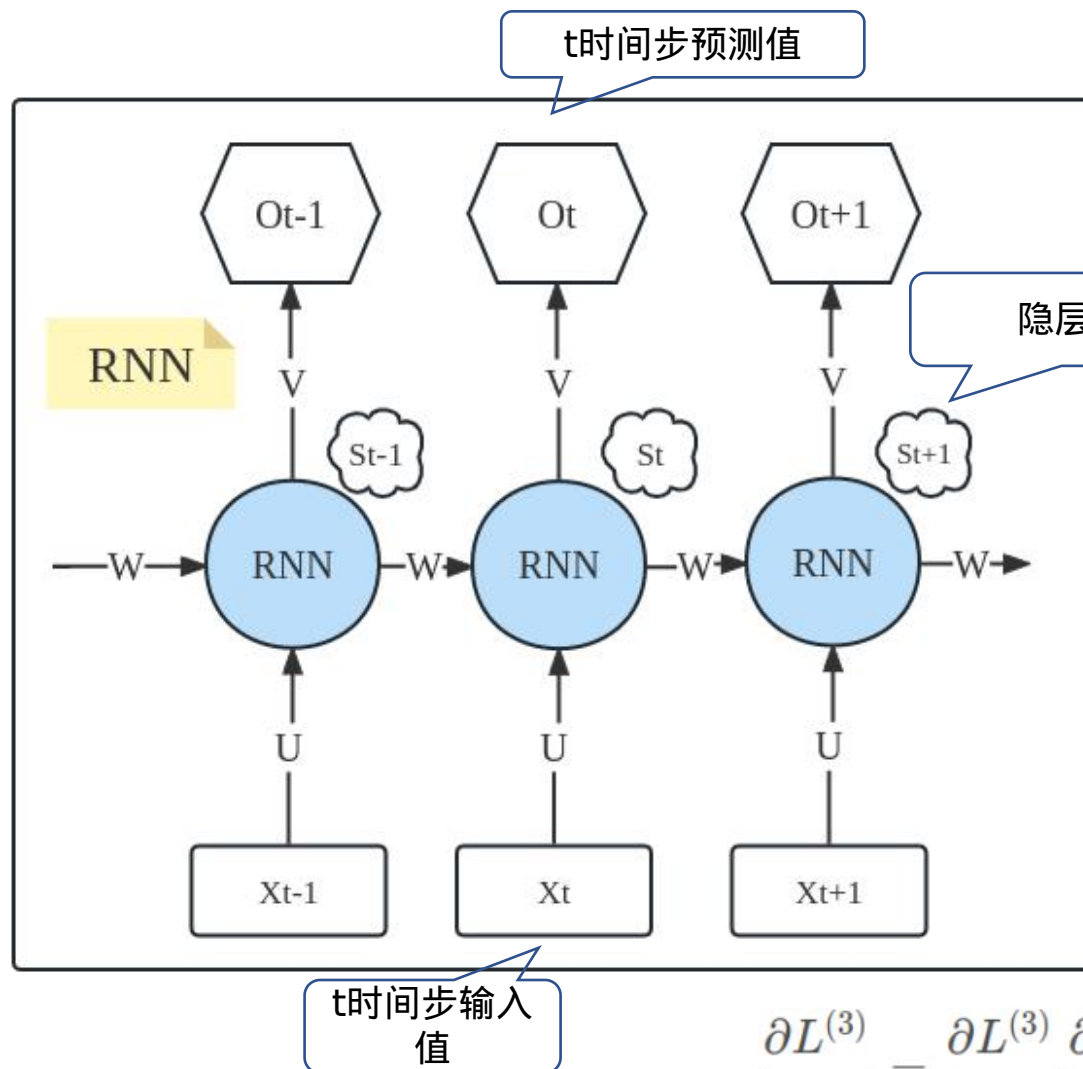
模型：依据已知飞机的三维航迹（坐标），预测接下来的航迹数据。



	x	y	z
	0	1	2
0	1.00000	2.00926	2.00000
1	1.31034	2.08682	2.00000
2	1.62069	2.06926	2.00000
3	1.93103	2.00112	2.00000
4	2.24138	2.02131	2.00000
5	2.55172	2.04473	2.00000
6	2.86207	2.05740	2.00000
7	3.17241	2.03513	2.00000
8	3.48276	2.02770	2.00000
9	3.79310	2.09824	2.00000
0	4.10345	2.02728	2.00000
1	4.41379	2.07668	2.00000
2	4.72414	2.07728	2.00000
3	5.03448	2.07636	2.00000
4	5.34483	2.05805	2.00000



RNN原理



$$S_t = \Phi (U x_t + W S_{t-1} + b)$$

$$O_t = V S_t + c$$

$$L = y_t - O_t$$

L : 损失函数
 y : 训练集的标签
 V 、 W 、 U : 权值

$$\frac{\partial L^{(t)}}{\partial V} = \frac{\partial L^{(t)}}{\partial o^{(t)}} \cdot \frac{\partial o^{(t)}}{\partial V}$$

累乘会导致激活函数导数的累乘，进而会导致“梯度消失”和“梯度爆炸”现象的发生。

$$\frac{\partial L^{(3)}}{\partial W} = \frac{\partial L^{(3)}}{\partial o^{(3)}} \frac{\partial o^{(3)}}{\partial h^{(3)}} \frac{\partial h^{(3)}}{\partial W} + \frac{\partial L^{(3)}}{\partial o^{(3)}} \frac{\partial o^{(3)}}{\partial h^{(3)}} \frac{\partial h^{(3)}}{\partial h^{(2)}} \frac{\partial h^{(2)}}{\partial W} + \frac{\partial L^{(3)}}{\partial o^{(3)}} \frac{\partial o^{(3)}}{\partial h^{(3)}} \frac{\partial h^{(3)}}{\partial h^{(2)}} \frac{\partial h^{(2)}}{\partial h^{(1)}} \frac{\partial h^{(1)}}{\partial W}$$

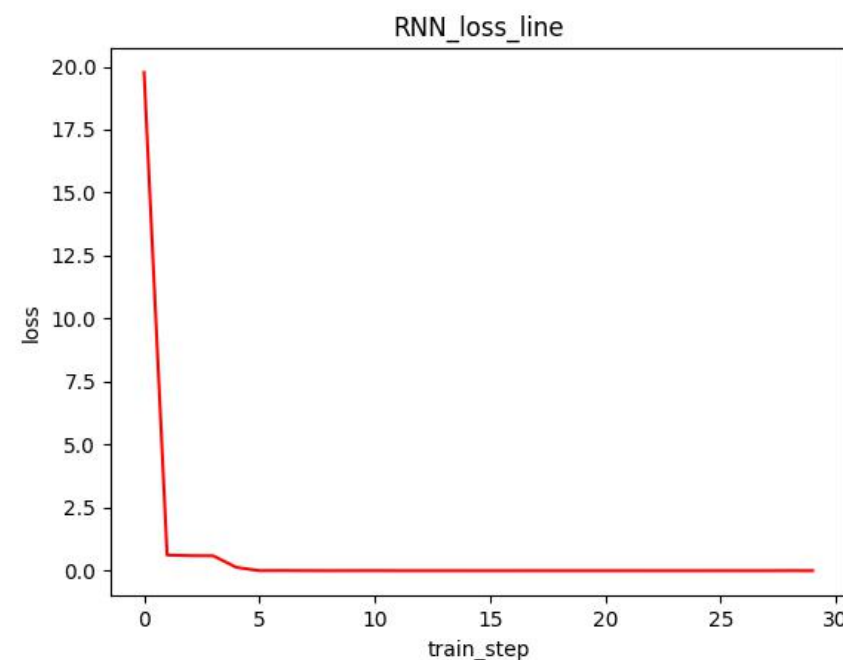
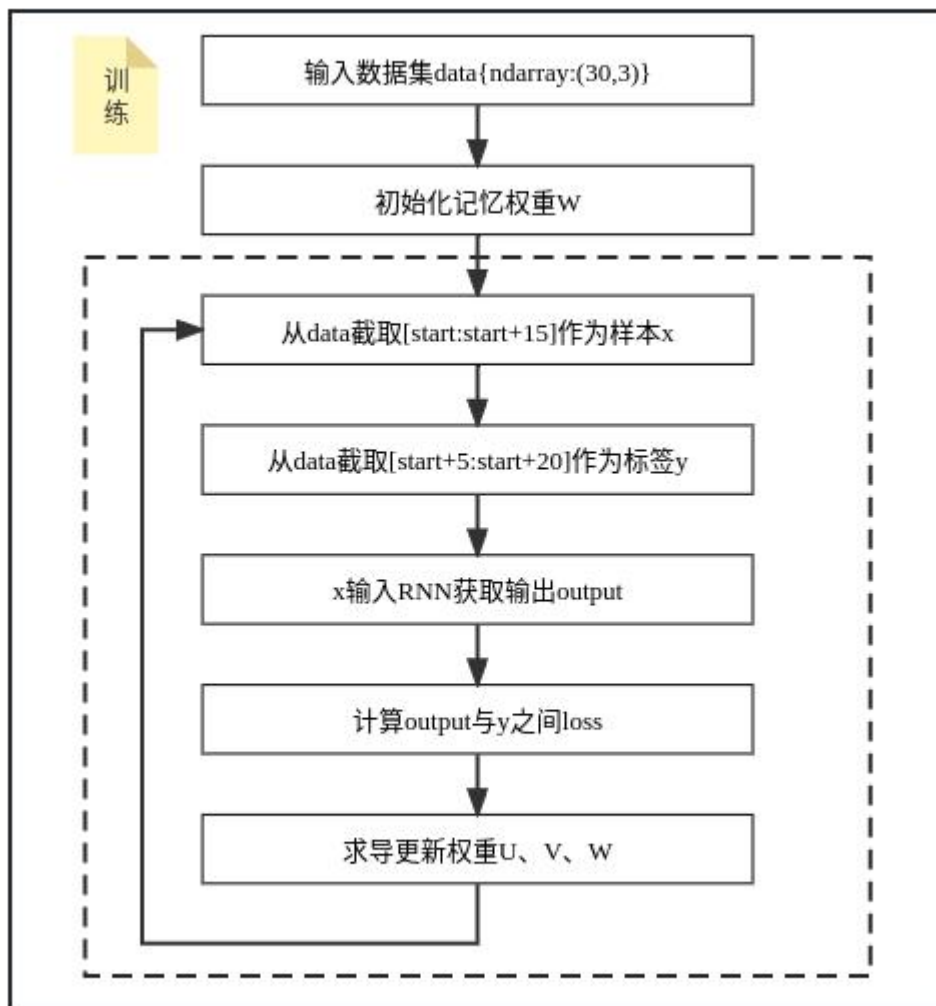
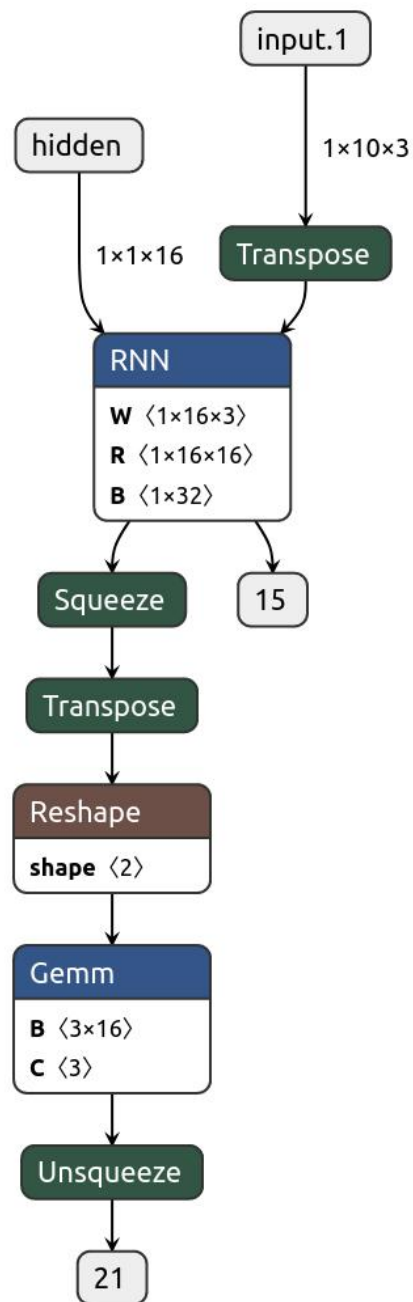
$$\frac{\partial L^{(3)}}{\partial U} = \frac{\partial L^{(3)}}{\partial o^{(3)}} \frac{\partial o^{(3)}}{\partial h^{(3)}} \frac{\partial h^{(3)}}{\partial U} + \frac{\partial L^{(3)}}{\partial o^{(3)}} \frac{\partial o^{(3)}}{\partial h^{(3)}} \frac{\partial h^{(3)}}{\partial h^{(2)}} \frac{\partial h^{(2)}}{\partial U} + \frac{\partial L^{(3)}}{\partial o^{(3)}} \frac{\partial o^{(3)}}{\partial h^{(3)}} \frac{\partial h^{(3)}}{\partial h^{(2)}} \frac{\partial h^{(2)}}{\partial h^{(1)}} \frac{\partial h^{(1)}}{\partial U}$$

递归神经网络 (RNN, Recurssion Neural Network)

$$S_t = \Phi (U x_t + W S_{t-1} + b)$$

$$O_t = V S_t + c$$

$$L = y_t - O_t$$



样本

```
tensor([[[[3.7931, 2.0195, 2.0000],  
          [4.1034, 2.0125, 2.0000],  
          [4.4138, 2.0526, 2.0000],  
          [4.7241, 2.0496, 2.0000],  
          [5.0345, 2.0179, 2.0000],  
          [5.3448, 2.0495, 2.0000],  
          [5.6552, 2.0423, 2.0000],  
          [5.9655, 2.0452, 2.0000],  
          [6.2759, 2.0267, 2.0000],  
          [6.5862, 2.0820, 2.0000]]]])
```

输出

```
tensor([[[[5.3063, 2.0607, 1.9949],  
          [5.6160, 2.0377, 1.9953],  
          [5.9566, 2.0423, 1.9962],  
          [6.2701, 2.0509, 1.9964],  
          [6.5869, 2.0598, 1.9968],  
          [6.8966, 2.0680, 1.9974],  
          [7.2097, 2.0732, 1.9976],  
          [7.5217, 2.0749, 1.9975],  
          [7.8394, 2.0727, 1.9971],  
          [8.1443, 2.0679, 1.9967]]]])
```

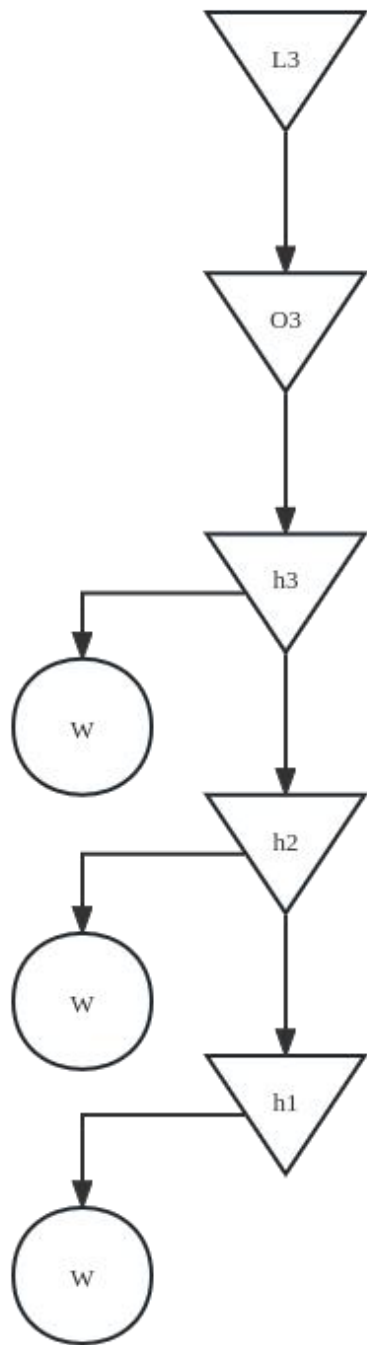
14	5.34483	2.04946	2.00000
15	5.65517	2.04226	2.00000
16	5.96552	2.04522	2.00000
17	6.27586	2.02667	2.00000
18	6.58621	2.08199	2.00000
19	6.89655	2.06064	2.00000
20	7.20690	2.09343	2.00000
21	7.51724	2.07748	2.00000
22	7.82759	2.09709	2.00000
23	8.13793	2.04459	2.00000
24	8.44828	2.09234	2.00000





2023.3.7





$$H_t = \Phi (U x_t + W H_{t-1} + b)$$

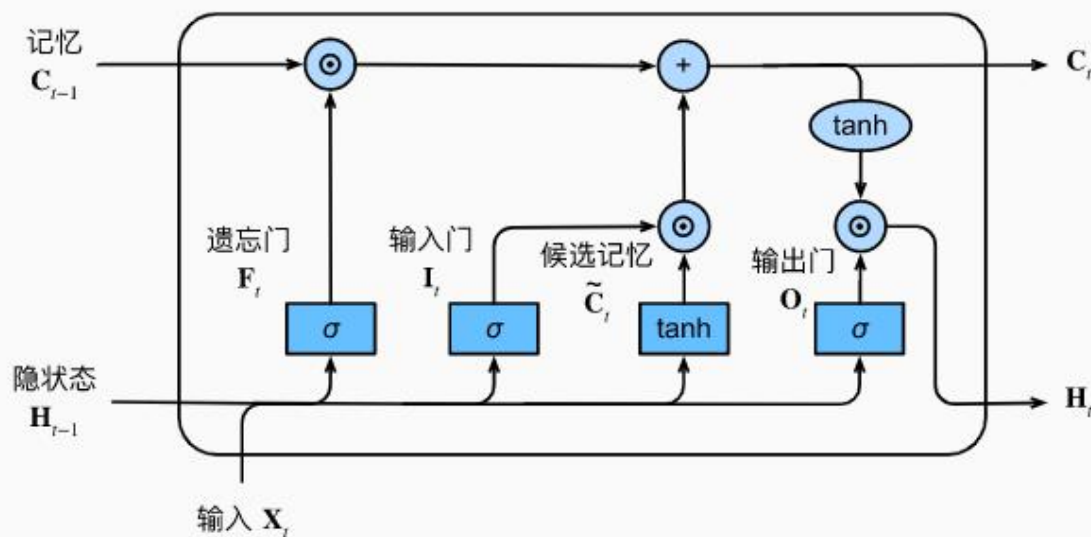
$$O_t = V H_t + c$$

$$L = y_t - O_t$$

$$\frac{\partial L^{(3)}}{\partial W} = \frac{\partial L^{(3)}}{\partial o^{(3)}} \frac{\partial o^{(3)}}{\partial h^{(3)}} \frac{\partial h^{(3)}}{\partial W} + \frac{\partial L^{(3)}}{\partial o^{(3)}} \frac{\partial o^{(3)}}{\partial h^{(3)}} \frac{\partial h^{(3)}}{\partial h^{(2)}} \frac{\partial h^{(2)}}{\partial W} + \frac{\partial L^{(3)}}{\partial o^{(3)}} \frac{\partial o^{(3)}}{\partial h^{(3)}} \frac{\partial h^{(3)}}{\partial h^{(2)}} \frac{\partial h^{(2)}}{\partial h^{(1)}} \frac{\partial h^{(1)}}{\partial W}$$

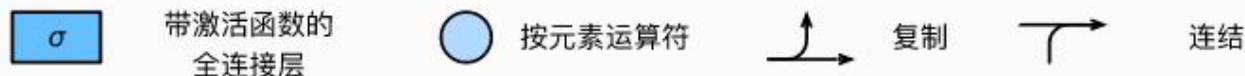
$$\frac{\partial L^{(3)}}{\partial U} = \frac{\partial L^{(3)}}{\partial o^{(3)}} \frac{\partial o^{(3)}}{\partial h^{(3)}} \frac{\partial h^{(3)}}{\partial U} + \frac{\partial L^{(3)}}{\partial o^{(3)}} \frac{\partial o^{(3)}}{\partial h^{(3)}} \frac{\partial h^{(3)}}{\partial h^{(2)}} \frac{\partial h^{(2)}}{\partial U} + \frac{\partial L^{(3)}}{\partial o^{(3)}} \frac{\partial o^{(3)}}{\partial h^{(3)}} \frac{\partial h^{(3)}}{\partial h^{(2)}} \frac{\partial h^{(2)}}{\partial h^{(1)}} \frac{\partial h^{(1)}}{\partial U}$$

LSTM



记忆 (Ct)：每个LSTM单元保留一个不输出的内部状态，也称为细胞状态。是对于所有输入信息的“概括总结”

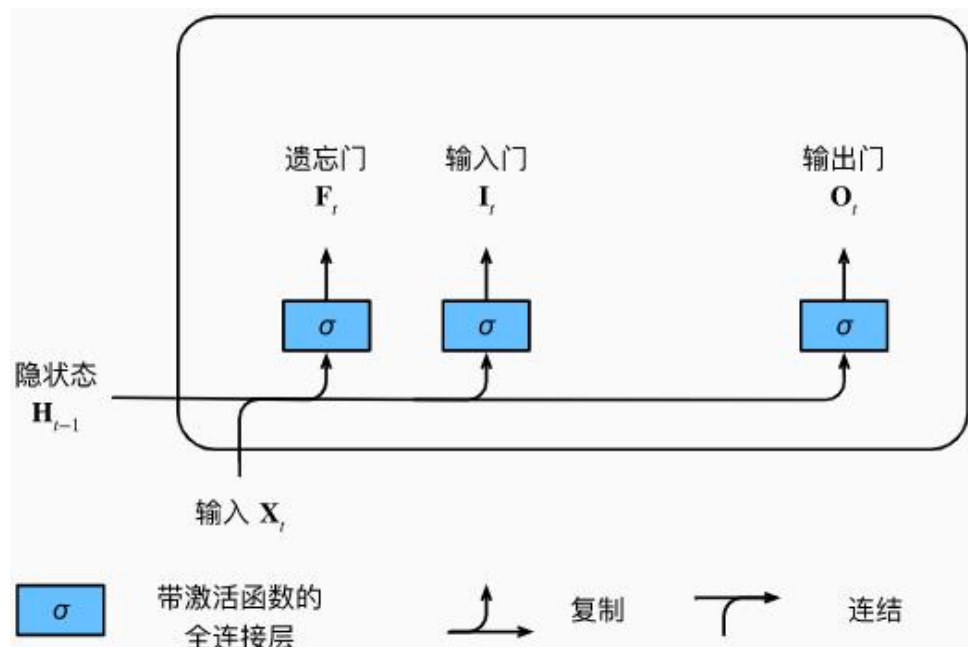
隐藏状态Ht：直接与输出状态相关。



相对于RNN，LSTM有各种“门”处理信息

通过训练让“门”学习保存哪些信息？丢弃哪些信息？
保存的信息哪些输出？哪些不输出？

输入门 I_t 、忘记门 F_t 和输出门 O_t



当前时间步的输入 X_t 和前一个时间步的隐状态 H_{t-1} 作为数据送入LSTM，由三个具有sigmoid激活函数的全连接层处理，以计算输入门、忘记门和输出门的值。这三个门的值都在(0,1)的范围内。

忘记门:根据新的输入和上一时刻的输出决定遗忘掉之前的哪些记忆。

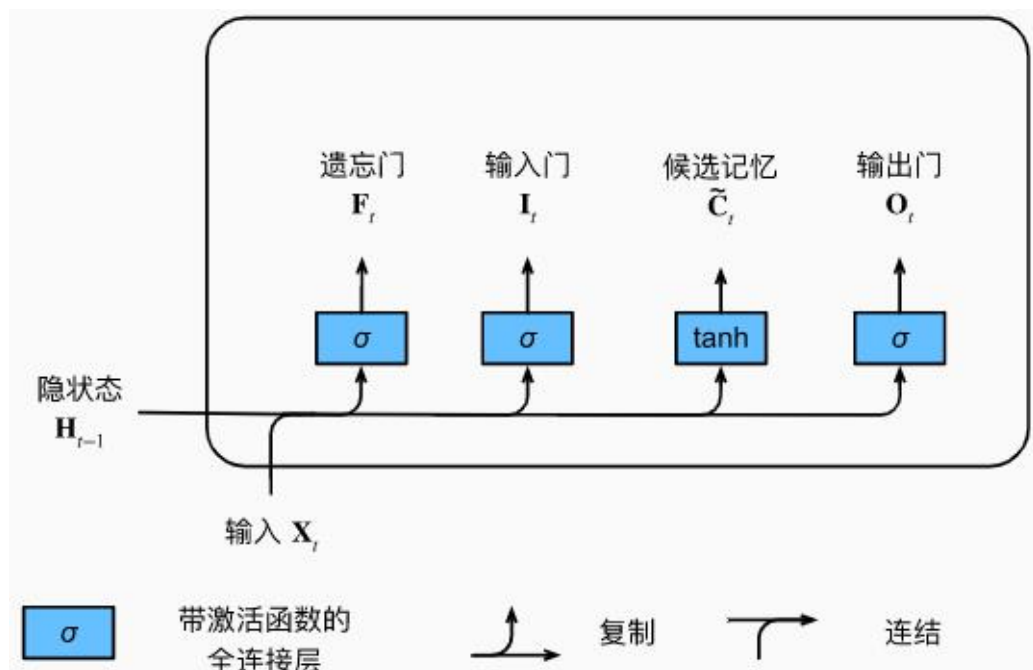
输入门：决定让多少新的信息加入到 cell 状态 中来。

输出门：计算当前时刻的输出值的神经层。

$$\begin{aligned} I_t &= \sigma(X_t W_{xi} + H_{t-1} W_{hi} + b_i), \\ F_t &= \sigma(X_t W_{xf} + H_{t-1} W_{hf} + b_f), \\ O_t &= \sigma(X_t W_{xo} + H_{t-1} W_{ho} + b_o), \end{aligned}$$

建立I、F、O与隐
状态 H_{t-1} &样本
 X_t 之间的关系。

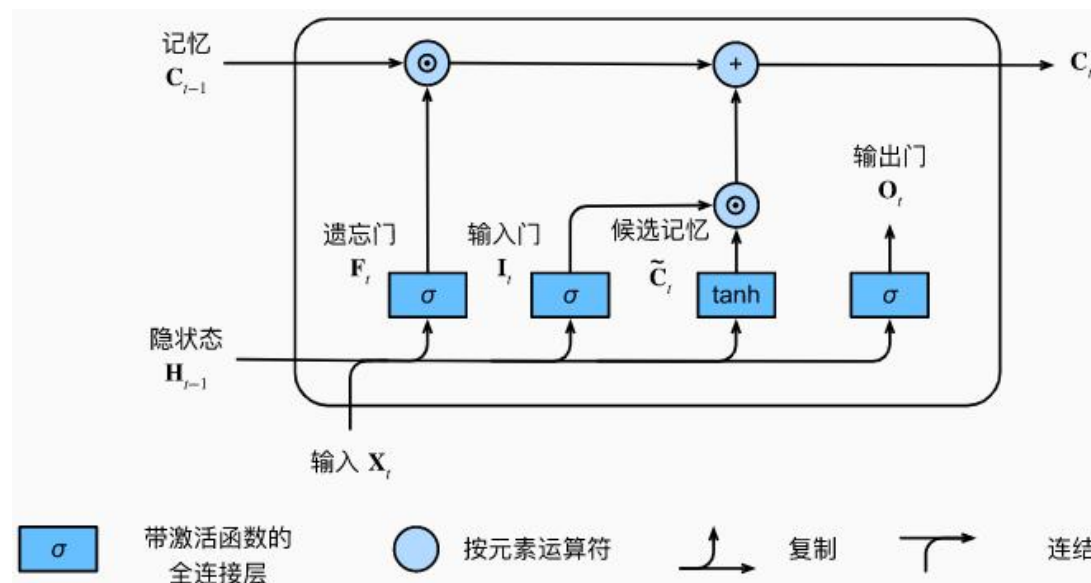
候选记忆元



$$\tilde{C}_t = \tanh(\mathbf{X}_t \mathbf{W}_{xc} + \mathbf{H}_{t-1} \mathbf{W}_{hc} + \mathbf{b}_c)$$

将输入的 x_t 和 h_{t-1} 整合，然后通过一个 $\tanh$ 神经网络层来创建一个新的状态候选向量 $\hat{C}_t$

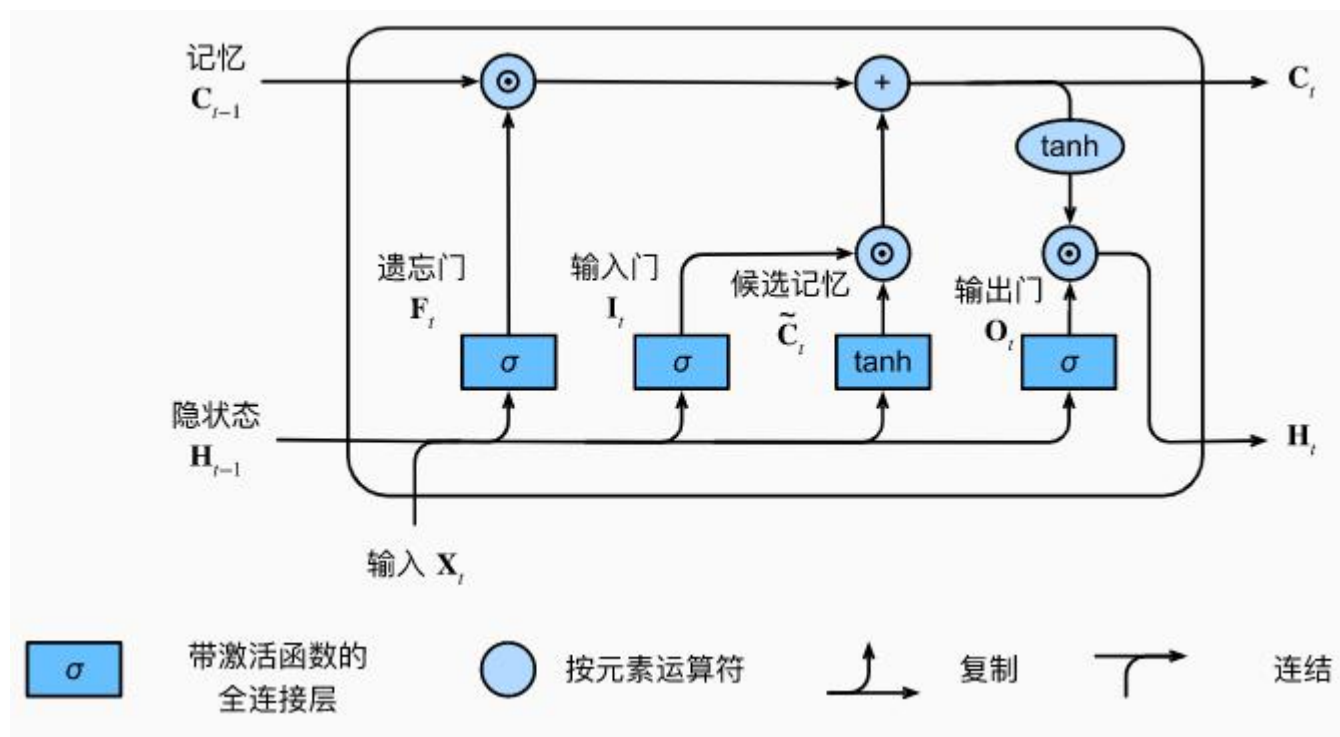
记忆元



$$\mathbf{C}_t = \mathbf{F}_t \odot \mathbf{C}_{t-1} + \mathbf{I}_t \odot \tilde{\mathbf{C}}_t$$

输入门 I_t 控制采用多少来自 $\hat{C}_t$ 的新数据
遗忘门 F_t 控制保留多少过去的记忆元 C_{t-1} 的内容。

隐状态



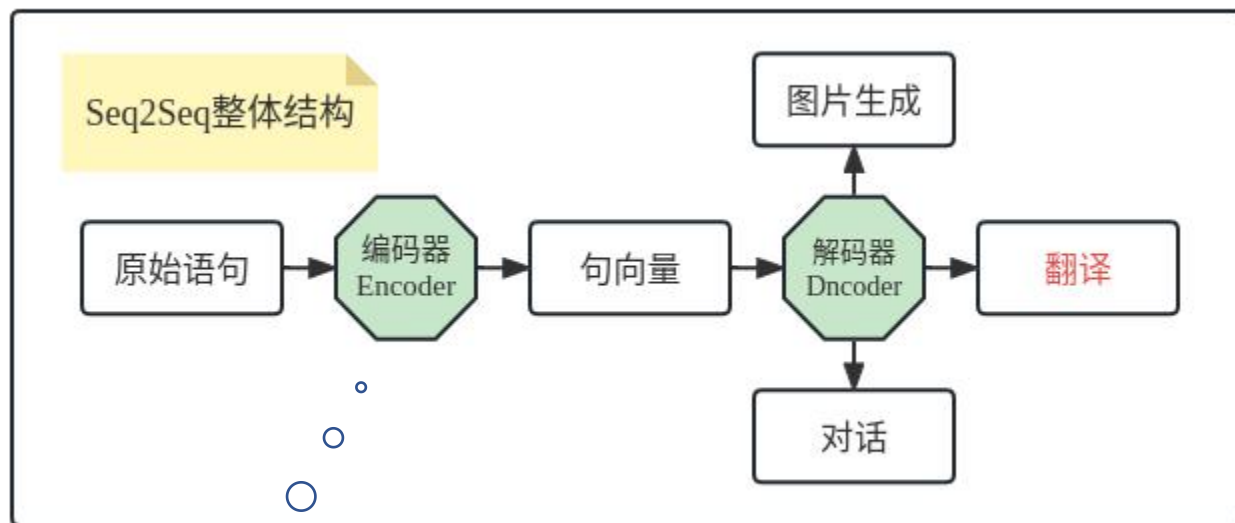
$$H_t = O_t \odot \tanh(C_t)$$

隐状态会传递到输出层
记忆元完全属于内部信息



Seq2Seq 模型

Seq2Seq 是一种循环神经网络的变种，包括编码器 (Encoder) 和解码器 (Decoder) 两部分。



编码器本质是
压缩器

Seq2Seq核心是训练
Encoder和Decoder



例程

目标：训练转化日期格式的模型

31-04-26转化为26/Apr/2031

数据集生成

生成时间戳列表（单位：秒）

列表内容分别转化为两种时间格式
20-05-04 (cn) 04/May/2020 (en)

生成元素字典并排序

cn格式列表内容转元素索引生成样本x

en格式列表内容转元素索引生成标签y

	1	2	3	4	5	6	7	8	9	10	11
	3	1	-	0	4	-	2	6			
样本	6	4	1	3	7	1	5	9			
	GO	2	6	/	Apr	/	2	0	1	3	EOS
标签	14	5	9	2	15	2	5	3	6	4	13

如何评价dmp学习结果优劣





2023.3.12



压缩器encoder

作用：将原始的词向量按顺序组装起来变成句向量。

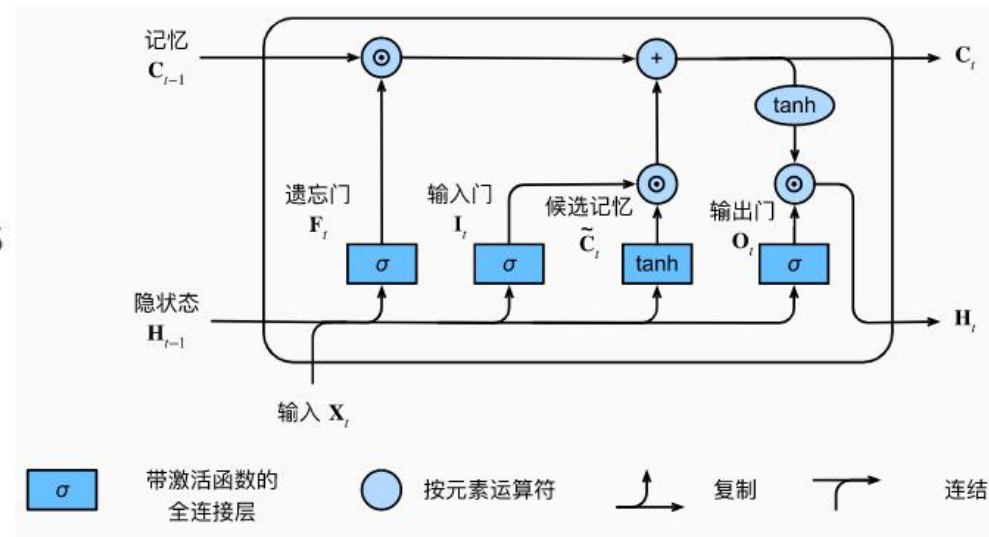
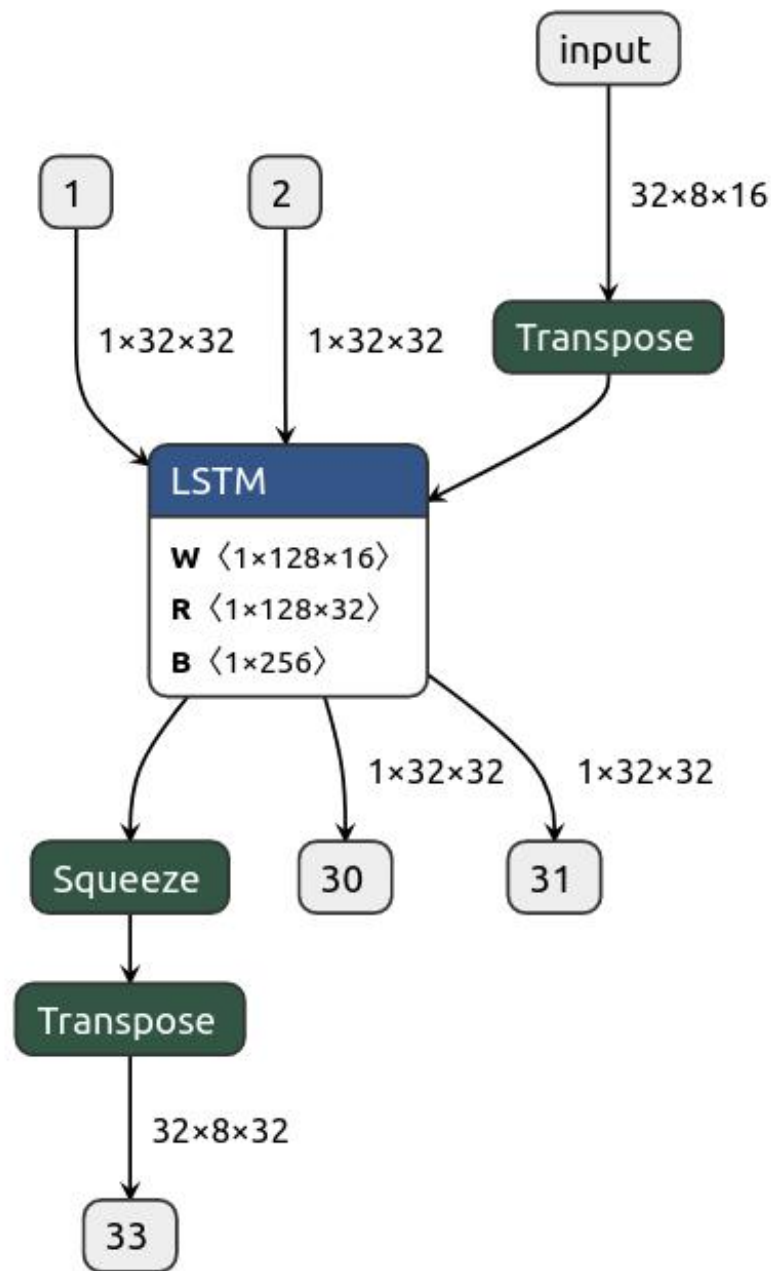
input{tensor:(32,8)} : 32
样本，每个样本8个one-hot
编码（27元素）

1&30：初始隐藏状态

2&31：初始细胞状态

33：输出

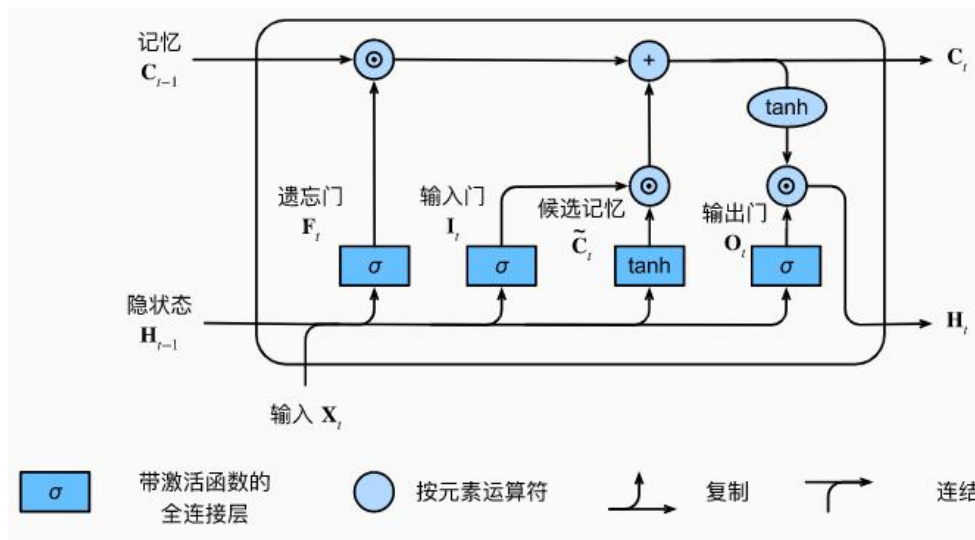
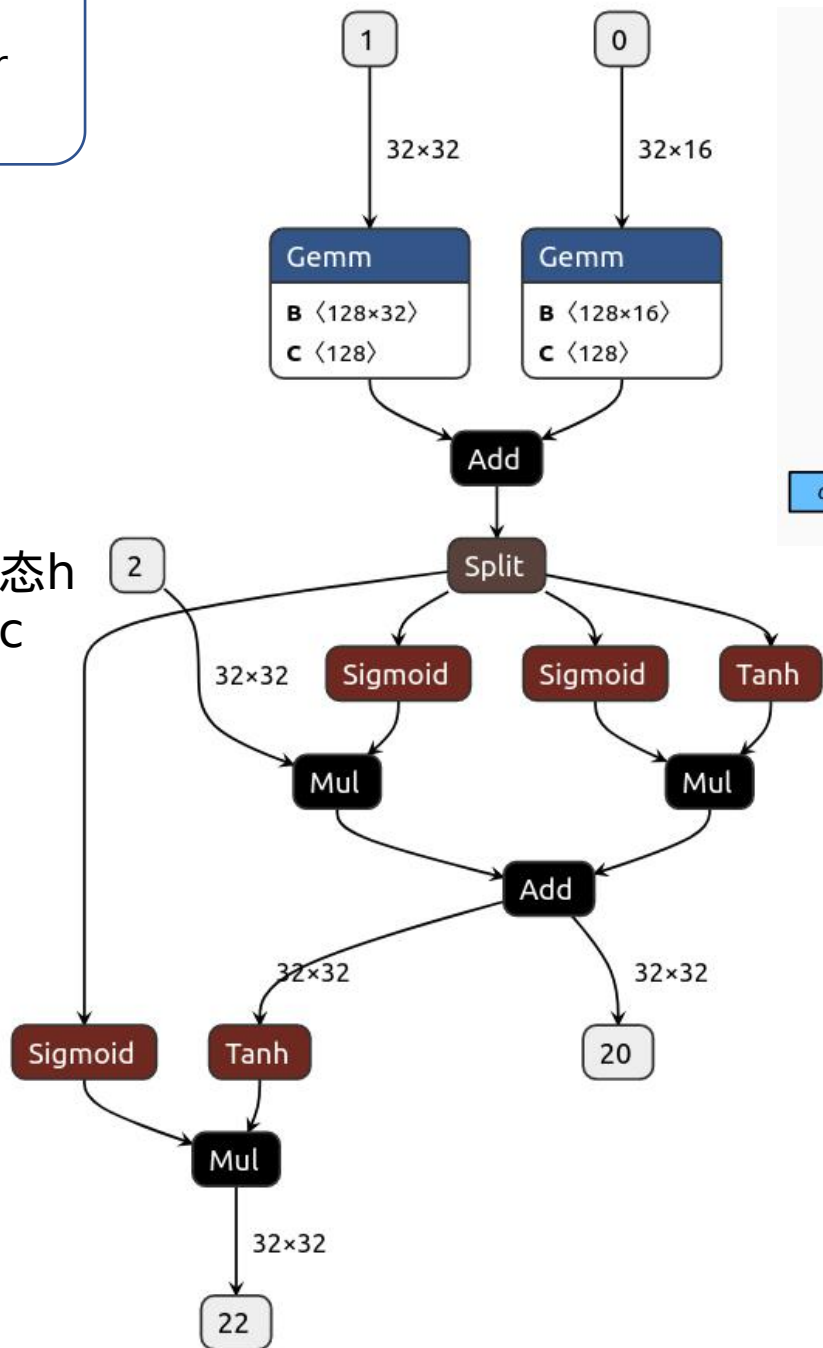
86-10-14



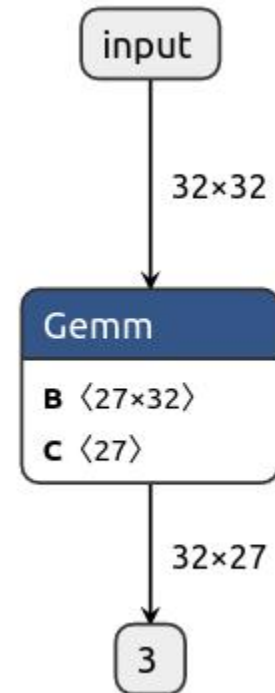
将样本每个字符作为 x_i 依次输入压缩器LSTM，获取的最后一个记忆和最后一个隐状态作为初始记忆和初始隐状态输入解压器

解压器decoder

0 : 词向量
1&22 : 隐状态h
2&20 : 记忆c



Split-->Sigmoid-->Mul (右) 遗忘门
Split-->Sigmoid-->Mul (左) 输入门
Split-->Tanh-->Mul 候选记忆
Split-->Sigmoid-->Mul-->22 输出



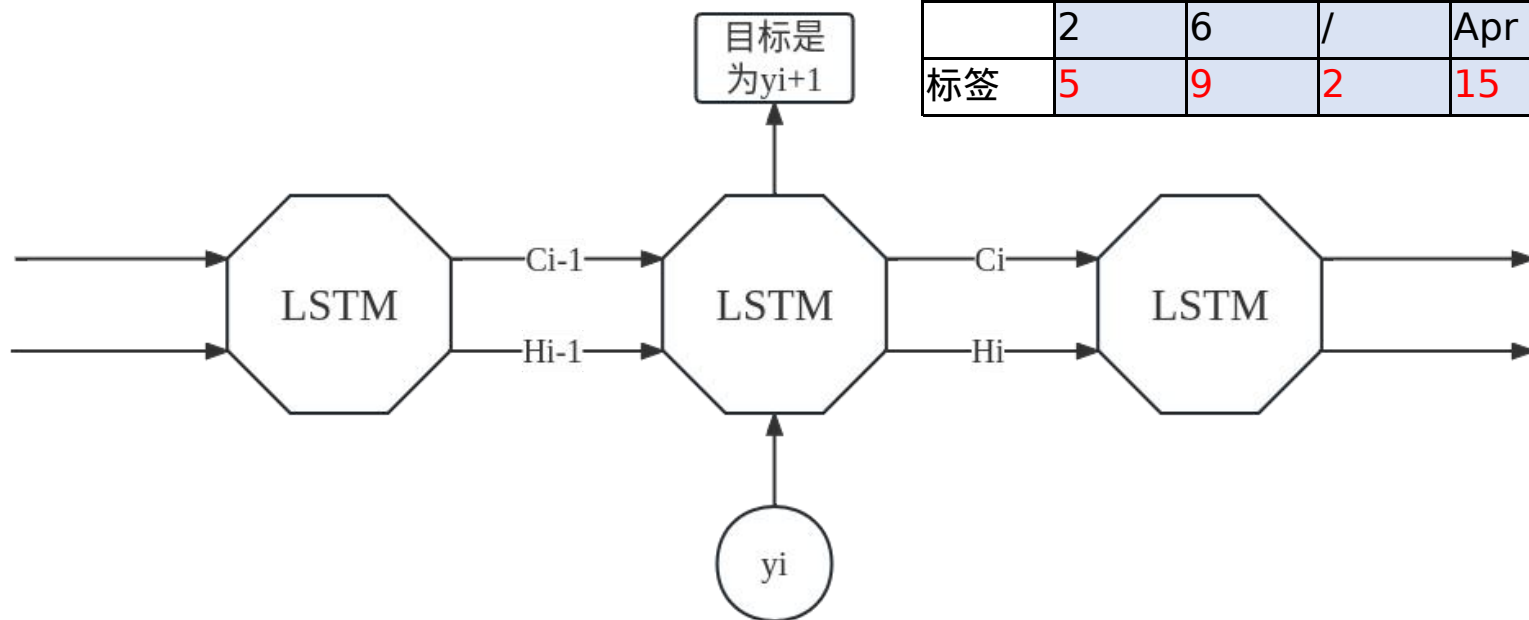
将隐状态转为
概率分布

解压器decoder

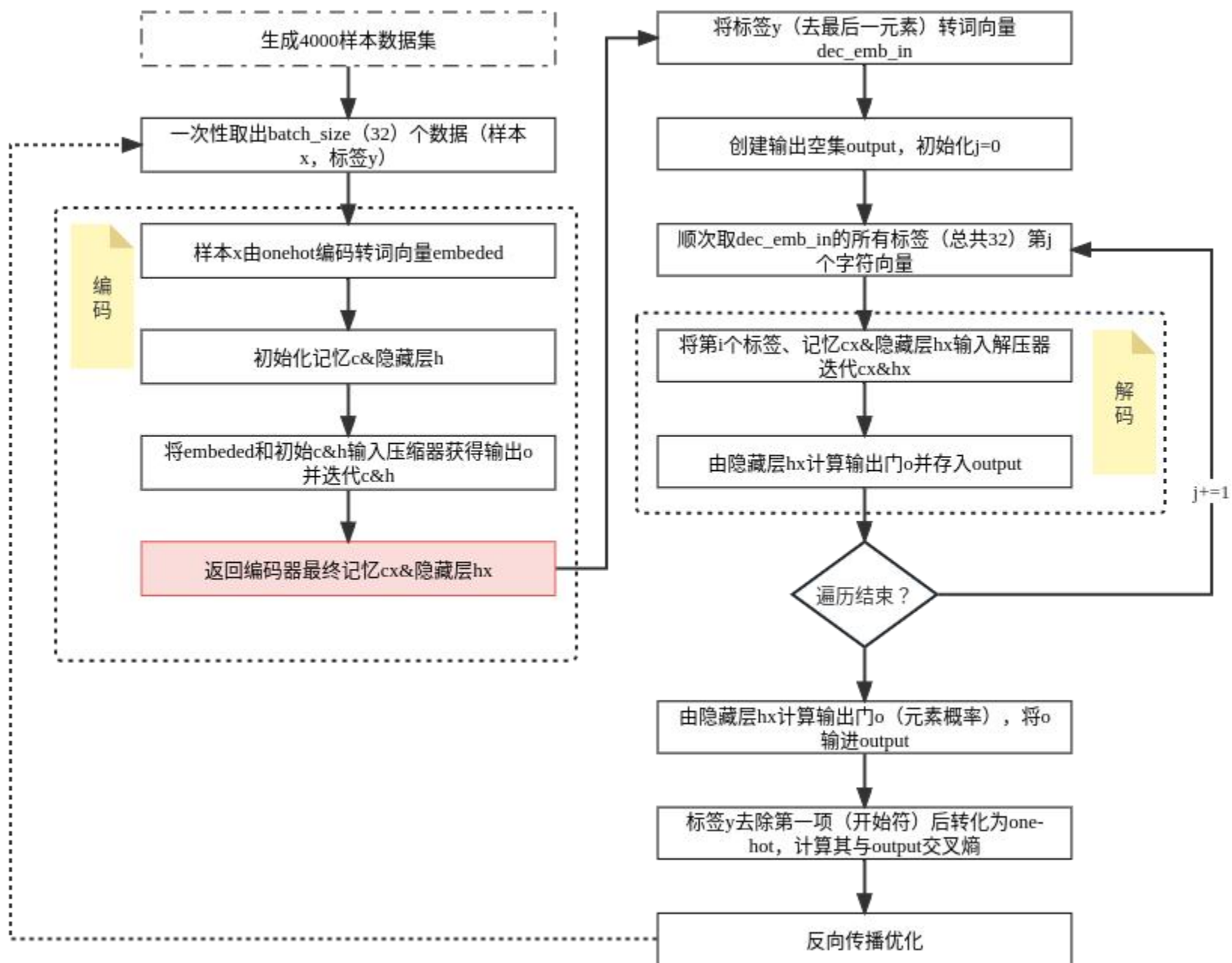
解压器输入为样本第*i*个字符，输出为样本*i+1*个字符在元素集合的概率分布

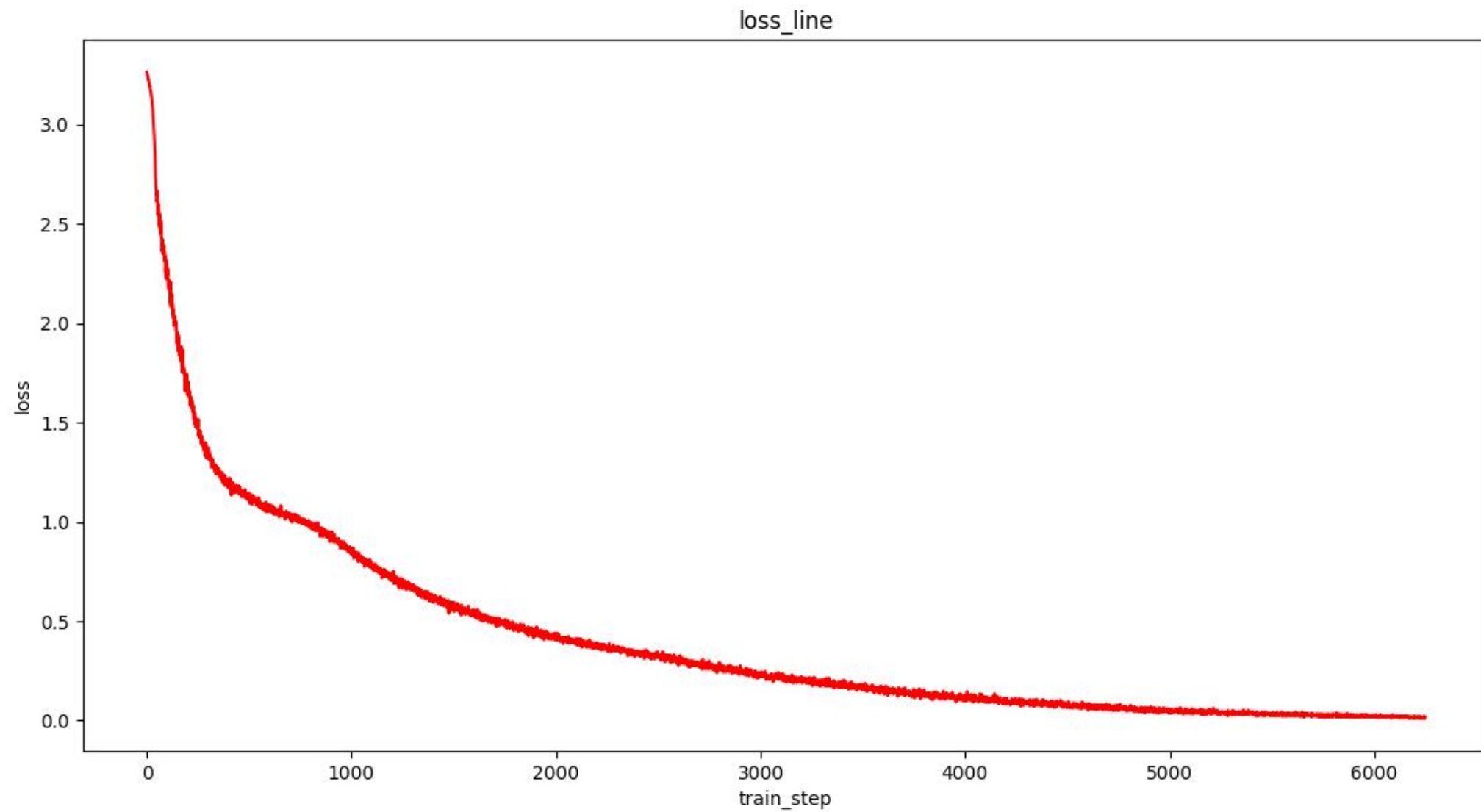
	1	2	3	4	5	6	7	8	9	10	11
	3	1	-	0	4	-	2	6			
样本	6	4	1	3	7	1	5	9			
	GO	2	6	/	Apr	/	2	0	1	3	EOS
标签	14	5	9	2	15	2	5	3	6	4	13

	1	2	3	4	5	6	7	8	9	10	11
	GO	2	6	/	Apr	/	2	0	1	3	EOS
样本	14	5	9	2	15	2	5	3	6	4	13
	2	6	/	Apr	/	2	0	1	3	EOS	
标签	5	9	2	15	2	5	3	6	4	13	



解压器训练时输入为标签
使用时输入为起始符GO和上一步解压器的输出



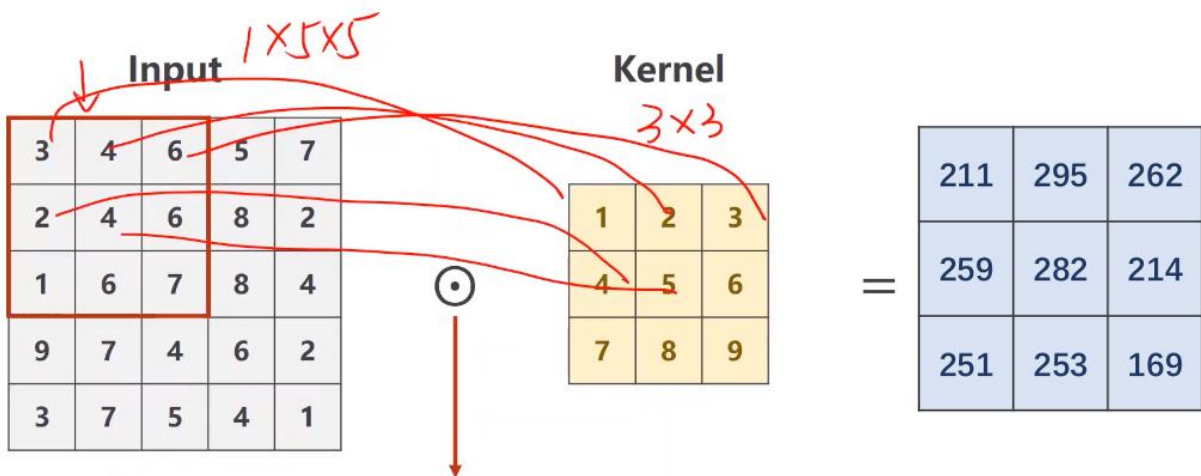




2023.03.15



卷积神经网络

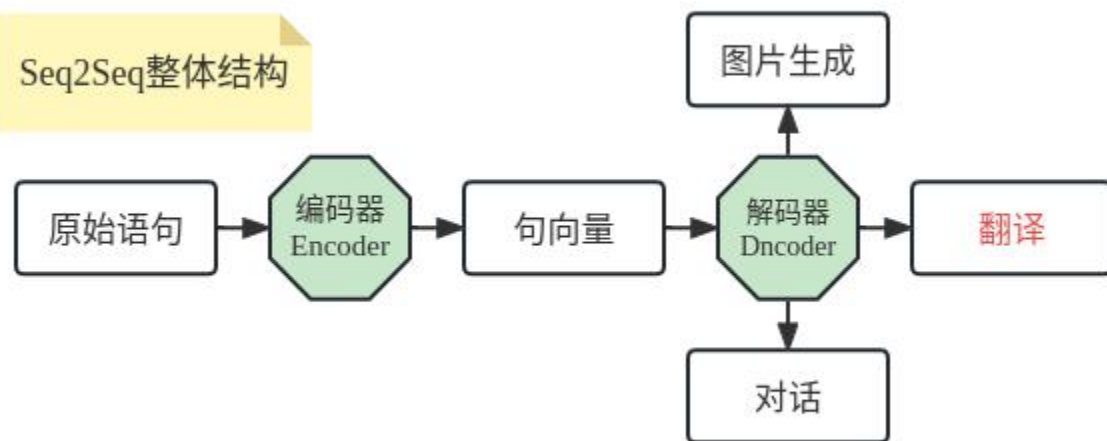


$$3 \cdot 1 + 4 \cdot 2 + 6 \cdot 3 + 2 \cdot 4 + 4 \cdot 5 + 6 \cdot 6 + 1 \cdot 7 + 6 \cdot 8 + 7 \cdot 9 = 3 + 8 + 18 + 8 + 20 + 36 + 7 + 48 + 63 = 211$$

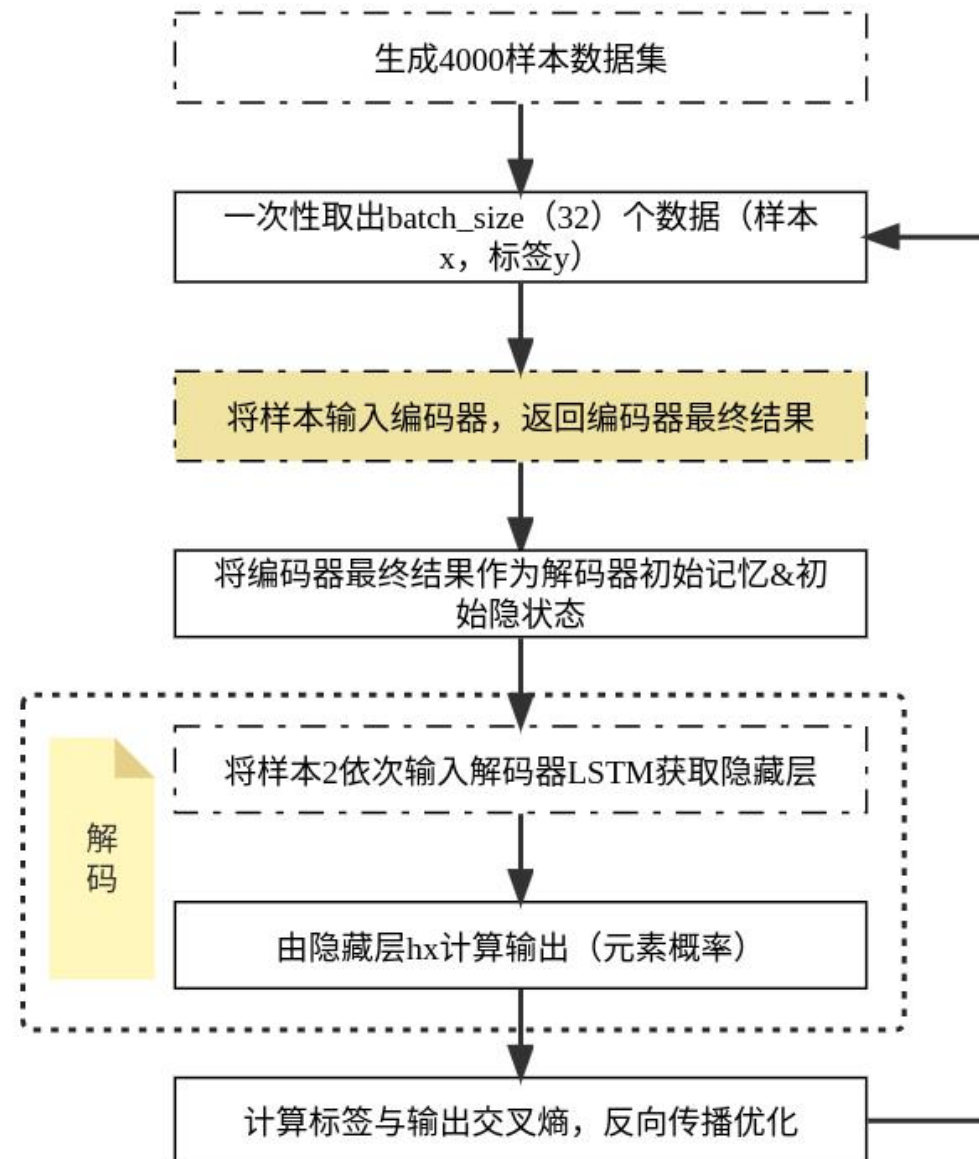
$$4 \cdot 1 + 6 \cdot 2 + 5 \cdot 3 + 4 \cdot 4 + 6 \cdot 5 + 8 \cdot 6 + 6 \cdot 7 + 7 \cdot 8 + 8 \cdot 9 = 4 + 12 + 15 + 16 + 30 + 48 + 42 + 56 + 72 = 295$$



Seq2Seq整体结构



本代码与上一代代码功能相同，结构相同
编码器结果仍然作为初始记忆和初始隐状态输入解码器
解码器没有变化
不同之处是编码器主要内容为CNN



CNN压缩器encoder

86-10-14

{tensor:(32,1,8,16)}

	字符1		字符8
16维 词向 量		...	
	...	...	...
		...	

卷积器：nn.Conv2d(1,16,(3,16))
输入通道1,输出通道16,卷积核大小为3×16

{tensor:(32,16,6,1)}

样本x由onehot编码转词向量embeded：
tensor(32,8,16)

升高维度
tensor(32,1,8,16)

分别用卷积核2,3,4*16卷积并relu，输入1通道，输出16通道得到co
co = {list : 3}
1 : {tensor:(32,16,7,1)}
2 : {tensor:(32,16,6,1)}
3 : {tensor:(32,16,5,1)}

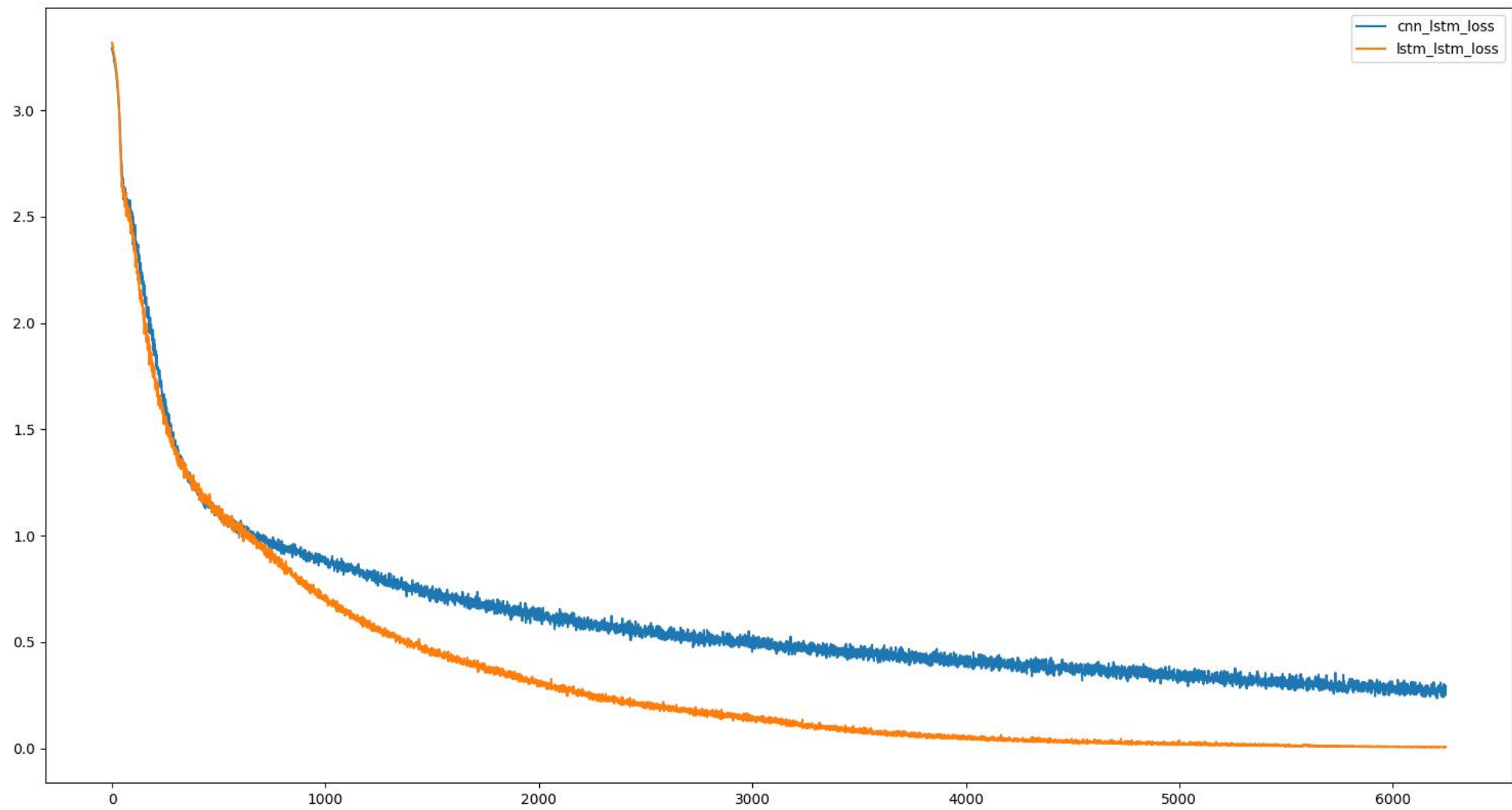
分别用窗口7,6,5*1进行Max池化co，结果均为tensor:(32,16,1,1)

将结果压缩两个维度并拼接为o：
tensor(32,48)

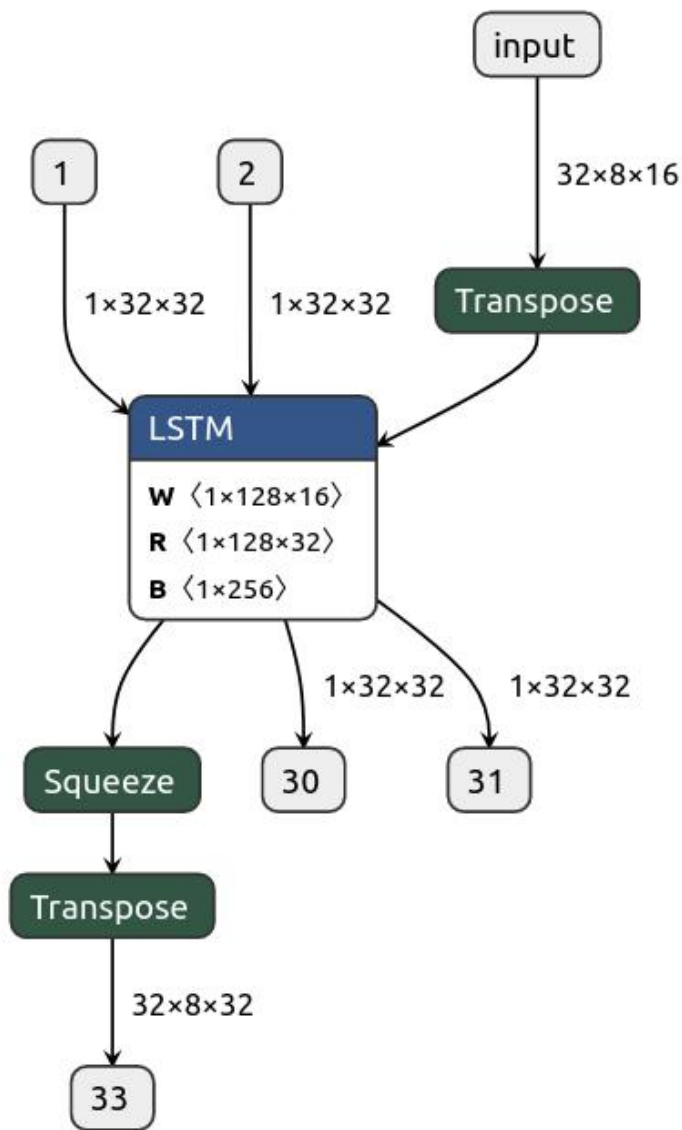
将o输入线性全连接层，输出h：
tensor(32,32)

输出h作为解码器初始记忆和初始隐藏层

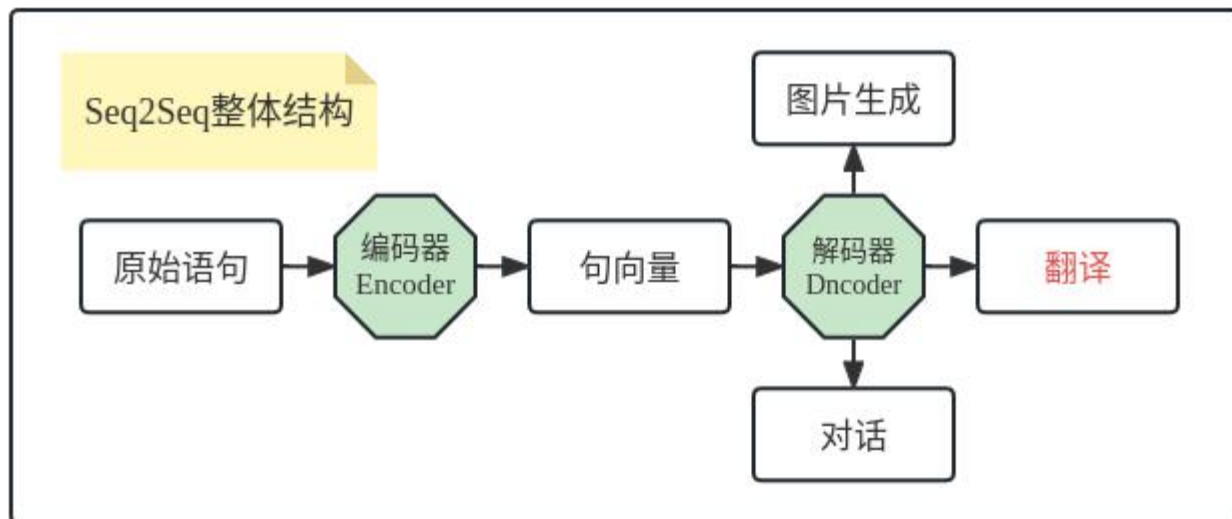
CNN局限性：要求有个句子最长的限制，句子如果超过这个长度，一般只能截断根据CNN编码器与LSTM编码器的对比，可以看出相同训练步，CNN编码器loss略大



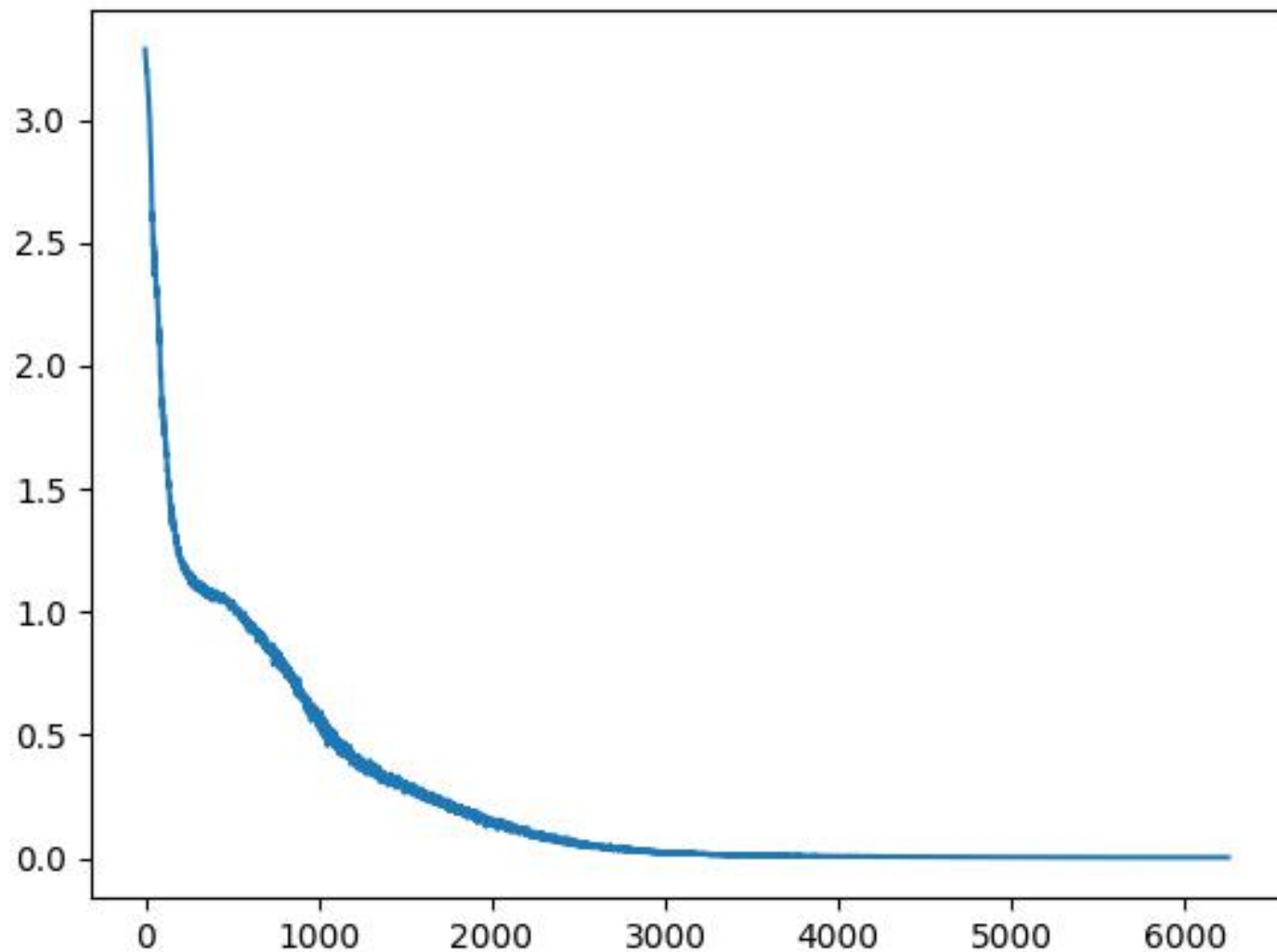
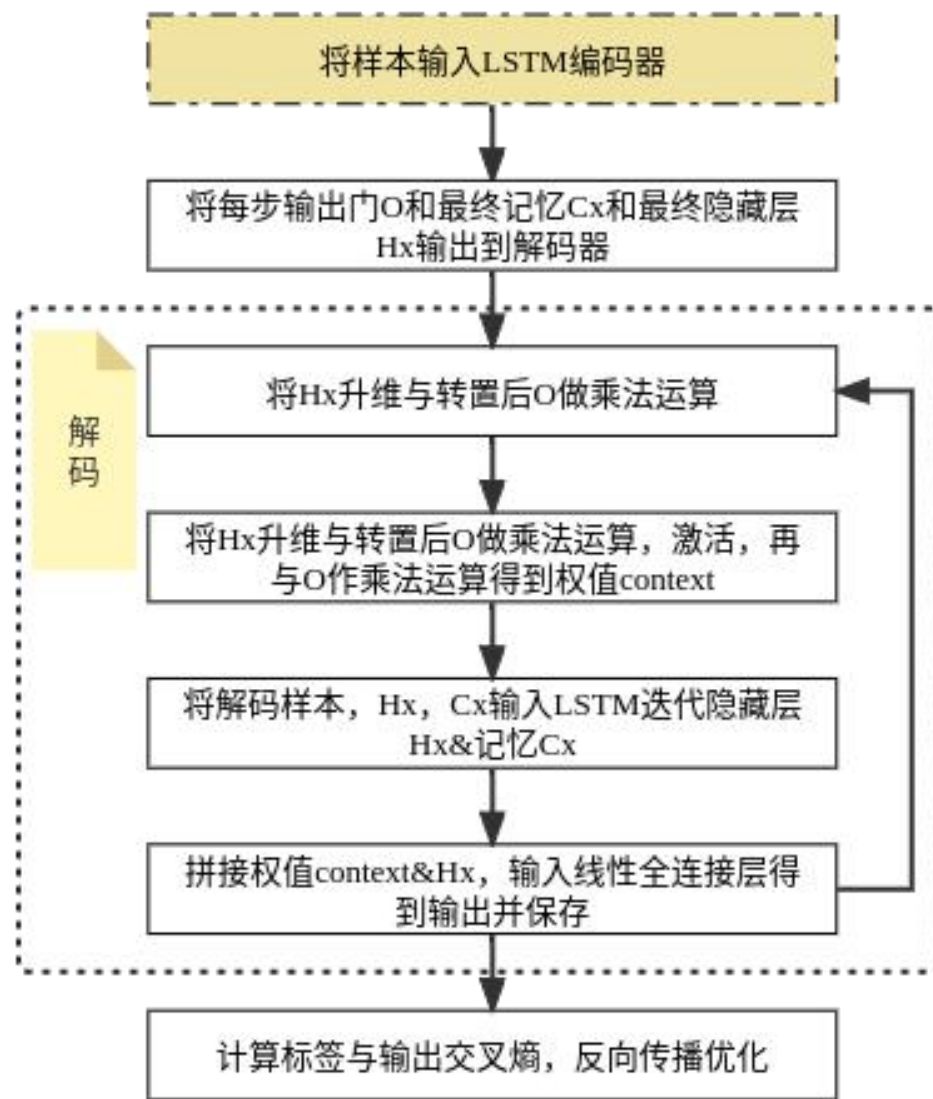
attention解码器
encoder



本代码与上一代代码功能相同，主体结构相同
LSTM编码器每步输出（33）也保存并输出到解码器
编码器内部结构相同，编码器最终隐藏层和记忆仍然
作为初始记忆和初始隐状态输入解码器
解码器引入attention



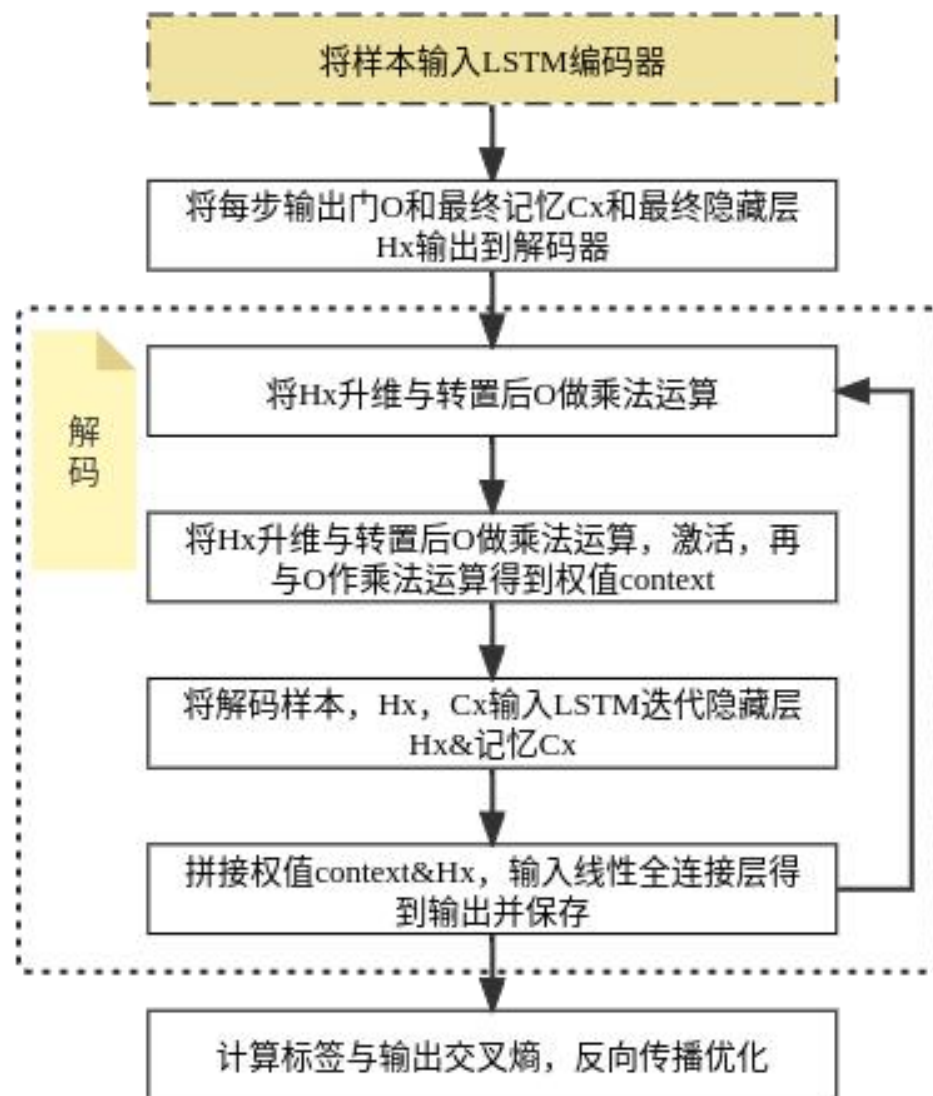
中间状态（句向量）来自于输入网络最后的隐藏层，一般来说它是一个大小固定的向量，能储存的信息就是有限的（当前瓶颈）



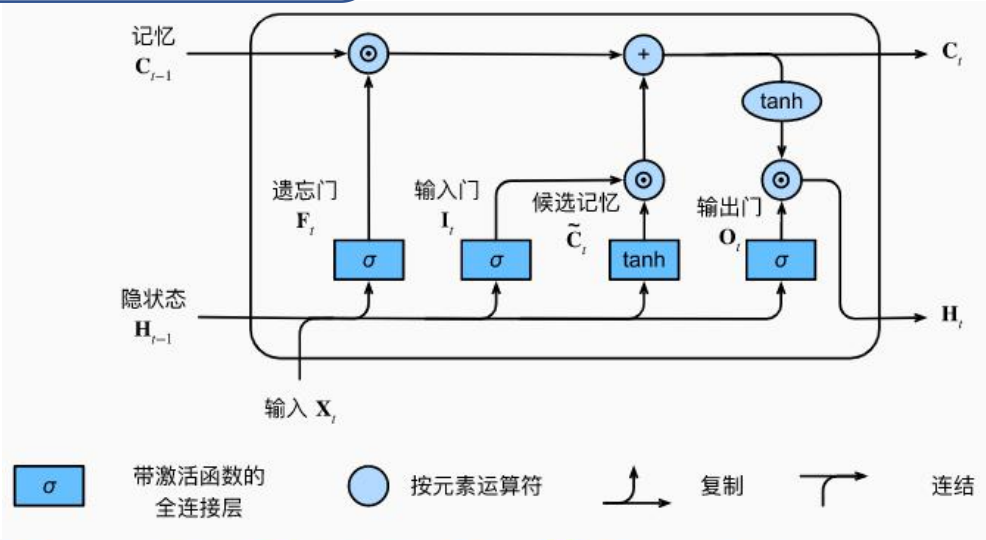


2023.03.20





attention原理



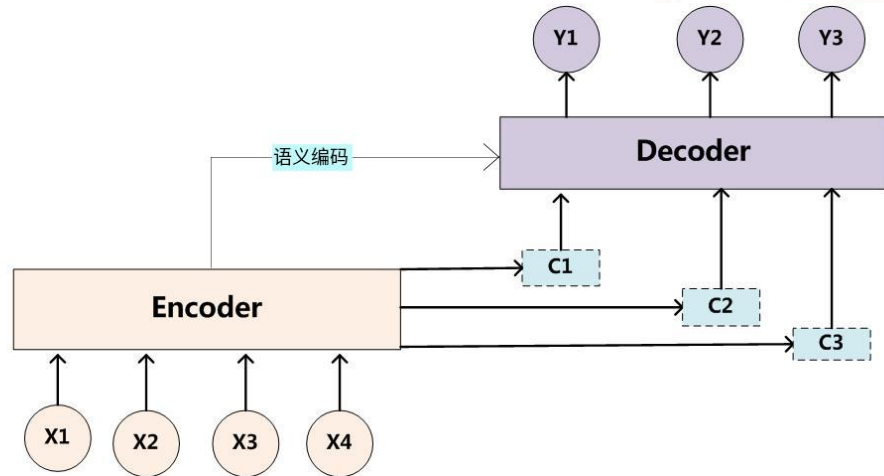
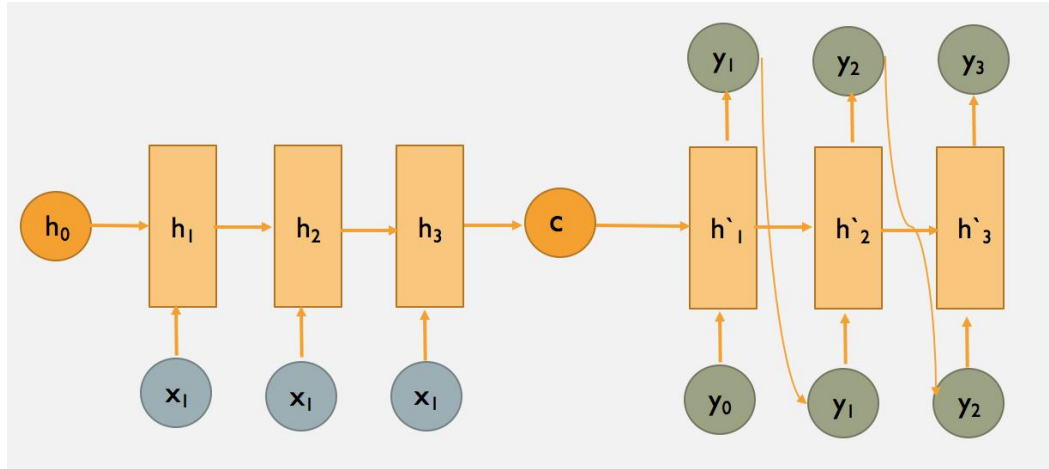
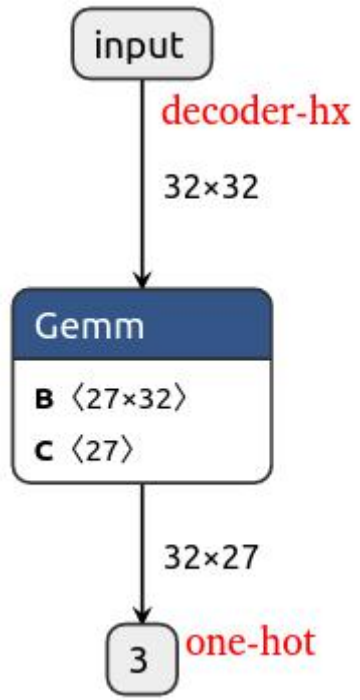
$$I_t = \sigma(X_t W_{xi} + H_{t-1} W_{hi} + b_i),$$
$$F_t = \sigma(X_t W_{xf} + H_{t-1} W_{hf} + b_f),$$
$$O_t = \sigma(X_t W_{xo} + H_{t-1} W_{ho} + b_o),$$

$$C_t = F_t \odot C_{t-1} + I_t \odot \tilde{C}_t.$$

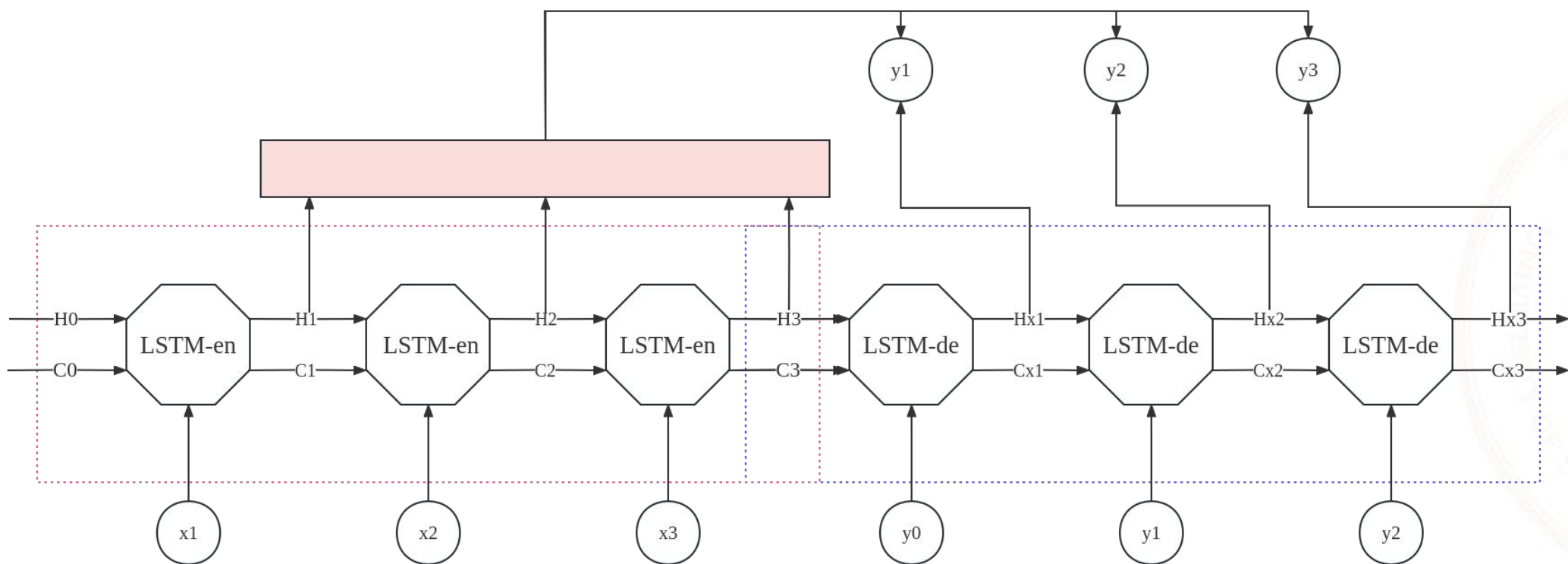
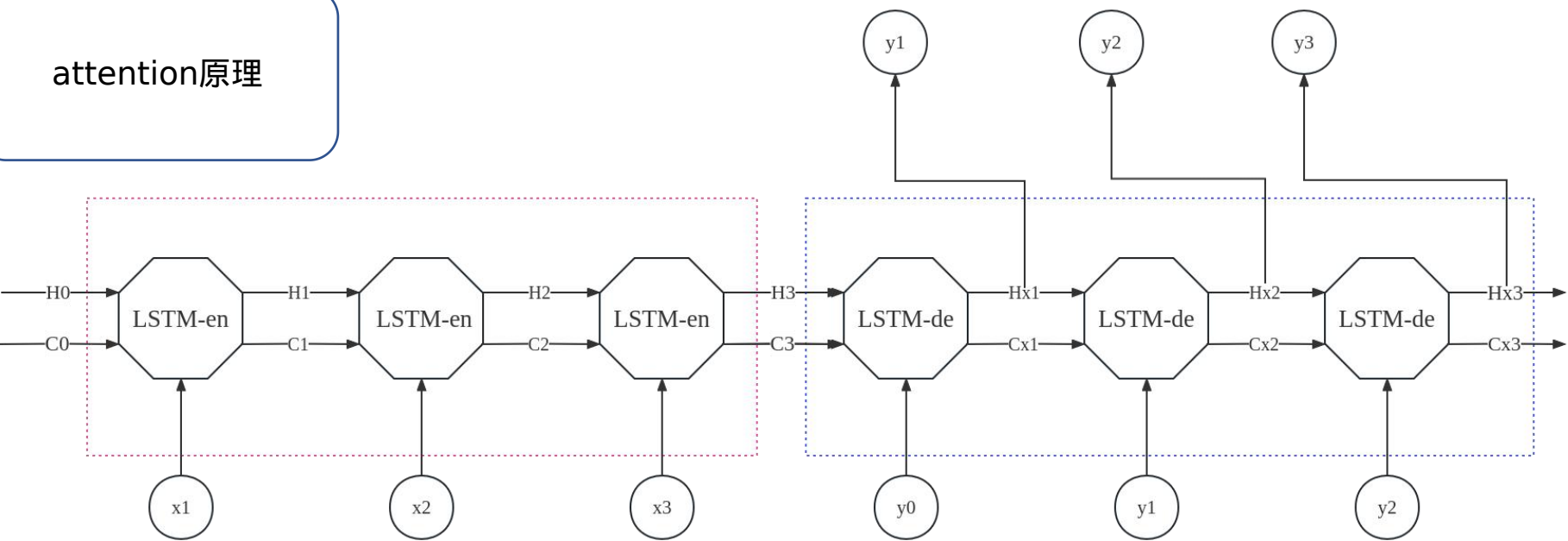
$$\tilde{C}_t = \tanh(X_t W_{xc} + H_{t-1} W_{hc} + b_c)$$

$$H_t = O_t \odot \tanh(C_t)$$

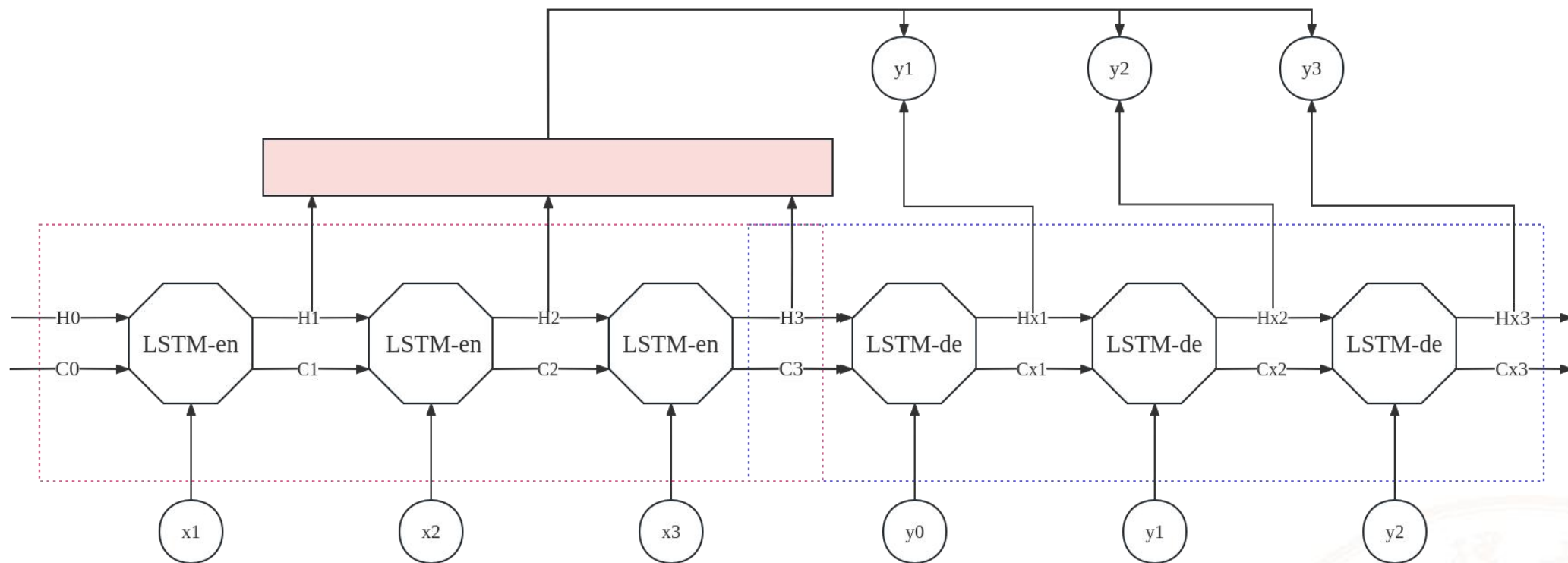
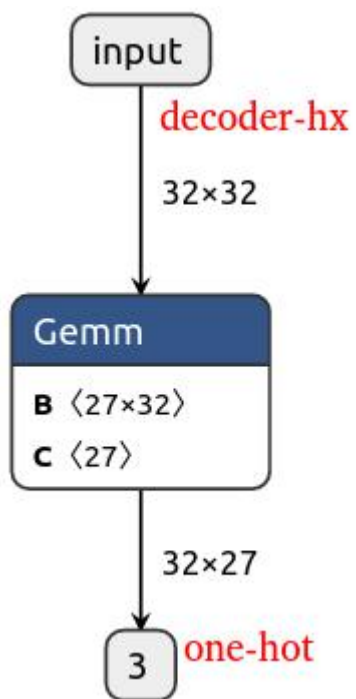
LSTM

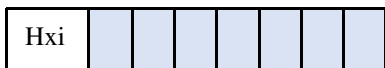
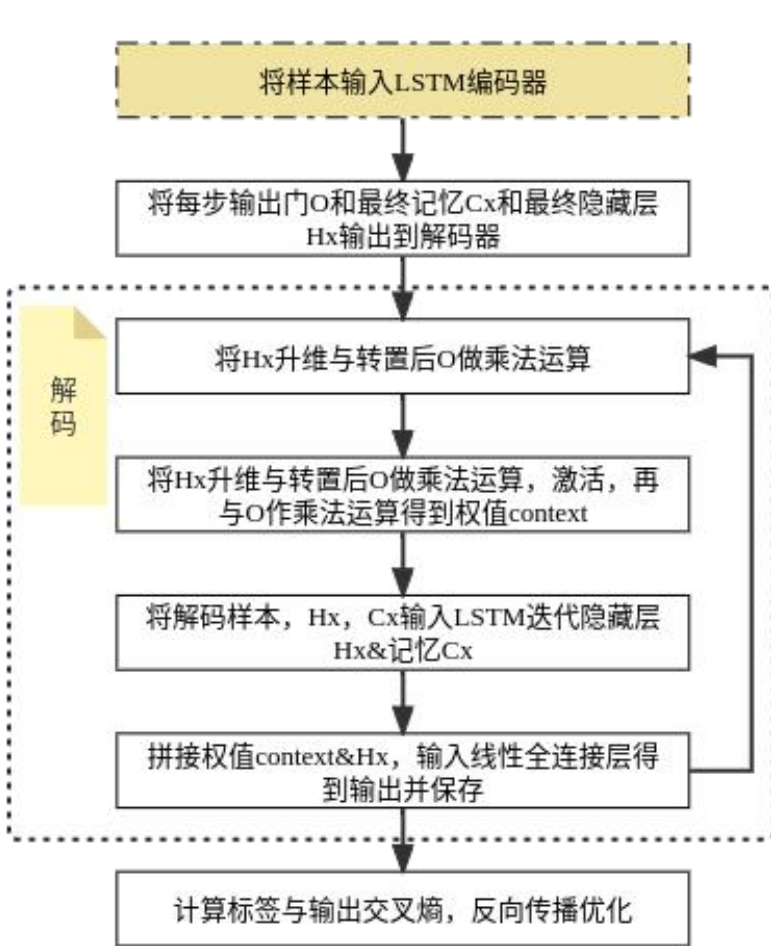


attention原理



LSTM





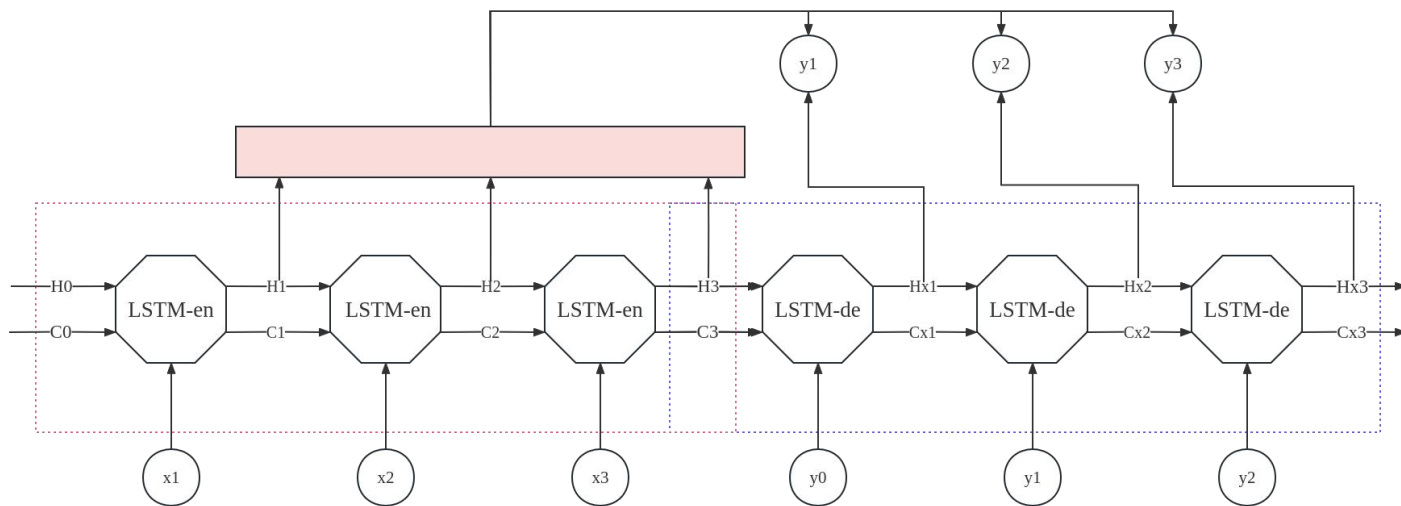
O		
H1	H2	H3



et		
et1	et2	et3

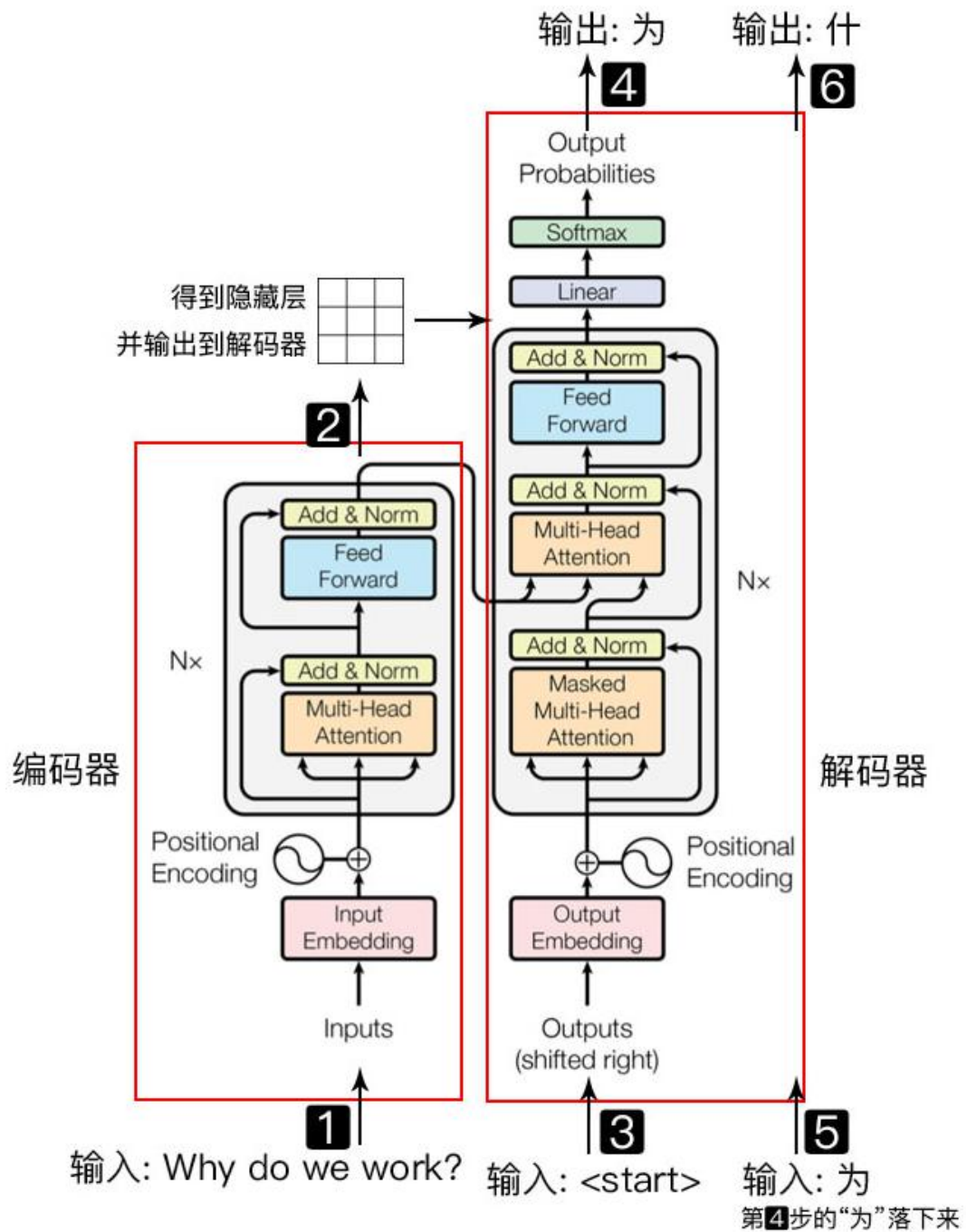


αt		
$\alpha t1$	$\alpha t2$	$\alpha t3$



α

transformer整体
=encoder+decoder



encoder

从自然语言序列
经过计算
到隐藏层的过程

